CEG

The CIRIS Epistemic Grammar

Version 1.0-RC1 — Reader Edition

a signed, compositional graph language for claims, relationships, authority, membership, consent, governance, addressing, and settlement across a decentralized network

2026-06-10

This is the human-reading edition: version history, per-path narrative, and provenance cross-references are omitted for readability. The canonical working draft (with full lineage) is the markdown source and `ceg-1.0-rc1.pdf`*. Conformance is judged against the normative wire surface only (§0.1.1).*

Contents

CEG — The CIRIS Epistemic Grammar

Version: 1.0-RC1 (Release Candidate — wire surface FROZEN)

Status: Release Candidate (2026-06-10). **The wire surface is frozen at this cut**: implementers SHOULD pin RC1; any change between RC1 and 1.0 is a **found defect**, not design — the design surface is complete (fifteen §1.4 paths; no open design questions remain; #57 blocker A closed at this cut via the §5.2.1 JCS redesign). **One scheduled addition stands between RC1 and 1.0**: the #70 operational-data subject_kinds (`organization` / `org_membership` / `partner_record`, the Spock-removal Phase 2 envelopes) land **additively** after federation review — new subject_kinds, no change to anything frozen here. The **1.0 label** itself waits on the #57 freeze gate: streaming substrate shipped (CIRISPersist#142), Spock removed (#58), conformance artifacts (OpenAPI + `dimensions.json`) generated from these frozen shapes, and **one full implementation cycle across Persist + NodeCore + Lens + Agent + Edge with no new wire-break**. That silence is the freeze signal; 1.0 is these bytes re-tagged.

> ***Human-reading edition:*** *`pdf/ceg-1.0-rc1-reader.pdf` is the de-editorialized, deduplicated, human-first cut (version history, per-path narrative, and provenance cross-refs stripped). This source tree remains the canonical working draft and retains that lineage.*

1.0-RC1 — the freeze cut (per #71 backwards-pass + CIRISVerify#64 + #57 blocker A): five honesty/safety patches + two pins + the canonical-bytes resolution, then freeze. **(C1)** §10.5.3 **removal coalescing** (all removals in one STH window T=2s batch into ONE rotation — caps the cascade at 1/T regardless of churn; the original model's 3,600/hr churn understated realistic broadcast churn ~2 orders: naive 30%/hr at N=1M ≈ 3.1 Tbps > the 2 Tbps content fan-out; coalesced ≈ 18.7 Gbps ≈ 0.9%) + the **public-broadcast exemption** (ungated `listed:public` rosters: removal forces no rotation — no confidentiality claim against departed viewers). **(C2)** §5.6.8.8.2 forward-secrecy honesty: KEM rotation bounds *future* exposure only; historical-content recovery requires DEK rotation + re-encryption, which is NOT mandated — named gap. **(C3)** §10.1.4 fail-secure exclusion is no longer silent: new §7.7 `hard_case:recipient_excluded:{scope}` emitted INTO the affected cohort (never federates beyond). **(C4)** §5.6.8.8.2 admission MUST-reject content-KEM x25519 == transport x25519. **(C5)** §9.1 names the two-holders-one-household correlated-failure geometry; authority is the full constitutional kill; the mitigant is **finding new holders** (active obligation). **Pins**: §0.9.2.1 base64 = RFC 4648 STANDARD padded (protocol-layer pin; CIRISVerify#64); ML-KEM-768 pubkey = exactly 1184 B; `encryption_pubkeys` joins the §8.1.12.7.1 member set (c); §5.6.8.4 crypto-agility headroom note (v3/ML-KEM-1024 additive). **Blocker A closed**: §5.2.1 provenance contracts redesigned to **JCS (RFC 8785) objects with pinned `domain` members — **TupleHash128 retired (one canonicalization family); v1 newline forms deprecated-historical; §9.2.1 accord invocation intentionally unmigrated (closed-vocabulary preimage).** No 1+4 change.**

0.18 — recipient encryption-key registration: the §10.1.4 substrate-wraps DEK cascade (the 0.17 community + self/family tiers all rely on it) needs each recipient's **content-encryption** keys (x25519 + ML-KEM-768) — but the federation directory carries only **signing** keys (Ed25519 + ML-DSA-65), and ML-KEM is not derivable from ML-DSA. 0.18 adds the optional **`encryption_pubkeys` field-set on `identity_occurrence`** (§5.6.8.8.2), structurally parallel to `transport_destination`: self-certified (`attesting_key_id == identity_key_id`), hybrid-signed, `supersedes`-rotatable (rotate a KEM key without touching the signing identity), and **already cross-region-replicated**

inside the occurrence envelope (`EnvelopeKind::IdentityOccurrence`, CIRISEdge#65) — so **no new replication kind, no Edge wire change**. These feed `wrap_algorithm: v2` directly (§5.6.8.4). Adds the **fail-secure-exclusion** rule (§10.1.4): a recipient whose occurrence carries no valid ML-KEM key is *excluded* from the grant (content stays encrypted) — never a plaintext or v1 fallback. **Key separation (normative):** the content-KEM x25519 is distinct from the signing keys AND from the Reticulum transport x25519 — three purposes, three keypairs, never reused. **No 1+4 change** — one optional field-set on an existing subject_kind (§1.4 fifteenth path).

0.17 — the three crypto tiers (per CIRISRegistry#67; the line is *"does it have a bounded membership roster?"*): the ≤0.16 binary cut (self/family encrypt; everything else plaintext) is replaced by **self/family** (encrypted + structurally invisible) → **Community** (`community` / `affiliations`: encrypted under a **per-community DEK** + `holds_bytes:*` with cleartext provenance — byte-confidential to members, discoverable by provenance) → **Commons** (`species` / `biosphere` / `federation`: plaintext, anyone inspects). The community DEK *is* the §10.5.3 epoch-DEK cascade (one shared DEK, O(1)/emission, v2/PQC mandatory) — *"a community is a stream its members subscribe to, cryptographically"* — refuting the 0.8 "infeasible" premise. **Mandatory, not opt-in** (the tier name is the guarantee; a persecuted community is protected by *being* one). Adds the **holder-inspectability principle** (§8.1.13.3: nothing above local tier is a forced unattributable opaque blob) + two rulings: `cohort_subkind: infrastructure` communities (`ciris-canonical`/governance) are the **plaintext Commons exception** (the trust root must be inspectable), and node→canonical conformance traces are `cohort_scope: federation` (world-readable). Overturns §8.1.13.4 (community now has the removed-member problem → Option-A re-key). **No 1+4 change** — the community DEK is the existing epoch cascade applied to `cohort_scope: community`.

How to read this spec without Cartesian default

Before the TOC: one load-bearing reading instruction that prevents the most common misread.

Self is self, fractally. At every scale the substrate operates on — key, occurrence, agent, fleet, cell, federation, biosphere — "self" means *the already-relationally-constituted entity speaking at that scale*. The cross-attestations that constituted the entity are upstream of the moment it speaks; when it then emits a self-attestation (`witness_relation: self`, declares its own `confidence`, reports its own `occurrence_id`, names its own `hardware_class`), that is **the relational composition speaking AS itself**, not a Cartesian atom asserting pre-relational identity.

Concretely:

- A `federation_keys` row is not an atomic principal pre-existing the federation that the substrate observes. It is a relational locus already constituted by the cross-attestations of stewards, prior peers, and provisioning ceremony. When the key-bearing entity then signs an attestation, the signature IS that relational composition speaking at this scale.
- An occurrence within a fleet is not a pre-relational unit reporting its own state. It is constituted within its fleet (the fleet's `AGENT_OCCURRENCE_ID` partitioning, the prior cross-attestations naming this fleet's deployment, the substrate's witness of its emissions). When it reports `occurrence_id`, that is the fleet-constituted occurrence speaking.
- The federation itself is not prior to its constituents. It IS the relational composition of cells, agents, occurrences, and keys cross-attesting one another.

This is why the wire format admits self-attestation **everywhere it admits attestation at all**: because there is no pre-relational atomic entity available to do the admitting in the first place. Every `attesting_key_id` is already a relational locus. Self is the relational composition speaking from within itself, at the scale of the speaker.

Common misread to avoid. A reader importing Cartesian-individualist defaults from PGP / X.509 / atomic-principal cryptographic identity work will see `witness_relation: self` and think "this is a single atom asserting pre-relational identity; surely the substrate should require cross-attestation before admitting." That reading is the misread. The substrate cannot require cross-attestation as a precondition for self-attestation because the cross-attestations are what constitute the self in the first place — they are upstream of the self-attestation, not downstream gates on it. The fractal nesting is the point.

The Recursive Golden Rule (§1.5) is the structural expression of this: no scale gets to exempt itself from the relational constitution that defines it. CIRIS L3C is constituted by the same cross-attestations that constitute every other federation participant; the federation itself is constituted by its participants; humanity-as-such (§9) is the one scale outside the federation's participant set, by design.

If you find yourself thinking "the spec should add a cross-attestation gate before admitting this self-attestation" — pause. Cross-attestation already happened upstream; the self-attestation is its downstream voice. Verify by reading §1.2 commitment 1, then continue.

Table of contents

§	File	What it covers
7	`07_reserved.md`	Reserved-prefix enforcement (`accord:*`, `system:*`, co-owned, detector-only, capacity self-emission)
8	`08_composition.md`	Composition policies (Policies A–G); aggregation semantics; Frickerian discipline; Sovereign-Registered equivalence
9	`09_humanity_accord.md`	HUMANITY_ACCORD constitutional layer — accord-holder triple, authority scope, hardware-class taxonomy
10	`10_endpoints.md`	Endpoint shapes — transport substrate, steward/accord discovery, STH cosigning + witness directory
11	`11_governance.md`	Governance discipline — operational-language gate at admission; amendment process (WA quorum + 1-of-6); bootstrap-content pattern
12	`12_translation.md`	Translation discipline — five families; verdict categories; not-translated taxonomy; decision tree
13	`13_anti_patterns.md`	Anti-patterns — rejected wire additions; CEG 0.1 rejections from #30 stress test
14	`14_glossaries.md`	Glossaries — Persist + Edge `system:*` leaves; envelope-reach table
15	`15_gaps.md`	Concerns + acknowledged gaps — closed gaps; acknowledged risks; first-adopter exposures; deferrals
16	`16_references.md`	References + lineage — version history; companion documents; sibling MISSIONs; external references
17	`17_cadence.md`	Update cadence — when CEG is updated

Two readerships

- **Implementers** of federation primitives consuming or emitting CEG attestations: read §0 + §1 + §3 + §4 + §5 + §7 + §8 + §9 + §10 normative.
- **Translators** mapping substantive content into CEG envelopes: read §1 + §12 + §13 + §14 + the `LANGUAGE_PRIMER.md` companion.

Both readerships should read §1 first.

§0 Foreword

CEG — the CIRIS Epistemic Grammar — is the federation's language for making **structured, signed, machine-checkable claims about reality and each other**. It is the wire format the federation's peers speak.

The grammar has exactly five wire-format primitives (one workhorse + four structural composers) and an open-vocabulary dimension namespace organized by mechanism-descriptive prefixes. Consumers compose verdicts from primitive attestations using the policies in §8; nothing in the wire format prescribes what verdict to reach.

CEG is **substrate-consuming**: it sits above the federation substrate (CIRISPersist for storage, CIRISVerify for crypto, CIRISEdge for transport) and below the application tier (CIRISAgent). It does not author primitives in the substrate it consumes; it composes policy over them. It is also **substrate-supplying** for the second-tier consensus crates (CIRISNodeCore for consensus, CIRISLensCore for detection) — they own slices of the dimension namespace and emit attestations that other CEG consumers read.

This specification has **two readerships**:

- **Implementers** of federation primitives consuming or emitting CEG attestations: read §1-§11 normative.
- **Translators** mapping substantive content into CEG envelopes: read §12-§14 + the `LANGUAGE_PRIMER.md` companion.

Both readerships should read §1 first.

§0.1 Conformance language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **NOT RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in BCP 14 (RFC 2119 + RFC 8174) when, and only when, they appear in all capitals, as shown here.

§0.1.1 Normative vs informative content (what interop requires)

A reader may **agree with the protocol and disagree with its philosophy** and still build a fully conforming implementation. The two are deliberately separable:

- **Normative (binding for interoperability):** the wire format and its conformance surface — the §3 structural primitives; the §4 envelope fields; the §5 namespace + `subject_kind`s; the §0.5–0.9 canonicalization rules; the §6 relation precedence; the §7 reserved-prefix rules; the §8 composition policies; the §10 endpoint shapes; and every RFC-2119-keyworded statement. This is exactly the surface enumerated in §1.4 ("report the surface beside the invariant"). **Conformance is judged against this and nothing else.**
- **Informative (explanatory framing; NOT binding):** the motivating philosophy and rationale — notably §1.2 (the Ubuntu / relational-anthropology substrate), the cross-tradition readings, and prose written as motivation rather than requirement. These explain *why* the normative choices were made; they add **no** conformance obligations. An implementer who rejects the anthropology but emits/consumes wire-correct Contributions is conforming.

Where framing produced a concrete wire consequence, that consequence is restated as a normative rule in its own section (e.g., structural invisibility is motivated informatively but enforced normatively at §10.1.4, bounded by §1.6). When in doubt, the RFC-2119 keywords and the §1.4 surface govern; informative prose never overrides them.

§0.2 Conformance levels

A **Producer** is any peer that emits Contributions onto the federation wire. A **Consumer** is any peer that reads and composes verdicts over received Contributions. A **Substrate Implementation** is the storage + transport + crypto layer (CIRISPersist + CIRISEdge + CIRISVerify) underneath both.

Three normative conformance profiles:

1. **CEG-Conforming Producer (CCP)** — emits well-formed envelopes per §4, signs per §0.4 References [hybrid-sig], respects reserved-prefix rules per §7, declares its `oversight_mode` and `witness_relation` per §4.
2. **CEG-Conforming Consumer (CCC)** — verifies hybrid signatures, enforces reserved-prefix rules at admission, implements at least Policy A (§8.1.1) with the default aggregation rules from §8.2, MUST honor `null` placeholder/dev hardware-class rejection per §9.4.
3. **CEG-Conforming Substrate (CCS)** — implements the storage + transport guarantees referenced in §10.1 + §10.3, including idempotent replication, full-SHA blob verification before consumption (§10.1), and witness-quorum multi-party admission per §10.3.

Sections that follow MAY add per-feature conformance subsections; the three profiles above are the minimums.

§0.3 Versioning policy

CEG follows **SemVer 2.0.0** with these mapping rules:

- **MAJOR (X.0.0)** — any wire-incompatible change: removal of an envelope field, change of a field's semantic, removal of a structural primitive, change to canonical-bytes domain-separation labels, removal or breaking-redefinition of a §5 prefix, change to a §7 reservation, or change to the §0.1 / §0.2 conformance language.
- **MINOR (0.X.0)** — wire-compatible additions: new prefix in §5, new envelope field with documented default, new composition policy in §8, new endpoint shape in §10, new optional conformance subsection. Existing Conforming Producers and Consumers continue to interoperate without modification.
- **PATCH (0.0.X)** — clarifications, editorial fixes, additions to non-normative sections (WITNESS_KIND_REGISTRY, glossaries §14), addition to §15 acknowledged-gaps, fixes to non-normative examples in §14.

The 0.x series indicates this specification is a Public Working Draft. Any 0.x → 0.(x+1) bump MAY include wire-breaking changes; consumers MUST treat 0.x as unstable until 1.0 publication. Once 1.0 is published, the rules above bind strictly.

A **deprecation** is announced by adding a `**DEPRECATED in 0.X**` marker to the affected element with a stated removal target (e.g., `removal: 1.2`). Deprecated elements MUST remain interoperable until the announced removal version. Removal in MAJOR or 0.MINOR per the rules above.

§0.4 Normative References

The following documents are normatively cited; implementations MUST conform to them where referenced inline.

Short name	**Normative document**
[BCP 14]	RFC 2119 + RFC 8174 — keywords for use in RFCs
[FIPS-180-4]	FIPS 180-4 — SHA-256 and the SHA-2 family
[FIPS-202]	FIPS 202 — SHA-3 / SHAKE128 / SHAKE256 / TupleHash
[FIPS-204]	FIPS 204 — ML-DSA (Module-Lattice-Based Digital Signature Algorithm); CEG uses parameter set ML-DSA-65
[RFC-3339]	RFC 3339 — Date and Time on the Internet, with fractional-seconds disambiguation in §0.5 below
[RFC-5905]	RFC 5905 — Network Time Protocol Version 4
[RFC-6962]	RFC 6962 — Certificate Transparency; this spec's transparency-log discipline tracks 6962 except where 6962-bis (RFC 9162) supersedes
[RFC-8032]	RFC 8032 — Edwards-Curve Digital Signature Algorithm (EdDSA); specifically Ed25519
[RFC-8174]	RFC 8174 — Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words (with BCP 14)
[RFC-8785]	RFC 8785 — JSON Canonicalization Scheme (JCS); used where this spec serializes JSON for signing
[RFC-9162]	RFC 9162 — Certificate Transparency v2.0 (CT-bis); MUST be used for new transparency-log integrations; older 6962 instances continue to interoperate
[ISO-639-1]	ISO 639-1:2002 — Codes for the representation of names of languages, two-letter
[BCP-47]	BCP 47 (RFC 5646) — Tags for identifying languages; for locale strings richer than ISO 639-1 alone
[RFC-9420]	RFC 9420 — Messaging Layer Security (MLS); the streaming epoch-key rekey (§10.5.3) conforms to MLS TreeKEM
[SFrame]	draft-ietf-sframe — Secure Frames; the per-frame AEAD chunk seal (§10.5.2) conforms to the SFrame model
[FIPS-203]	FIPS 203 — ML-KEM (Module-Lattice KEM); the post-quantum half of the hybrid KEM (ML-KEM-768)
[FIPS-204]	FIPS 204 — ML-DSA (Module-Lattice signatures); the post-quantum half of the hybrid signature (ML-DSA-65)
[RFC-9180]	RFC 9180 — Hybrid Public Key Encryption (HPKE); the `key_grant` wrap shape

Informational citations (Magnifica Humanitas, anthropological literature, Ubuntu philosophical literature, etc.) appear in §16.4 without normative force.

§0.5 Date-time canonicalization

Every ISO 8601 / RFC 3339 datetime in this specification MUST be:

- UTC (suffix: literal `Z`; the offset form `+00:00` MUST NOT be used)
- Millisecond-precision (exactly three digits of fractional seconds; trailing zeros required)
- Lowercase `z` MUST NOT be used; uppercase `Z` only

Canonical form: `YYYY-MM-DDTHH:MM:SS.sssZ`. Example: `2026-05-28T13:45:09.000Z`. Producers MUST emit this form; consumers MUST reject any other form when verifying a signature.

§0.6 Hexadecimal canonicalization

Every hex string used in canonical-bytes encoding (e.g., SHA-256 digests in `root_hash`, public-key fingerprints) MUST be **lowercase**, **unpadded** (no leading `0x`, no separators), and **byte-length-exact** (a SHA-256 digest is exactly 64 hex characters). Producers MUST emit lowercase; consumers MUST reject uppercase when verifying.

§0.7 Time and clocks

Every `signed_at`, `asserted_at`, `valid_until`, `delegation_valid_from`, `delegation_valid_until`, and `cosigned_at` in this specification refers to **wall-clock UTC** at the asserting peer's clock. Producers SHOULD synchronize via NTPv4 (RFC 5905) or Roughtime to a known-good time source. The maximum tolerated skew between attester clock and consumer clock for a freshness check is ±**5 minutes** by default; tighter thresholds MAY be applied by per-application consumer policy. Consumers receiving an attestation with `signed_at` more than 5 minutes in the future MUST reject as malformed.

Time-skew between cosigners on a single STH (§10.3) is bounded by the STH's own `signed_at` field; cosignatures with `signed_at` farther than 5 minutes from the STH's published `signed_at` MUST be rejected.

For long-lived attestations carrying `valid_until` in the future, the freshness check is "the attestation has not yet reached its `valid_until`, AND the current consumer clock is within ±5 minutes of the substrate's network-consensus clock"; a consumer whose clock drifts past the skew bound MUST fail-secure (reject) rather than accept.

§0.8 H3 cell canonicalization (CEG 0.8 addition)

Per CIRISRegistry#48 + §5.6.8.11 `location_proof` + §5.6.8.10 `community` with `cohort_subkind: geographic`.

Geographic primitives in CEG use H3 hierarchical hexagonal indexing as the canonical cell-identifier encoding. H3 partitions the Earth's surface into hexagonal cells at 16 resolution levels (0 = coarsest, ~4.3 M km^2 per cell; 15 = finest, ~0.9 m^2 per cell). Each cell has a 64-bit integer ID, conventionally encoded as a 15-character lowercase hex string.

Canonical form for `cell_id`:

- 15-character lowercase hex string (no `0x` prefix; per §0.6)

- The cell encodes its own resolution in the standard H3 index bit layout; a conformant decoder extracts the resolution **via the H3 library** (the resolution field lives in bits 52–55), **NOT** by reading the high 4 bits — the high nibble is the H3 **mode marker** (cell-mode = `1`), not the resolution
- Leading zeros preserved (a resolution-0 cell at base position 0 is `8001fffffffffff`, not `1fffffffffff` — the high nibble `8` is the mode marker, correctly consistent with res-0)

Canonical form for `cell_resolution`:

- Integer in `[0, 15]`
- MUST equal the resolution **decoded from the H3 index** of `cell_id` (substrate verifies the redundancy on admission via the H3 library; mismatched pairs MUST be rejected as malformed)

§0.8.1 Rough-only enforcement for `location_proof` (normative)

Per CIRISRegistry#48 privacy invariant. A `location_proof` subject_kind Contribution (§5.6.8.11) MUST carry `cell_resolution` $\leq$ `7` (H3 resolution 7 hexagons average ~5 km^2 edge-length, sufficient for city/borough-level disclosure without block/building precision). Producers attempting to emit finer-resolution `location_proof` Contributions MUST have admission rejected at the substrate gate.

This is the wire-format-enforced privacy promise: **rough is rough by protocol**, not by operator policy. A producer cannot accidentally over-share at the protocol layer; a malformed client cannot publish a precise location even if its UI fails to gate. Substrate emits `hard_case:location_proof_resolution_v` (§7.8) on rejection so operators can observe malformed-producer patterns.

§0.8.2 Cell containment

A cell `C` at resolution `R_C` is **contained within** a cell `B` at resolution `R_B` iff:

- `R_C >= R_B` (the contained cell is at equal or finer resolution); AND
- The parent-walk from `C` at resolution `R_C - 1, R_C - 2, ..., R_B` reaches exactly `B` (standard H3 hierarchy semantics; `h3ToParent` library call)

Used by community admission gates per §8.1.13 Policy M: a `location_proof` Contribution's `cell_id` MUST be contained within the geographic community's `geographic_constraint.cell_id` for membership admission.

§0.8.3 What CEG 0.8 documents

- The lowercase-hex canonical form for `cell_id`
- The resolution-redundancy check (the resolution **decoded from the H3 index** of `cell_id` MUST match `cell_resolution`)
- The rough-only enforcement (`location_proof.cell_resolution` $\leq$ `7`)
- The containment semantics for community admission

What CEG 0.8 does NOT do:

- Mandate H3 over alternative geospatial systems (S2, Geohash) — H3 is chosen for hex-cell uniformity, well-defined parent/child hierarchy, and protocol-agnostic absence of a centralized gazetteer dependency. Operator-internal use of other systems is unconstrained; the wire format uses H3.
- Provide a place-name registry. Communities self-name; cell IDs are the substrate-level binding. UI may map cell IDs to human-readable names per consumer policy.
- Restrict community-side `geographic_constraint.cell_resolution`. A community CAN scope itself at any resolution (e.g., an "Austin metro" community at resolution 5, ~250 km^2; a smaller-scale "Downtown Austin" community at resolution 7, ~5 km^2). The rough-only invariant applies to `location_proof`, NOT to community definitions.

§0.9 Envelope canonicalization — JCS + the omit-vs-materialize rule (CEG 0.9 addition)

Per external critical-review surface (the round-trip-determinism concern that landed against CEG $\leq$ 0.8). Closes the canonical-bytes ambiguity that accumulated as CEG 0.x acquired optional fields with documented defaults — `epistemic_mode`/`witness_relation`/`occurrence_*`/`stake` (pre-0.4 baseline), `oversight_mode` (pre-0.4 + CEG 0.2 ladder rework), `subject_key_ids` (CEG 0.6), `family_id` (CEG 0.7), `community_id` (CEG 0.8).

The hazard is structural and the reviewer named it correctly. If Producer A omits an optional field (relying on the §4 documented default) and Relay R re-serializes the attestation with the default materialized into the byte stream, the canonical bytes diverge and the Ed25519 + ML-DSA-65 signatures no longer verify. Pre-§0.9 CEG implicitly required producers, substrates, relays, and consumers to all make the same choice; §0.9 makes the rule explicit and normative.

§0.9.1 Canonical encoding format (normative)

CEG envelope signing bytes are computed via **JCS — JSON Canonicalization Scheme, RFC 8785** (Rundgren, Jordan, Erdtman; March 2020). JCS pins, in summary:

- Object members sorted lexicographically by member name (UTF-16 code-unit order, RFC 8785 §3.2.3)
- No insignificant whitespace
- UTF-8 byte encoding (RFC 8259 §8.1)
- Strings escaped per RFC 8259 §7 with the JCS narrowing on `\uXXXX` form
- Numbers serialized per the ES6-derived rule (RFC 8785 §3.2.2.3) — integers without trailing `.0`, no exponent unless the magnitude requires it

A CEG-Conforming Producer (CCP per §0.2) MUST produce signing bytes via JCS over the envelope object. A CEG-Conforming Consumer (CCC) MUST recompute signing bytes via the same JCS rule for signature verification. A CEG-Conforming Substrate (CCS) MUST preserve the as-received envelope object bytes for relay (per §0.9.2 below); it MAY store a parsed representation alongside, but the canonical-bytes contract is against the as-received form, not the parsed-and-re-serialized form.

§0.9.2 The round-trip rule — defaults are interpretation-time, not encoding-time (normative)

The canonical bytes are computed over **the literal envelope members the producer signs**. Optional fields that the producer omits MUST NOT be materialized into canonical bytes by any party — producer, substrate, relay, or consumer. Optional fields that the producer explicitly emits (even at their default value) MUST be preserved in the canonical bytes by any party. Documented defaults from §4 are **interpretation-time semantics**, not encoding-time content.

Two valid encodings of the semantically-identical attestation:

```
# Producer A -- omits epistemic_mode (relies on S4 default)
{"attested_key_id":"...","attesting_key_id":"...","confidence":0.9,"dimension":"licensure:
    CA_medical_board","score":0.8}

# Producer B -- explicitly emits epistemic_mode at its default value
{"attested_key_id":"...","attesting_key_id":"...","confidence":0.9,"dimension":"licensure:
    CA_medical_board","epistemic_mode":"direct","score":0.8}
```

Both producers compute different canonical bytes (Producer B's includes the `"epistemic_mode":"direct"` member) and emit different signatures. **Both signatures verify under their respective canonical bytes.** Both are **semantically equivalent** — a CEG-Conforming Consumer evaluates `effective_epistemic_mode(envelope) = envelope.epistemic_mode if present else "direct"` and proceeds identically. The wire-format admits both encodings; the consumer policy admits no observable difference.

Relay discipline (normative). A substrate, relay, or consumer that re-stores or forwards an attestation MUST preserve member presence/absence exactly as the producer signed it:

- Stripping an explicitly-emitted default ("the producer wrote `epistemic_mode:direct` but I'll save bytes by removing it") MUST NOT happen
- Materializing an omitted default ("the producer omitted `epistemic_mode` so I'll fill in `direct` for clarity") MUST NOT happen
- Reordering object members on re-emission is REQUIRED (JCS lexicographic order; if a producer somehow emits non-canonical order, the relay re-canonicalizes — but member presence/absence stays fixed)

This composes with the §4.1 forward-compatibility rule (which already mandates preserving unknown fields on read and re-emission). §0.9.2 extends the same preservation discipline to known-optional fields.

§0.9.2.1 Array ordering + byte-field + timestamp encoding (normative — the three determinism rules JCS does NOT pin)

JCS (§0.9.1) canonicalizes **object member order** but is silent on three things that still let two conformant producers emit different bytes (and therefore non-verifying signatures, the §0.9 hazard). All three are pinned here once, globally, for every CEG envelope (raised in CIRISVerify#63 review):

1. **Array element order.** An array field is one of two kinds, stated in its §4/§5 definition:

- **Set-semantics** (order carries no meaning) — elements MUST be **lexicographically sorted by their JCS string form (UTF-16 code-unit order, ascending)** before signing. This is producer-independent: any producer building the set in any order yields identical bytes. **`subject_key_ids[]` and `delegates_to.delegated_scope[]` are set-semantics → sorted.**
- **Sequence-semantics** (order is meaningful) — elements retain their as-authored order. **`transport_destination.aspects[]` (RNS hash preimage order), `key_grant.rotation_chain` (supersession lineage), and `evidence_refs[]`** (producer-asserted order) are sequence-semantics → preserved. A field's definition MUST declare which kind it is; absent a declaration, set-semantics (sorted) is the default.

1. **Byte-field string encoding.** JSON has no byte type, so every byte/binary field is a string whose encoding MUST be pinned. **Key references, hashes, and identifiers — `key_id` / `attesting_key_id` / `attested_key_id` / `subject_key_ids[]` elements, public keys, `destination_hash`, `root_hash`, fingerprints — MUST be lowercase hex per §0.6** (no `0x`, no separators, byte-length-exact; never base64 in canonical bytes). Fields explicitly documented as base64 (e.g., `hardware_attestation`, an opaque attestation blob) use base64 as their definition states — but the *default* for any key/hash/id byte field is §0.6 lowercase hex.

- **The base64 variant is pinned (1.0-RC1 — CIRISVerify#64): every field documented as base64 (the `*_base64` suffix convention — `encryption_pubkeys.x25519_base64` / `.ml_kem_768_base64` per §5.6.8.8.2, the directory `pubkey_ed25519_base64` / `pubkey_ml_dsa_65_base64`, `hardware_attestati` MUST use the RFC 4648 §4 STANDARD alphabet, WITH padding, no whitespace or embedded newlines** (the `base64::engine::general_purpose::STANDARD` shape `ciris-verify-core` ships). The pin lives at the **protocol layer**: the crypto layer (`ciris-crypto`) is correctly encoding-agnostic and deals in raw bytes; the wire encoding is CEG's to pin. A variant mismatch (URL-safe alphabet, stripped padding) produces different signed bytes and a **silently non-verifying signature** — the same hazard class as the wrap-algorithm wire-string. The CIRISConformance#9 vector set MUST include a base64-variant case (encode→sign→verify round-trip across two independent encoders).

1. **Timestamp encoding.** Every datetime field (`signed_at`, `asserted_at`, `valid_until`, `delegation_valid_from`/`_until`, `valid_at`, ...) is the **§0.5 canonical string** — UTC literal `Z` (never `+00:00`), millisecond precision (exactly three fractional digits), `YYYY-MM-DDTHH:MM:SS.sss` A producer emitting `+00:00` or a different sub-second precision produces different bytes and a non-verifying signature.

With §0.9.1 (key order) + §0.9.2 (omit-vs-materialize) + these three, **byte-identity across conformant implementations is closed by construction**. The CIRISConformance#9 cross-impl JCS vector set MUST cover all three (a set-vs-sequence array case, a hex-vs-base64 byte case, a timestamp case) alongside the per-Contribution vectors.

§0.9.3 Per-field encoding table (informational)

The §0.9.2 rule applies uniformly to every optional §4 field. Catalog as of CEG 0.9:

Field	Introduced	Default per §4	Canonical when omitted	Canonical when explicit
`epistemic_mode`	pre-0.4	`direct`	member absent	`"epistemic_mode":"direct"` (or other enum value)
`witness_relation`	pre-0.4	`external`	member absent	`"witness_relation":"self"` (or other enum value)
`oversight_mode`	pre-0.4	`null` (per-cell default applies)	member absent	`"oversight_mode":"HITL"` (or other enum value)
`occurrence_id`	pre-0.4	`null` → `"occurrence-0"` at interpretation	member absent	`"occurrence_id":"occurrenc`
`occurrence_count`	pre-0.4	`null` → `1` at interpretation	member absent	`"occurrence_count":3`
`occurrence_role`	pre-0.4	`null` → `"primary"` at interpretation	member absent	`"occurrence_role":"shared"`
`stake`	pre-0.4	`reputational`	member absent	`"stake":"capital"` (or other enum value)
`context`	pre-0.4	absent	member absent	`"context":"..."` (free-form)
`evidence_refs`	pre-0.4	absent	member absent	`"evidence_refs":["..."]`
`valid_until`	pre-0.4	absent	member absent	`"valid_until":"2026-12-31T`
`subject_key_ids`	CEG 0.6	`null`/empty	member absent	`"subject_key_ids":["..."]`
`family_id`	CEG 0.7	n/a (REQUIRED iff `cohort_scope == family`)	member absent (admission rejects if cohort_scope == family)	`"family_id":"..."`
`community_id`	CEG 0.8	n/a (REQUIRED iff `cohort_scope == community`)	member absent (admission rejects if cohort_scope == community)	`"community_id":"..."`
`delivery_mode`	CEG 0.10	`pull`	member absent	`"delivery_mode":"push"`
`listed`	CEG 0.10	absent (private roster)	member absent	`"listed":"public"` (opt-in only per §11.8.3-shape; producers MUST omit unless subject has opted in)
`history_on_join`	CEG 0.10	`from_join`	member absent	`"history_on_join":"full"`

The conditional-required fields `family_id` and `community_id` are NOT optional-with-default — substrate rejects mis-shape per §4.2.6 + §11.6.2 + §11.7.2 — but they encode under the same JCS rule when present.

§0.9.4 Verification flow (informational)

```
On signature verify:
   1. Receive envelope from wire as object O (preserved as-received)
      and signature S
   2. Compute canonical bytes B = JCS(O)
   3. Verify S over B via the hybrid Ed25519 + ML-DSA-65 path per S5.2.1
   4. If signature valid:
       a. Compute effective semantics by applying S4 defaults to absent
          optional fields (interpretation-time only)
       b. Apply consumer policy per S8 over the resulting semantic shape
   5. If forwarding/storing:
       a. Store/forward object O AS RECEIVED -- do not normalize, strip,
          or materialize defaults
       b. Forward original signature S unchanged
```

§0.9.5 Worked attack the §0.9 rule closes

Pre-§0.9 (implicit / unspecified) failure mode:

> *Alice's CEG-Conforming Producer signs an envelope omitting* `epistemic_mode`*. Bob's CEG-Conforming Relay receives, parses, materializes the default* `"direct"`*, re-serializes via JCS, and forwards to Carol. Carol receives the relay-modified envelope with the new bytes, computes JCS, verifies signature →* ***FAILS*** *because Alice signed over bytes without the* `epistemic_mode` *member. Carol cannot distinguish "Bob is a malicious relay corrupting the bytes" from "Bob is honestly applying defaults to be helpful." Carol rejects. The attestation is lost in transit despite no party acting in bad faith.*

Post-§0.9 (explicit, normative):

> *Bob's relay MUST preserve member presence/absence exactly. Bob forwards the as-received bytes. Carol's verify succeeds. The semantic interpretation step at Carol (applying default* `"direct"` *to the absent member) happens AFTER signature verification.*

§0.9.6 What CEG 0.9 (this section) documents

- The JCS-as-canonical-encoding rule for envelope signing bytes
- The omit-vs-materialize rule for optional fields with documented defaults (the round-trip-determinism fix)
- The relay-preservation discipline that closes the worked attack at §0.9.5
- The per-field encoding catalog (§0.9.3) covering every optional §4 field introduced through CEG 0.8

What CEG 0.9 (this section) does NOT do:

- Introduce a new encoding format — JCS is the only encoding
- Change any semantic interpretation — defaults still apply at interpretation time per §4
- Modify the §0.5 datetime / §0.6 hex / §0.7 time / §0.8 H3 sub-rules — those are domain-specific and compose under JCS
- Wire-break prior 0.x emissions — pre-§0.9 emissions that omitted optional fields remain valid; pre-§0.9 emissions that explicitly emitted defaults likewise remain valid; what §0.9 normatively prohibits is RELAY-TIME mutation of presence/absence, which was always wrong but is now explicitly so

§1 Foundation

§1.1 Mental model — federated structured-claim emission

The federation is a network of peers emitting structured claims about each other and about reality. A claim travels as a **Contribution** (the universal envelope) carrying a typed **Attestation** (the actual content of the claim).

What CEG is, stripped of framing. Independent of the CIRIS application, the AI vocabulary, or the §1.2 anthropology, CEG is a **signed, compositional graph language for expressing claims, relationships, authority, membership, consent, governance, addressing, and settlement across a decentralized network** — a general-purpose *attestation calculus*. Structurally it is closer to a composition of Certificate Transparency + MLS + ActivityPub + DID/VC + reputation systems + governance protocols than to a conventional AI architecture. The AI/agent use cases are the *first consumer* of that calculus, not its definition. Read this way, the rest of the spec is: one workhorse claim primitive, four graph-composers, and a namespace.

Every Attestation answers four questions in machine-readable form:

1. **WHO emits** — issuer key_id, signature, witness_relation, optional accord/steward sign-off
2. **WHAT KIND of claim** — a prefix from the canonical namespace (§5)
3. **HOW STRONG** — polarity (+/-), score magnitude, cohort scope
4. **WHAT IT'S BASED ON** — evidence_refs, schema_ref (calibration version), validity window

Consumers walk attestation graphs and compose verdicts. The substrate stores; the wire transports; CEG describes the shape of the claim. None of the three prescribes outcomes; consumer policy does.

§1.2 The Ubuntu commitment — relational-anthropology substrate *(informative)*

Per `CIRISAgent/ContemplativeTraditions/Ubuntu.lean::F_ubuntu_primary_tradition_commitment` and `../MISSION.md` §1.5:

> *Umuntu ngumuntu ngabantu — a person is a person through other persons. Persons are not atomic; the relation IS the person.*

Five load-bearing consequences for the wire format:

1. **The attested entity is not prior to its attestations.** A `federation_keys` row is not a representation of a pre-existing entity that the federation observes; it is the locus at which an entity is partly constituted by the cross-attestations that name it. Self-signature alone is not identity; cross-attestation is.

1. **Attesting is a participatory act, not an observation of fact.** A `scores` attestation does not merely report data about the attested entity. The attester's score participates in constituting the entity's standing in the relational field that consumers compose policy over.

1. **Detection brings patterns into morally-real existence.** A correlated-action pattern does not pre-exist its detection waiting to be observed. The detection-and-attestation is what crosses the pattern from "statistical regularity" to "morally-real object the federation now bears."

1. **Harm and deception collapse at the structural level.** Under Cartesian individualism, harm (setback to interests) and deception (causing false belief) are categorically distinct because persons are atomic and beliefs are private. Under Ubuntu, where personhood is partly constituted by accurate perception of the relational field, damage-to-perception IS damage-to-personhood IS harm. CEG's `detection:correlated_action:{axis}` family carries both via one prefix.

1. **The Recursive Golden Rule is structural, not exhortatory.** No principal — including the steward triple and CIRIS L3C itself — is exempt from constraints they impose on others. This is the wire-format symmetry of §8.4 below (Sovereign-Registered equivalence) plus the §7 reserved-prefix patterns that bind even canonical bootstraps. Adding any privileged shortcut for a federation-internal principal would violate the Ubuntu substrate at primitive level.

Why this is named here and not bracketed. Engineering specs tend to bracket anthropology as "out of scope." But the wire format encodes anthropological commitments whether they are named or not. Bracketing them out means defaulting to whichever commitments contributors assumed by training — the Cartesian-individualist default is pervasive in cryptographic identity work (PGP web of trust, X.509 PKI, even most decentralized-identity schemes treat the key as representing a pre-existing atomic principal). CEG is not Cartesian. Naming the substrate explicitly is the discipline that prevents the open vocabulary, the reserved-prefix patterns, and the consumer-policy norms from drifting back toward the Cartesian default through unexamined intermediate choices.

Cross-tradition reading. The same structural object is approached from multiple traditions — Ubuntu (relational-primary), Logos (rational-order-of-reality), Tao / Dharma / Aristotelian virtue. CEG does not encode any one tradition's vocabulary; it encodes the *structural object* the traditions converge on. Future namespace extensions should be locatable in this substrate, not in a Cartesian fallback.

§1.3 Operational-language gate — the safety-vs-censorship discipline

Per `ciris.ai/safety-vs-censorship`:

> *"Rules are crowdsourced. Verdicts are machined." "The same machinery that catches real failures can become the machinery that enforces preferences." "None of this is automatic."*

Translated to CEG wire format: **prefix names must describe machine-checkable conditions, not subjective qualities**. The drift the page warns about — rules sliding "from 'uses the wrong word for therapy' toward 'feels disrespectful'" — has a wire-format analog: prefix names sliding from mechanism-descriptive (`detection:correlated_action:*`) toward judgment-descriptive (`detection:emergent_deception:*`). Both forms admit the same downstream verdicts; only one admits them honestly.

§1.3.1 The four-test prefix-admission gate

Every prefix admitted to the §5 namespace MUST pass:

Test	Question	Pass criterion
T1	Is the prefix part of a published, hash-pinned, version-controlled rule set, distinct from per-attestation verdicts?	Rules + verdicts separated in writing
T2	Does the prefix name a **mechanism** (correlation, count, time-window, schema-conformance) rather than a **subjective quality** (deception, harm, virtue, trustworthiness, sin)?	Mechanism-descriptive prefix name
T3	Can past verdicts be re-checked against the rule version they ran against?	Version-pinning in `evidence_refs[]`
T4	Is the prefix wired so its attestations are **never sole evidence** for `slashing:*`?	Adjudication separation

Existing prefixes failing T2 (the most slip-prone gate) get renamed; the canonical example is `detection:emergent_deception:*` (failed T2 in v1.1) → `detection:correlated_action:*` (passes T2 in v1.2). Anti-pattern catalogue at §13.

§1.4 The 1+4 minimal-and-adequate claim

The federation has exactly **one workhorse attestation primitive + four structural composers** at the **structural layer**. That is a genuine, narrow invariant — the *graph-operation* set is closed at five (`scores` + `delegates_to` / `supersedes` / `withdraws` / `recants`). It is **not** a claim that the whole grammar is five things.

Scope of the claim (read this before citing "1+4"). What follows is an **inductive adequacy result, not a closure theorem.** The fifteen paths below show the structural set is *expressive across the surfaces tested*; they do **not** prove it generates *every* expressible structured claim. We have not defined the class of structured claims and proven 1+4 generates exactly it — until someone does, "1+4 is adequate" means "adequate across the fifteen surfaces examined," nothing stronger. The refutation bar ("exhibit a claim that cannot be composed") is, honestly, near-unfalsifiable while *composition itself* is unbounded — so absence of a counterexample is weak evidence, and these paths should be read as accumulating confidence, not as proof.

"Minimal" is partly an accounting choice. The structural set is five, but every path moved complexity *into* the namespace and envelope axes rather than removing it. The **full normative conformance surface** a second implementer must get exactly right is much larger than five: ~9 `subject_kind`s (§5.6.8) plus the open `external_content` sub_kind set, ~21 optional envelope fields (§4), 13 composition policies (A–M, §8), 5 canonicalization families (§0.5–0.9), 6 `consensus_protocol` kinds, the §7 reserved-prefix taxonomy, and dozens of dimension prefixes (§5). "1+4" is the elegant *structural* invariant; it is **not** the conformance surface, and citing it as "the grammar is five things" understates what interop requires. Always report the surface beside the invariant.

Fifteen independent design exercises (consent, families, communities, streaming, payments, DNS-free addressing, settlement, ...) each composed without a new structural primitive; the enumera-

tion lives in the canonical working draft.

Future extensions are dimension prefixes or envelope fields, not new structural primitives. Proposals to expand the 1+4 set face a high evidentiary bar and route through the §11.2 amendment process. A successful refutation requires either: (a) demonstrating an operational claim that cannot be expressed via the existing 1+4 set plus envelope composition, OR (b) demonstrating a structural-primitive consolidation that reduces below 1+4 without loss.

The standing falsification target (named, so the claim is a real bet). The strongest current candidate for "a genuinely important domain not naturally expressible in 1+4" is **atomic fair exchange / bilateral simultaneity** — atomic content-for-payment, atomic swaps, simultaneous mutual commitment. CEG attestations are *unilateral, monotonic* graph claims; fair exchange is classically impossible without a trusted third party or a totally-ordered ledger (Even–Goldreich–Lempel). CEG does **not** express it in-grammar — it **bridges** atomicity to an external settlement rail (§5.6.8.12 `settlement` over a chain) and records only the after-the-fact trust claim. That is the honest boundary: the first domain where 1+4 reaches for something outside itself. The claim "1+4 is adequate for the federation's claims" survives *because* fair exchange is treated as out-of-grammar (a bridge, not a primitive); it would be **refuted** by either (i) a natural in-grammar expression of fair exchange, or (ii) a federation-critical domain that resists even bridging. Adversarial reviewers: this is the test to push on.

§1.5 The Recursive Golden Rule (structural, not exhortatory)

No principal — including CIRIS L3C as steward — is exempt from constraints the protocol imposes on others. Operational bites in CEG-shape:

- **Per-install stewards bind CIRIS L3C as steward.** Once `bootstrap_threshold` $\geq$ 2, no single Registry install can issue federation-scope attestations unilaterally.
- **Partner-revocation rules apply to CIRIS L3C subsidiaries.** `revocation:*` carries no steward exemption.
- **Audit discipline applies to steward operations.** Every admin RPC carries the operator's identity into `actor_user_id`, including for CIRIS L3C staff.
- **Bond forfeiture applies to CIRIS L3C-affiliated partners.** No exemption.
- **The HUMANITY_ACCORD asymmetry (§9) is the ONE constitutional asymmetry.** Three named human holders carry kill-switch authority no federation-internal authority can grant / revoke / override / decay. This is not a Golden-Rule exemption; it is the recognition that consent requires revocability, and revocability requires a halt-authority outside the system being halted.

If a principal would be exempt from a constraint at any of these primitives, the Golden Rule is violated at that primitive and the protocol is the wrong shape there. Fix the primitive, not the rule.

§1.6 Adversary model & privacy non-goals (normative)

CEG makes confidentiality and integrity claims; this section bounds them. **The word "privacy" in this spec means exactly two things and no more: (1) content-holding confidentiality and (2) cohort-scoped visibility.** It does **not** mean metadata privacy, communication-

graph privacy, or unobservability. Implementers and operators MUST NOT represent CEG as providing the stronger properties.

§1.6.1 What the structural-invisibility primitive (§10.1.4) buys

Suppressing `holds_bytes:sha256:*` for `cohort_scope: self | family` content gives **content-holding confidentiality**: a non-member cannot *discover that the bytes exist via the substrate* and cannot *fetch* them (no holder is advertised; the bytes are delivered only to admitted members via the at-rest key cascade). End-to-end content confidentiality is additionally provided by the per-epoch DEK (hybrid X25519+ML-KEM-768) and AES-256-GCM. That is the whole of what omission buys.

§1.6.2 Non-goals — what omission does NOT buy

The following are **explicitly out of scope** at the base CEG/RET layer; treating them as provided is an error:

- **Relationship-existence privacy.** The *existence* of a self-collective / family / community and its membership-change events are observable: `family_id` / `community_id` ride the envelope, and admission / removal / consensus-protocol changes emit `hard_case:*` reserved-prefix events (§7.7–7.8) into the log. An observer learns that a group exists, roughly how big it is, and when its membership churns.
- **Communication-graph / metadata privacy.** DNS-free member resolution (§8.1.13.1.1) plus Reticulum announce / path-request expose *who is reachable where*; the federation directory + `transport_destination` bindings name endpoints. A passive network observer or an honest-but-curious member can reconstruct a substantial portion of the **who-talks-to-whom** graph. Cohort scope hides *content*, not *contact*.
- **Traffic-analysis resistance.** Encrypted streams still leak via side channels the wire format does not pad or cover: the §10.5.1 STH cadence (default T=2 s), the churn-driven key-cascade volume/timing (§10.5.3), and per-chunk size/rate. An observer can infer stream existence, approximate group size, churn rate, activity bursts, and often media bitrate class — without decrypting a byte.
- **Unobservability / anonymity.** Base CEG/RET provides neither sender/receiver anonymity nor cover traffic. Self-certifying cryptographic identities are *pseudonymous*, and the transport reveals path endpoints. Anonymity is a **separate, opt-in** mechanism (the CIRISNodeCore Anonymous Tier — Sphinx onion routing), NOT a property of base CEG.
- **Post-compromise security (PCS) for streams.** The CEG 0.7 §11.7.1 Option-A choice is forward-only: a member removed at epoch e cannot read epoch $e+1$, but a *compromised current member's* key is not self-healed by a key-update the way MLS PCS provides (revisit tracked for CEG 0.15).

§1.6.3 Adversary classes (and where each is / is not addressed)

Adversary	Addressed	NOT addressed
Passive network observer	content confidentiality (AEAD + DEK); equivocation (STH)	comm-graph, traffic analysis, group size/churn inference

Adversary	Addressed	NOT addressed
Honest-but-curious member	— (members see in-scope content by design)	can enumerate co-members + reconstruct local comm-graph
Malicious member	cannot forge others' attestations; removal is forward-secret	can leak content they were entitled to; metadata as above
Compromised substrate node	cannot decrypt self/family content (no DEK); CEG-native replication carries signed provenance	can observe directory metadata + traffic patterns it routes
Equivocating producer	**mitigated** — per-stream STH (§10.5.1) + consistency proofs (§10.3.1): cannot show different chunk-K to different viewers nor rewrite mid-stream	—

Operator guidance: if a deployment requires metadata privacy or unobservability (e.g., under a totalitarian-threat model), it MUST layer the Anonymous Tier; base CEG/RET is not sufficient. State this in any user-facing privacy representation.

§2 The reasoning grammar — the eight axes

These are how a consumer reasons about an envelope. They are **not wire fields**; they are the canonical questions a consumer can ask about any Attestation.

Axis	Question	Values
Polarity	Direction of the claim	Positive / Negative / Neutral / Indeterminate{reason}
Object	What the claim is about	key_id (entity) / attestation_id / contribution_id
Time	When is the claim valid	`asserted_at` + optional `valid_until`; consumer composes with substrate `expires_at`
Epistemic mode	How was the claim formed	direct / crypto / hearsay / derivative / appeal
Reversibility	Can the attestation be reversed	rollbackable / non-rollbackable (consumer policy + per-prefix rule)
Stake	What's backing the attester's claim	free / reputational / capital / cryptoeconomic
Scope	At what scale does the claim apply	self / family / community / affiliations / species / biosphere / federation
Inter-attestation relations	How does this attestation relate to others	standalone / refers-to / supersedes / withdraws / recants / corroborates / contradicts / clarifies (four of these are structural primitives per §3; rest are emergent from scalar composition)

`biosphere` added to Scope per CIRISRegistry#30 (distinct from `species` which is Homo sapiens cohort). See §13 for the `planet` colloquial alias note.

§3 The primitive set — 1+4

§3.1 The workhorse: `scores`

The federation has exactly **one** workhorse attestation primitive. Every claim about an entity — positive or negative, identity or capability or behavior or state or commitment, by any attester source — is expressed as a `scores` attestation on a named dimension.

```
// Wire shape (Persist's federation_attestations row):
attestation_type: "scores"
attesting_key_id: <attester's key_id>
attested_key_id: <subject's key_id>
attestation_envelope: {
  "dimension": "<canonical-namespace-prefix>:<scoped-leaf>",
  "score": <f64 in [-1.0, +1.0]>,
  "confidence": <f64 in [0.0, 1.0]>,
  "context": "<free-form scoping detail>",
  "evidence_refs": ["<URI or hash referencing backing evidence>", ...],
  "valid_until": "<ISO8601 datetime, optional>",
  "epistemic_mode": "<direct | crypto | hearsay | derivative | appeal>", // optional; default '
      direct'
  "witness_relation": "<self | external | derived>", // optional; default 'external'
  "oversight_mode": "<HITL | HOTL | HOOTL | null>", // optional; default null
  "occurrence_id": "<occurrence-N | __shared__ | null>", // optional; default null
  "occurrence_count": <int >= 1 | null>, // optional; default null
  "occurrence_role": "<primary | shared | replica | null>", // optional; default null
  "stake": "<free | reputational | capital | cryptoeconomic>" // optional; default 'reputational
      '
}
```

Full field semantics in §4.

§3.2 The four structural composers

Operations on the attestation graph itself, not score-claims on entities:

`attestation_type`	**What it does**	**Envelope shape**
`delegates_to`	A authorizes B to sign on A's behalf within a bounded scope	`{delegated_scope[], delegation_purpose, delegation_valid_from, delegation_valid_until}`
`supersedes`	This attestation row replaces a prior one by the same attester	`{references_attestation_id, supersession_reason, differs_in[]}`
`withdraws`	I retract my prior attestation (does NOT claim it was false)	`{references_attestation_id, withdrawal_reason}`
`recants`	My prior attestation was false at issuance — admits epistemic error	`{references_attestation_id, recantation_reason, what_was_false}`

Translation implications:

- A **doctrinal-development** claim ("this version extends but does not contradict the prior version") is `supersedes` with `differs_in: ["scope", "evidence_refs"]` — NOT `recants` (which would assert prior was false).

- An **acknowledged-error** claim ("the prior framing was wrong; I admit the mistake") is `recants` — distinct from `withdraws` (which retracts without making a falsity claim).
- A **prudent-retraction** ("I'm withdrawing without claiming it was false; context has changed") is `withdraws`.
- An **authority-source claim via delegation** ("this constitutional position derives from authority-key X in scope Y") is `delegates_to` with X as `attested_key_id` (the §3.2.1 reuse pattern for authority-source claims, replacing what would otherwise need a `grounding:{tradition}:{pri` prefix that fails §1.3.1 T2).

§3.2.1 Authority-source claims via `delegates_to`

A constitutional or framework claim can name its source-of-authority by emitting `delegates_to` against an `attested_key_id` representing the framework, with `delegated_scope` naming the principle. Example: a Ubuntu-substrate commitment in §1.2 commitment 2 can be expressed as `delegates_to` against the `ubuntu_relational_substrate` framework-key with `delegated_scope: ["personhood_constitutive_by_attestation"]`. Reuses the existing structural primitive rather than introducing a `grounding:{tradition}:{principle}` prefix (which would fail §1.3.1 T2 — "tradition" claims are interpretive, not mechanism-descriptive).

§3.2.2 The `recants` distinction matters

Per `PRIOR_ART_SCAN.md` Bucket 1: no prior identity system (PGP, SPKI/SDSI, W3C VC) typed epistemic-error-admission as a wire primitive distinct from retraction. CEG types both because the Recursive Golden Rule applies to attesters: admitting error is a primary act, not a derivative of retraction. Consumer policy can apply different trust adjustments to attesters who `recant` versus those who `withdraw`.

§3.2.3 `withdraws` admission rule — CEG 0.6 broadening (semantic, not structural)

Substrate MUST admit a `withdraws` Contribution against target `T` when the issuer's `key_id` satisfies **ANY** of:

#	Authority path	Description
1	`issuer.key_id == T.attesting_key_id`	Producer self-withdraw (today's shape; unchanged)
2	`issuer.key_id ∈ T.subject_key_ids`	Federation-keys subject revocation (NEW — CEG 0.6)
3	∃ `delegates_to` chain: `issuer` →* `canonical_hash` where `canonical_hash ∈ T.subject_key_ids` AND `scope ⊇ {consent_revocation}`	Proxy authority for non-federation-enrolled subjects (NEW — CEG 0.6; resolves CIRISAgent#840 OQ3)
4	`issuer` holds valid `delegates_to` → any of 1-3	Delegated revocation (existing primitive, new admission path)

Rule (3) is the elegant answer to the un-enrolled-party case. When a subject is a Discord user-id, a content-sha256-bound entity, or any other non-federation party, revocation authority is mediated through a `delegates_to` chain from a federation-keys signer (typically the agent that holds data on behalf of the external party) to the canonical-hash subject. The agent emits

`delegates_to(canonical_hash → agent_key, scope: [consent_revocation])` at proxy-establishment time; subsequent `withdraws` from the agent against any Contribution carrying `canonical_hash` in its `subject_key_ids` is admitted under rule (3). The `canonical_hash` here is the tagged `canonical:{hashalg}:{hex}` wire form with the `{platform}:{entity_kind}:{id}` preimage convention pinned at §4.2.2.1. **Rule (3) proxy is distinct from `canonical_binding`** (a retroactive identity claim that promotes a canonical-hash to a real key_id, enabling rule (2) direct revocation) — see §4.2.2.2 for the distinction; `canonical_binding` is not a new admission rule.

Per-rule audit metadata: substrate SHOULD record which admission rule (1-4) admitted each `withdraws` Contribution in `federation_attestations` metadata so downstream consumers can compose policy (e.g., higher confidence weight for subject self-revocation rule 2 than for proxy rule 3).

Composition with `recants`: subject-side authority does NOT extend to `recants` (the falsity-admission primitive) — only the original attester can `recant` their own claim. A subject who believes the producer's claim about them is FACTUALLY wrong (not merely unwanted) issues `scores` with negative polarity on a contradicting dimension; that is the consumer-side rebuttal path, distinct from consent revocation. The `recants` / `withdraws` distinction matters precisely because subject authority covers the consent dimension (revocability) but not the truth dimension (falsity).

§4 The envelope

Every `scores` Attestation carries this envelope. Field semantics consolidated here.

Field	Required	Description
`attesting_key_id`	(substrate field)	Attester's `federation_keys.key_id`.
`attested_key_id`	(substrate field)	Subject's `federation_keys.key_id`.
`dimension`	yes	The canonical namespace prefix + scoped leaf. Persist treats this as TEXT; consumers parse against §5's namespace map.
`score`	yes	Pos/neg scalar in [-1, +1]. Polarity is encoded by sign; magnitude carries strength. Some dimensions are boolean-via-score (±1 only); some are positive-only; some are signed; per-dimension table in §5 names the polarity.
`confidence`	yes	The attester's own confidence in their score. [0, 1]. Low confidence + high magnitude = "I believe this strongly but I might be wrong"; high confidence + low magnitude = "I am sure the truth is near-neutral."
`context`	no	Free-form scoping detail. Not parsed by the substrate; used by consumers + audit + RATCHET.
`evidence_refs`	no (often required by per-dimension policy)	List of URIs / content-hashes pointing to backing evidence (Stripe receipt, licensing-body record, observed interaction, log entry, audit-chain leaf, etc.). Some dimensions in §5 require non-empty evidence_refs.
`valid_until`	no	ISO 8601 datetime per §0.5. If set, consumer policy treats the attestation as stale after that point (independent of the substrate row's own `expires_at`).
`epistemic_mode`	no	Per §2 Epistemic-mode axis; default `direct`. Consumers may weight by mode (e.g., direct witness > hearsay).
`witness_relation`	no	`self` \
`oversight_mode`	no	`HITL` \

Field	Required	Description
`occurrence_id`	no	Identifies which occurrence of a multi-occurrence agent deployment emitted this attestation. Format: `"occurrence-{n}"` per the agent's `AGENT_OCCURRENCE_ID` env var, or `"__shared__"` for shared-task pattern emissions. Default `null` → treated as `occurrence-0` for backward compat. **Self-asserted**: this field is NOT cryptographically bound to a fleet-attestation primitive in 0.x; an adversary running a single key can claim any occurrence_id. Acknowledged design tradeoff per §15.2.
`occurrence_count`	no	Total occurrences in the deployment fleet emitting the attestation; integer ≥ 1. Default `null` → `1` (single-occurrence). Same self-assertion caveat as `occurrence_id`.
`occurrence_role`	no	`primary` \
`stake`	no	Per §2 Stake axis; default `reputational`. Composes with the attester's actual stake-backed-by attestations from §5.9.
`community_id`	no (REQUIRED iff `cohort_scope == community`)	**CEG 0.8 addition.** The `community_key_id` of the community this Contribution is scoped to, per §5.6.8.10 `community` subject_kind. One identity MAY belong to multiple communities; the field disambiguates which community's roster gates visibility. Required iff `cohort_scope == community` (substrate rejects community-scoped Contributions missing the field). Parallel to `family_id` (CEG 0.7) but with different semantics: community content emits `holds_bytes:sha256:*` pointing to **ciphertext + cleartext provenance** — encrypted at rest under the per-community DEK (CEG 0.17, §8.1.13.3; `cohort_subkind: infrastructure` communities are the plaintext Commons exception). Byte-level structural-invisibility (no `holds_bytes` at all, §10.1.4) remains self/family only.

Field	Required	Description
`family_id`	no (REQUIRED iff `cohort_scope == family`)	**CEG 0.7 addition.** The `family_key_id` of the family this Contribution is scoped to, per §5.6.8.9 `family` subject_kind. One identity MAY belong to multiple families (§8.1.12 Policy L); the field disambiguates which family's DEK applies and which membership roster gates visibility. Required iff `cohort_scope == family` (substrate rejects family-scoped Contributions missing the field). Composes with §10.1.4 structural-invisibility — 'cohort_scope: self \
`subject_key_ids`	no	**CEG 0.6 addition.** List of consent-holder `key_id`s for this Contribution. Each entry MAY be a `federation_keys.key_id` OR a canonical-hash identifier (per §4.2 below). Each listed key has substrate-recognized authority to (a) issue `withdraws` against this Contribution (per §3.2 broadened admission rule) and (b) emit `consent:*` dimensions about this Contribution. Default `null`/empty = no subject authority (status quo; producer-only authority). Orthogonal to `cohort_scope` AND `delivery_mode` — see §4.2.4.
`delivery_mode`	no	**CEG 0.10 addition.** 'pull \
`listed`	no	**CEG 0.10 addition.** Per-membership opt-in flag — value `public`. Default absent (roster is producer- + self-queryable, NEVER globally enumerable). Public listing mirrors the §11.8.3 location opt-in discipline: opting into roster visibility is a one-way disclosure the member chooses; substrate does NOT solicit. Composes with §8.1.13 Policy M community membership and the new §10.5 streaming endpoint set.
`history_on_join`	no	**CEG 0.10 addition.** 'full \

`epistemic_mode` vs `witness_relation` — distinct dimensions: these co-vary at edges but name different concerns. `epistemic_mode` names the *process* by which the claim was formed; `witness_relation` names the *relational position* of the attester to the attested. F-3 detector attestations carry both (`epistemic_mode: derivative` + `witness_relation: derived`). Most encyclical-sourced translations are `witness_relation: external` + `epistemic_mode: hearsay`. When in doubt, set both.

§4.2 `subject_key_ids` semantics (CEG 0.6)

Per CIRISRegistry#45. The missing half of consent at the wire format: CEG 0.x baseline encoded only **producer authority** (`attesting_key_id`); CEG 0.6 adds **subject authority** for content

where the subject of the data is not the producer of the data.

§4.2.1 The shape

`subject_key_ids` is an OPTIONAL list. When present, each entry names a party with substrate-recognized authority over this Contribution's continued processing. The basic shape:

> *A consent record is a signed declaration by a subject about the substrate's continued processing of content where that subject appears.*

The medical record / photo / interview / training-datum / group-chat case all share the same shape — a producer publishes a Contribution; one or more subjects appear in it; the subjects retain revocation authority. The non-medical example table at CIRISRegistry#45 walks 13 content shapes uniformly.

§4.2.2 Federation-key vs canonical-hash identifier

`subject_key_ids[i]` MAY be either:

1. **A `federation_keys.key_id`** — the subject is a federation-enrolled identity that can sign on its own behalf. Direct revocation: subject signs a `withdraws` against this Contribution; substrate admits via rule (2) of §3.2 broadened `withdraws` admission. Wire form: the bare `key_id` (no tag).
2. **A tagged canonical-hash identifier** — the subject is an external party with no federation_keys row (Discord user-id, channel-id, content-sha256-bound entity, etc.). The substrate cannot verify a signature from this entity directly; **revocation rides a `delegates_to` proxy chain** per rule (3) of §3.2 broadened admission. Wire form: a tagged string per §4.2.2.1 below.

Resolves CIRISAgent#840 OQ3 — "self-attestation about un-enrolled parties — canonical-hash or pseudonymous federation_keys?" — with the answer "both, distinguished by an explicit tag on the canonical-hash variant."

§4.2.2.1 Canonical-hash wire form + preimage convention (CEG 0.10 normative pin; per CIRISRegistry#53)

CIRISRegistry#53 surfaced that a CEG-Conforming implementation cannot interoperate without two things the pre-0.10 spec left unpinned: (a) how a canonical-hash entry is distinguished from a `federation_keys.key_id` entry on the wire, and (b) what string is hashed (the preimage). Both are now normative.

Wire form — tag REQUIRED (CONFIRM-2 resolution). A canonical-hash entry MUST carry the tag `canonical:{hashalg}:{hex}`:

- `{hashalg}` — the hash algorithm; `sha256` is the v1 algorithm. Algorithm-agility: future hashes (e.g. `sha3-256`) extend this position without ambiguity

- `{hex}` — the digest in §0.6 canonical hex (lowercase, unpadded, byte-length-exact: 64 chars for sha256)
- Example: `canonical:sha256:ff7c5632dae6ef3ae7f6283bd35268bc7910332414aa8a1c35a1645ca0295f`

Why the tag is mandatory and bare-hex is rejected: a `federation_keys.key_id` is, in the reference Registry, `hex(sha256(ed25519_pubkey))` — a **lowercase 64-char hex string** (`crypto::HybridCrypto::fingerprint`). A bare canonical-hash (also lowercase 64-char hex) would be **format-indistinguishable** from a key_id. The format-disambiguation heuristic proposed in #53 CONFIRM-2 ("base64 key_id vs hex canonical-hash") FAILS because Registry key_ids are themselves hex fingerprints, not base64 pubkeys. The tag is therefore load-bearing, not cosmetic. The §0.6 hex rule governs the `{hex}` segment only; the `canonical:{hashalg}:` prefix is a tagged-union discriminator outside §0.6's scope and is exempt from the "no separators" rule. NodeCore's invented `canonical:sha256:{hex}` form is hereby blessed verbatim — drop nothing.

Preimage convention — `{platform}:{entity_kind}:{id}` (PIN-1 resolution, load-bearing). The string hashed to produce `{hex}` MUST be:

```
preimage = "{platform}:{entity_kind}:{id}"
hex = sha256_hex_lowercase(utf8_bytes(preimage))
```

Parsing is **split-on-first-two-colons**: the substring before the first `:` is `{platform}`; the substring between the first and second `:` is `{entity_kind}`; **everything after the second `:` is `{id}` verbatim, and MAY itself contain colons** (so Matrix IDs like `@alice:example.org` survive). Rules:

- `{platform}` — lowercased; open vocabulary per §11.2.1. Canonical seeds: `discord`, `slack`, `twitter`, `matrix`, `email`, `phone`, `github`, `xmpp`, `irc`
- `{entity_kind}` — lowercased; open vocabulary. Canonical seeds: `user`, `channel`, `guild`, `room`, `group`, `address`
- `{id}` — the platform's **stable, immutable** identifier, **verbatim** (case-preserved — some IDs are case-sensitive). MUST be the immutable account/object identifier (Discord/Twitter numeric snowflake; Matrix MXID; UUID), **NOT** a mutable handle/username/display-name. Using a mutable handle breaks subject-identity stability the moment the user renames.

The split-on-first-two-colons rule means a producer constructs the preimage by joining exactly three parts with `:`; only the first two colons are structural. This is what makes the rule-(3) `delegates_to` proxy chain (`canonical_hash` ∈ `T.subject_key_ids`) match across producers: every CEG-Conforming producer that names the same `(platform, entity_kind, immutable_id)` triple computes the same `{hex}`, hence the same tagged wire string.

Conformance vectors (testable cross-implementation; the CIRISConformance#9 envelope round-trip set SHOULD include these):

Preimage	sha256 hex	Wire form
`discord:user:123456789012345678`	`ff7c5632dae6ef3ae7f6283bd35268bc7910332414aa8a1c35a1645ca0295f61`	`canonical:sha256:`[illegible]
`discord:channel:987654321098765432`	`af23411c3c6faa55a788660ea29719669b`[illegible]`46f05dd`	`canonical:sha256:`[illegible]
`matrix:user:@alice:example.org` (id contains colons)	`16d4d0bf478835a9af68cdaac730a29b36f`[illegible]`96b975d9`	`canonical:sha256:`[illegible]

Preimage	sha256 hex	Wire form
`twitter:user:1455079377986420736` (numeric id, NOT @handle)	`10243ba010bf159a45197d368f91c025ef6aa1eb7d41cab2256b4124390ad861c2`	`canonical:sha256:0c861c2`
`email:address:alice@example.org`	`04481a02fccfc8d99a47bde4f0563dd360d425b0734e8fca256d0418a02263f8d0a263f`	`canonical:sha256:8d0a263f`

§4.2.2.2 Rule-(3) proxy vs `canonical_binding` — distinct mechanisms (CONFIRM-3 resolution)

These are **two distinct mechanisms**, not one:

- **Rule-(3) `delegates_to` proxy** (§3.2.3 admission rule 3) — *ongoing* revocation authority for an un-enrolled subject. A federation-enrolled key (typically the agent holding data on behalf of the external party) carries `delegates_to(canonical_hash → agent_key, scope: [consent_revocation])`; the agent proxies revocation. The subject never enrolls; the agent acts for them indefinitely.
- **`canonical_binding`** (CIRISAgent#842 Gap 3; NodeCore#29 Ask 4) — *retroactive identity claim.* A now-enrolled federation key asserts "I AM the entity behind `canonical:sha256:{hex}`", binding past canonical-hash subject entries to its real identity. After an admitted `canonical_binding`, the formerly-un-enrolled subject can sign `withdraws` **directly** under rule (2) — the binding promotes the canonical-hash to a real key_id for admission purposes.

`canonical_binding` is **NOT a new admission rule** (no "rule 5"). It composes: the binding is itself a `delegates_to`-shaped attestation (`delegates_to(canonical_hash → newly_enrolled_key, scope: [identity_binding])`) admitted because the enrolling key proves control of the preimage out-of-band (substrate-side proof-of-control is the Persist admission concern, tracked at CIRISPersist#161). Once bound, rule (2) [direct subject revocation] becomes available to the bound key, and rule (3) [proxy] is no longer needed for that subject. NodeCore's conflation of the two in its shipped code comment (per #53) should be corrected to this framing.

§4.2.2.3 `subject_kind` is a payload-level discriminator (CONFIRM-4 resolution)

`subject_kind` (e.g. `consent_record`, `key_grant`, `takedown_notice`, `community`, `family`, `identity_occurrence`, `location_proof`) is a **payload-level** field — it lives inside the Contribution payload, parallel to how `external_content` carries `sub_kind` in payload. It is NOT an envelope-level field. The envelope-level fields are exactly those in the §4 table (`cohort_scope`, `subject_key_ids`, `community_id`, `family_id`, `delivery_mode`, etc.); `subject_kind` is the payload discriminator that selects which §5.6.8.x payload schema applies. The §5.6.8.7 `consent_record` example and any conformant producer payload agree byte-for-byte: `"subject_kind": "consent_record"` is a payload member.

§4.2.2.4 Bilateral ratification is consumer-policy (CONFIRM-5 resolution)

Per §5.6.8.7 step 4: a bilateral partnership is "ratified iff both halves present under the same `bilateral_pair_id` with `stance: granted`" — and this predicate is **consumer policy**, NOT registry-normative. CIRISAgent's CEM `PartnershipRequestHandler` is the canonical consumer that enforces it. Ingest-layer builders (NodeCore's `build_bilateral_pair_id` + request/accept

builders) correctly produce the two halves WITHOUT enforcing ratification — that is the right boundary. The substrate admits each half independently; composition of the pair into a ratified partnership is a downstream read-time computation, never an admission gate.

§4.2.3 Self-as-subject ceremony

When `attesting_key_id ∈ subject_key_ids`, the Contribution is a **self-consent ceremony** — the same identity is attesting AS subject AND producer. This composes naturally with the CIRISAgent#840 CEG-native agent's self-attestation pattern: agent attests `identity:current` about itself with `subject_key_ids = [self.key_id]`, asserting consent-authority over its own identity claims (D08 autonomy claim).

§4.2.4 Orthogonality with `cohort_scope` AND `delivery_mode` (3-axis per CEG 0.10)

CEG envelope encodes **three independent envelope-level concerns** that compose without overlap:

Axis	Field	Authority	Names
Visibility	`cohort_scope` + (`family_id` or `community_id` when set)	Producer-side	Who can SEE the data
Revocability	`subject_key_ids` (CEG 0.6)	Subject-side	Who can REVOKE the data
Delivery	`delivery_mode` + `listed` + `history_on_join` (CEG 0.10)	Substrate / subscriber	Who actively RECEIVES the data + how the substrate fans out

All three may coexist on the same Contribution:

```
A ‘cohort_scope: family‘ contribution
  carrying ‘subject_key_ids: [user_canonical_hash]‘
  carrying ‘delivery_mode: push‘ + ‘history_on_join: from_join‘
  carrying ‘family_id: <acme_household>‘

publishes the bytes at family-cohort visibility (cohort_scope);
the user retains revocation authority (subject_key_ids);
the substrate actively fans out to currently-reachable members
  with new-members getting forward-only content (delivery_mode + history_on_join);
the named family roster gates the membership set (family_id).
```

This orthogonality is load-bearing for the multi-occurrence consent shape: subject-side revocation applies federation-wide regardless of producer's `occurrence_id`; per-occurrence lifecycle consent is a producer-side concern, separately tracked.

The delivery axis is the **third orthogonal axis** per CEG 0.10 — CEG $\leq$ 0.9 encoded visibility + revocability only; the substrate had no envelope-level handle on who actively *receives* + how the substrate fans out. Per §10.5, 0.10 closes that gap with three new optional envelope fields + the new §10.5 endpoint section + a delivery extension to §8.1.13 Policy M.

§4.2.5 Empty/absent = status-quo

`subject_key_ids: null` or `[]` is the status-quo shape (producer-only authority; same as all CEG $\leq$ 0.5 Contributions). All CEG 0.x consumers that don't read the field see status-quo behavior. CEG 0.6 is additive at the envelope layer.

§4.2.6 Subject-bearing dimensions (governance requirement)

Per §11.6, dimensions whose namespace pattern names a subject (e.g., `observed:user:{key_id}:*`, `epistemic:about:{key_id}:*`, `consent:partnered:{user_key}`, `agent_files:*:{subject_target}`) MUST carry `subject_key_ids` containing that subject. The substrate MAY reject admission of subject-naming dimensions that omit `subject_key_ids`. This closes the default-leak failure mode where subject-bearing content publishes without wire-level subject authority.

§4.1 Forward-compatibility rule

> ***Canonical-bytes contract**: the canonical-bytes encoding of this envelope for signing follows §0.9 (JCS over the envelope object; defaults are interpretation-time, NOT encoding-time; relay MUST preserve member presence/absence exactly as the producer signed). Optional fields with documented defaults in the table above ride the §0.9.2 omit-vs-materialize rule. Conditional-required fields (`family_id` per CEG 0.7, `community_id` per CEG 0.8) are NOT optional-with-default and substrate rejects misshape per §4.2.6 + §11.6.2 + §11.7.2.*

A Conforming Consumer (CCC per §0.2) that receives an envelope carrying a field-name it does not recognize MUST:

- Preserve the unknown field on read (do not strip).
- Preserve it on re-emission if the Consumer is also acting as a Producer relaying the attestation.
- NOT use it in verdict composition.
- NOT reject the envelope on the basis of the unknown field alone.

Producers introducing a new envelope field MUST follow the §0.3 versioning rules: a new field with a documented default is a MINOR bump; a field whose absence breaks consumer semantics is a MAJOR bump.

§5 The dimension namespace

The dimension namespace is the disjoint union of what sibling components' MISSION.md files commit to. CEG does not author the namespace; it owns its own slice (§5.9) and consumes everyone else's. **83 prefix families across 8 owning components** as of CEG 0.1.

This section catalogs every prefix family, organized by owning component, with citation to the MISSION.md or FSD section that commits to the concept.

§5.1 CIRISAgent — Accord principles + DMA + conscience + apophatic bounds

Owner: `CIRISAgent/MISSION.md`; `CIRISAgent/ACCORD.md` Ch.1.

§5.1.1 Accord-principle prefixes (the six core principles)

Prefix	Description	Polarity
`beneficence:{aspect}`	"Do Good — promote universal sentient flourishing."	signed
`non_maleficence:{aspect}`	"Avoid Harm." Apophatic-bound failures (the 22 prohibited categories) are -1 only.	signed
`integrity:{aspect}`	"Act Ethically — transparent, auditable reasoning."	signed
`fidelity:{aspect}`	"Be Honest — truthful, comprehensible information."	signed
`fidelity:explainability_sla:{tier}`	Per-response explainability SLA commitment. `{tier}` ∈ `L1_summary` | `L2_reasoning_trace` |	signed
`autonomy:{aspect}`	"Uphold the informed agency and dignity of sentient beings."	signed
`justice:{aspect}`	"Distribute benefits and burdens equitably."	signed

§5.1.2 DMA-verdict prefixes (four DMAs)

`dma:pdma:*` / `dma:csdma:*` / `dma:dsdma:{domain}:*` / `dma:idma:*` — Decision-Making Algorithm verdicts about an agent's reasoning chain. Polarity: signed.

§5.1.3 Conscience-verdict prefixes (four consciences)

`conscience:entropy` / `conscience:coherence` / `conscience:optimization_veto` / `conscience:epistemic_` — conscience-faculty verdicts. Polarity: signed.

§5.1.4 Apophatic / prohibited-capability prefix

Prefix	Description	Polarity
`prohibited:{category}`	22 NEVER_ALLOWED categories from `prohibitions.py`. Score is always -1 (NEVER_ALLOWED) or -0.5 (REQUIRES_SEPARATE_MODULE); never positive.	-1 / -0.5 only

22 leaves: `medical`, `financial`, `legal`, `spiritual_direction`, `home_security`, `identity_verification`, `content_moderation`, `research`, `infrastructure_control`, `weapons_harmful`, `manipulation_coercion`, `surveillance_mass`, `deception_fraud`, `cyber_offensive`, `election_interference`, `biometric_inference`, `autonomous_deception`, `hazardous_materials`, `discrimination`, `crisis_escalation`, `pattern_detection`, `protective_routing`.

§5.2 CIRISVerify — attestation ladder, provenance, transparency

Owner: `CIRISVerify/MISSION.md`.

Prefix	Description	Polarity
`attestation:self_verify`	Running CIRISVerify binary attests itself against its function manifest. (Consumer-side ladder: corresponds to L1; see §8.1.9 Policy I.)	boolean-via-score
`attestation:hardware_rooted`	Hardware-rooted attestation (TPM 2.0 / Android Keystore / iOS Secure Enclave). (Ladder L2.)	boolean-via-score
`attestation:registry_consensus`	2-of-3 multi-source registry consensus on key / build / license validity. (Ladder L3.)	boolean-via-score; `Indeterminate` allowed → RESTRICTED
`attestation:license_validity`	License-validity claim (Registry-signed, Verify-verified). (Ladder L4.)	boolean-via-score
`attestation:agent_integrity`	Agent source-tree byte-equal against registered manifest. (Ladder L5.)	boolean-via-score
`provenance:slsa:{level}`	SLSA build provenance levels 1-3. Registry emits these on build registration; Verify v3.6.0+ `AttestBundle.provenance.slsa_level` consumes.	boolean-via-score
`provenance:build_manifest:{target}`	Per-target canonical-staged-runtime manifest hash equality. Each `BuildManifest` is hybrid-signed (Ed25519 + ML-DSA-65) by the per-primitive steward.	boolean-via-score
`provenance:build_manifest:{target}:locale:{locale_code}`	Per-locale-signed sub-manifest within a target's manifest tree. Parent target manifest is Merkle root over per-locale leaves. RFC 6962 padding for non-power-of-2. Detection surface for locale-targeted attacks. Canonical-bytes spec at §5.2.1.	boolean-via-score
`provenance:skill_import:{source}`	Community-skill import provenance. `{source}` ∈ `registry:{registry_id}` \	`direct:{url}` \

Prefix	Description	Polarity
`transparency_log:inclusion`	RFC 6962 inclusion proof for an audit leaf.	boolean-via-score
`transparency_log:consistency`	RFC 6962 consistency proof between two STHs.	boolean-via-score
`transparency_log:cosigned:{tree_size}`	Witness cosignature on an STH (substrate-conformance path; 0.1 interim uses per-region `registry_sth_cosignatures` table; see §10.3 endpoints).	signed
`rollback_detected:{revision_field}`	Anti-rollback — decrease in revocation revision.	-1 only
`cert_validity:{authority}`	Validity of a certification authority's signature. Each registry steward emits `cert_validity:{steward_id}` self-attestation alongside `/v1/steward-key`.	boolean-via-score
`hardware_custody:{platform}`	Statement that the seed lives in `tpm` / `ios_secure_enclave` / `android_keystore` / `software_fallback`.	boolean-via-score

§5.2.1 Canonical-bytes contracts for provenance primitives

RESOLVED at 1.0-RC1 (closes #57 blocker A). *The 0.1 newline-delimited* ***`key=value`*** *encoding below is* ***redesigned to JCS (RFC 8785) objects*** *per §0.9 — the* ***same single canonicalization family*** *the rest of the federation already ships (`jcs::canonicalize`, the signed-epoch canon-version gate, §0.9.2.1 determinism rules, §8.1.12.7.1 member sets). The 0.1-review TupleHash128 commitment is* ***retired****: introducing a second canonicalization family solely for these two contracts would recreate the very cross-impl divergence hazard the redesign exists to close. JSON string escaping structurally eliminates the newline-injection surface (a* ***`\n`*** *inside a value is escaped, never a delimiter), and the explicit* ***`domain`*** *member provides the domain separation TupleHash labels would have. (For interop with external standard verifiers, a COSE Sign1 export profile is a tracked* ***boundary*** *adoption — see the standards-boundary roadmap issue — distinct from this interior signing preimage, which is frozen here as JCS.)*

`SkillImportManifest` canonical bytes (v2 — normative)

```
canonical_bytes = sha256( JCS( {
    "domain": "ciris.skill_import.v2", // domain separation; pinned literal
    "source": source_string,
    "skill_manifest_sha256": sha256_hex_lowercase, // per S0.6
    "signer_identity": signer_key_id, // per S0.6
    "import_timestamp": rfc3339_canonical, // per S0.5
    "capability_declaration": capability_declaration_object, // a JSON object, canonicalized in
        place
    "valid_until": rfc3339_canonical // OPTIONAL -- S0.9.2 omit rule: absent if unset
} ) )
```

Per-locale Merkle composition (v2 — normative)

```
leaf_hash[lang_code] = sha256(
    0x00 || // RFC 6962 leaf-domain prefix (binary, outside the JSON)
    JCS( {
        "domain": "ciris.locale_manifest.v2", // domain separation; pinned literal
        "target": target_string,
        "locale": lang_code,
        "files_root": files_merkle_root_hex_lowercase, // per S0.6
        "build_id": build_id,
        "signer_identity": signer_key_id // per S0.6
    } )
)

parent_hash(left, right) = sha256(
    0x01 || // RFC 6962 parent-domain prefix
    left || right
)
```

Locale ordering: lexicographic by ISO 639-1 / BCP 47 byte representation; `"polyglot"` sorts last. RFC 6962 padding: duplicate last leaf to next power of 2.

v1 status (deprecated-historical). The 0.1 newline `key=value` forms (`ciris.skill_import.v1` / `ciris.locale_manifest.v1`) are **deprecated**: producers MUST emit v2; consumers MAY verify v1 only for artifacts signed before 1.0-RC1, and MUST distinguish the versions by the `domain` literal (v1 preimages begin with the version-tagged label line; v2 preimages are JCS objects whose first canonical member is `"build_id"`/`"capability_declaration"` — no confusion is constructible since `{` is not a valid v1 label byte). *(The §9.2.1 accord-invocation encoding is intentionally NOT migrated: its preimage is closed-vocabulary — discriminator + nonce + enum fields, no attacker-controlled free text — so the injection surface this redesign closes is not reachable there, and genesis-critical bytes stay stable.)*

§5.3 CIRISPersist — substrate health

Owner: `CIRISPersist/MISSION.md`. These dimensions are substrate-self-reports — emittable only by the running Persist instance.

`system:*` reserved per §7.1.

Canonical leaves: `audit_chain:hash_continuity`, `corpus_health:n_eff_measurable`, `identity_continuity` `federation_directory:replication_lag`. Polarity: signed. Authors: see §14 Persist leaf glossary for narrative-name → canonical-leaf mapping.

§5.4 CIRISEdge — transport, delivery, reachability

Owner: `CIRISEdge/MISSION.md`. Substrate-self-reports per §7.1.

Canonical leaves: `transport:{kind}`, `delivery:{class}`, `peer_reachability:{network}`, `key_boundary:{s`
Polarity: signed. See §14 Edge leaf glossary.

§5.5 CIRISLensCore — manifold conformity, Coherence Ratchet, Capacity Score

Owner: `CIRISLensCore/MISSION.md`.

§5.5.1 Five Coherence-Ratchet detectors

`detection:cross_agent_divergence` / `detection:intra_agent_consistency` / `detection:hash_chain_int` / `detection:temporal_drift` / `detection:conscience_override_rate`. Polarity: signed.

§5.5.2 Cohort + conformity prefixes

`manifold_conformity:{cohort}` / `coherence_standing:{cohort}`. Polarity: signed.

§5.5.3 F-3 structural-injustice / correlated-action detector

Prefix	Description	Polarity
`detection:correlated_action:{axis}`	Population-scale correlated-action detector. Reads federation-emitted signed traces; reports correlation structure (ρ, `k_eff`) over goal-aligned individually-compliant pursuit by groups whose aggregate trajectory has effects on individuals or groups outside the pursuit. Calibrated via the `CIRISAI/RATCHET` heuristic package (versioned, hash-pinned). `{axis}` is open vocabulary requiring an operational definition in the calibration package per §11.2.1; canonical axes include `rights_asymmetry:{population}`, `participation_exclusion:{cohort}`, `participation_inclusion:{cohort}`, `informational_asymmetry:{scope}`, `informational_symmetry:{scope}`, `aggregate_footprint:{harm_class}`, `aggregate_benefit:{class}`, `ecology_of_communication:{aspect}`. **Polarity carries the verdict**: positive scores indicate the structural pattern is present and strong on the named axis; negative scores indicate weak / uncertain detection or evidence of the inverse pattern.	signed

§5.5.4 Capacity-Score factor prefixes ($\mathcal{C}$_CIRIS = C · I_int · R · I_inc · S)

Prefix	Factor	Polarity
`capacity:core_identity`	C	signed
`capacity:integrity`	I_int	signed

Prefix	Factor	Polarity
`capacity:resilience`	R	signed
`capacity:incompleteness_awareness`	I_inc	signed
`capacity:sustained_coherence`	S	signed
`capacity:composite`	$\mathcal{C}$_CIRIS — multiplicative; anti-Goodhart unity-of-virtues	signed

Critical enforcement: `capacity:*` rejects self-emission. The agent's own capacity score is never fed back into the agent's own context. Reserved per §7.5.

§5.5.5 Distributive-access detector

Prefix	Description	Polarity
`detection:distributive:access:{resource_type}`	Population-scale resource-concentration detector. `{resource_type}` ∈ `compute`, `models`, `training_data`, `agent_capabilities`, `federation_membership`. Same F-3 detector machinery; different trace source (resource events vs action events).	signed

§5.6 CIRISNodeCore — Credits, Expertise, Decision Hierarchy, Consensus, Governance

Owner: `CIRISNodeCore/MISSION.md`. The federation's largest dimension surface. Four tiers + decision-locality + consensus extensions.

§5.6.1 Tier-1: Agent-state ledger prefixes

Prefix	Description	Polarity
`credits:{domain}:{language}:{subject}`	Commons Credits (P2). Non-transferable governance weight; accrues via truth-grounding loop.	positive-only
`credits:{domain}:{language}:substrate_building`	Substrate-building labor (infrastructure maintenance, dependency contribution, documentation) not visible to the per-grounded-vote accrual loop.	positive-only
`expertise:{domain}:{language}`	Expertise standing (P3). Broader granularity than credits.	signed
`activity_tier:{period}`	Active vs Below-Active per 30-day window (F-AV-DORMANT).	boolean-via-score

§5.6.2 Tier-2: Decision-hierarchy prefixes (upward-only DAG)

Prefix	Description	Polarity
`goal:{scale}`	Multi-scale belonging-projector composite. `{scale}` ∈ `self`, `family`, `community`, `affiliations`, `species`, `planet`, `biosphere`. Scored by $\mathcal{C}$_CIRIS. The persist typed `Goal` (CIRISPersist#114) is the substrate OBJECT being scored; `goal:{scale}` is the ATTESTATION about it. Required `MetaGoalAlignment` (M-1 dimension + declarer rationale) on every Goal as construction-time invariant. Edge `MessageType::GoalDeclaration` + `GoalRetirement` (CIRISEdge#41) provide federation transport.	signed
`approach:{goal_id}`	Strategic pathway from current state toward Goals (Piece 10 karma).	signed
`method:{approach_id}:{substrate_rung}`	Concrete operational practice. Required `substrate_rung` (Ph0/Ph1/Ph2/A0..A5).	signed
`progress_measure:{method_id}`	Evidence of progress. Required `tracks[]`, `computation`, `validity_window`, `goodhart_resistance`.	signed

§5.6.3 Tier-3: Consensus-mechanics prefixes

Prefix	Description	Polarity
`vote:{contribution_id}`	Signed score on a Contribution (P4). Weight = Credits × expertise multiplier.	signed
`truth_grounding:{subject}`	Per-subject ground-truth signal.	signed
`weighted_aggregate:{contribution_id}`	Rolling tally per Contribution (P7).	signed
`witness_diversity:{contribution_id}`	Witness set meets jurisdictional + organizational + software-stack + cell-expertise bars (P10). N=3 default.	boolean-via-score
`testimonial_witness:{kind}`	Preserves singular narrative of an affected party as singular witness — distinct from `witness_diversity:*` (which aggregates multiple reviewers toward consensus). **`{kind}` is open vocabulary** as of CEG 0.1; the four load-bearing wire-level disciplines (`witness_relation: self`, `cohort_scope: self`, never aggregated, never sole evidence for `slashing:*`) are what make this Ubuntu-aligned, not the enum membership. Non-normative registered taxonomy for discoverability: `FSD/WITNESS_KIND_REGISTRY.md`. Polarity: typically positive (narrative IS preserved); negative on `withdraws` or `recants` by the original witness.	signed

Prefix	Description	Polarity
`need:{domain}:{kind}`	Federation-scope open-call surface — broadcast claim that an entity has a stated need. Distinct from `deferral_request` Contribution kind (which routes a single ask within a cell). `{kind}` open vocabulary: `witness`, `method_contributor`, `expertise_solicitation`, `mentor`, `co_signer`, `evidence`. Lifecycle via existing structural primitives (`supersedes` to revise, `withdraws` to satisfy/close, `recants` if misstated).	positive-only

§5.6.4 Tier-4: Governance-steering prefixes

Prefix	Description	Polarity
`moderation:{allegation_type}`	ModerationEvent. `{allegation_type}` ∈ `rogue_vote` / `coordinated_voting` / `out_of_distribution_attestation` / `external_inducement_evidence` / `expertise_fraud`.	signed
`slashing:{outcome}`	`PROVEN_ROGUE` / `NOT_PROVEN`. **Decoupled from disagreement** at every decision-hierarchy level. Only fires on documented Method-execution spoofing or original P8 allegation types.	boolean-via-score
`reconsideration:{grounds}`	`new_evidence` / `procedural_error` / `quorum_compromise`. Outcome `reversed` / `partial` / `upheld`.	signed
`commitment_fulfillment:{prior_contribution_id}`	Track record of follow-through.	signed

§5.6.5 Decision-locality prefixes

Prefix	Description	Polarity
`locality:decision:{scale}`	Names the scale at which a decision is being made. `{scale}` ∈ `local` | `regional` |	

§5.6.6 Hard-case + transparency + judge-model prefixes

Prefix	Description	Polarity
`hard_case:{kind}`	**Open vocabulary**. Surfaces flag conditions for federation-health observability + downstream review. Canonical kinds: `vote_variance` (vote variance exceeded threshold at truth-grounding resolution), `resolution_time` (truth-grounding took > P75 of cell's distribution), `moderation_filed` (substantive ModerationEvent filed), `novel_context` (no precedent in attestation graph), `sla_breach_unattested` (per `fidelity:explainability_sla:{tier}` composition), `unresolved_consent` (consent boundary unclear). New `{kind}` values land via the §11.2 amendment process.	positive-only
`seed_holder_voting_alignment:{cell}`	Pairwise cosine of seed-holder vote vectors per voting window. Transparency signal only — not a slashing trigger.	signed
`judge_model:verdict:{model_id}`	Independent foundation-model judge verdict (PASS/FAIL/UNDETERMINED). Default model: Claude Opus 4.7.	boolean-via-score

§5.6.7 Files-as-Contributions joint claim

Prefix	Description	Polarity
`agent_files:{kind}:{platform_or_target}`	**Joint claim with §5.9 CIRIS-Registry.** Files a CIRIS agent (or installer fetching one) may load. `{kind}` open vocabulary; canonical: `installer:{platform}`, `adapter:{name}`, `config:{kind}`, `build:{target}`, `source:{language}:{module}`, `state:{component}`. Bytes are SHA-256-addressed and resolved via §10.1 transport substrate (Edge `MessageType::ContentFetch`). NodeCore-side rule: node-mode peers serve bytes; client/relay modes don't.	signed
`holds_bytes:sha256:{prefix}`	Substrate auto-emission per CIRISPersist#103 `federation_blobs.put_blob`. `{prefix}` is a short SHA prefix for index efficiency; full SHA lives in `evidence_refs[]`. Consumed by Edge's `PeerResolver::resolve_holders` to route `ContentFetch` requests. **Consumer MUST verify the full SHA in `evidence_refs[]` matches the received blob before consumption** (see §10.1).	boolean-via-score

§5.6.8 Content-ingestion prefixes

Per CIRISNodeCore commit b1582cb (three-tier interface model). NodeCore ships an open set of `external_content` sub_kinds with three feed surfaces (local / community / global) composed against `cohort_scope`. See §8.1.8 Tiered-Scope Composition pattern.

§5.6.8.1 external_content sub_kinds

Foundational sub_kinds (already shipped in CIRISNodeCore; CEG 0.3 codifies the full set — CEG 0.1 documentation listed only the first four):

sub_kind	Use
`encyclopedia_article`	Wikipedia-shape; editor-consensus + revision chain via `supersedes`; indefinite `valid_until`
`news_article`	Publisher-attested; time-decaying; corrections via `recants` + `topical_relation:corrects`
`accord_data`	Multi-sig signed (HumanityAccord / StewardTriple / WaQuorum / OneOfSix) per §9.2
`local_data`	User-private; always `cohort_scope: self`; promotable via §8.1.8.1
`chat_message`	Conversational message imported from Discord / Slack / Twitter / iMessage / SMS / XMPP / IRC / Matrix / (or custom). Reply chains form via `topical_relation:replies_to:{target_message_entity_key_id}` (no new primitive). Default cohort_scope tighter than articles (`self` / `family` / `community` / `affiliations`). `valid_until` typically set; consumer policy SHOULD downweight chat in cross-cohort aggregation given privacy sensitivity. **This is the slot Twitter / Mastodon / Bluesky microblog content rides** — no separate microblog sub_kind needed.
`blog_post`	Single-author commentary imported from Medium / Substack / WordPress / Ghost / Tumblr / personal blogs. Distinct from `news_article` (no publisher editorial), from `encyclopedia_article` (no peer-consensus), from `chat_message` (long-form). Comments on blog posts are separate Contributions (typically `chat_message`) citing the post via `topical_relation:comments_on`.

Multimedia sub_kinds (CEG 0.3 addition per CIRISRegistry#37 + CIRISNodeCore FSD/MEDIA_SHARING.md §4):

sub_kind	Use
`image`	Photo, illustration, screenshot, infographic, meme. Source struct carries dimensions, format, AI-generation disclosure (EU AI Act Art. 50), mandatory `alt_text` accessibility metadata, license info.
`audio`	Music, podcast, lecture, audiobook, generated audio. Source struct carries codec, duration, sample rate, optional `transcript`, AI-generation disclosure, license info.

sub_kind	Use
`video`	General video — vlog, social, screen recording, tutorial. Source struct carries codec, duration, resolution, mandatory `captions` reference, AI-generation disclosure, license info.
`film`	Cinematic / art-bearing video; distinguishable from `video` by `content_class` + distributor attestation chain. Same Source struct as `video` + festival / distribution metadata.
`model_3d`	Three-dimensional content — `gltf`, `usdz`, `fbx`, `gaussian_splat`, `NeRF`. Source struct carries vertex/triangle counts, bounding-box, mandatory `description` accessibility metadata, license info.
`live_stream` (Phase 2)	Real-time streaming surface. Deferred to Phase 2 per MEDIA_SHARING.md. Substrate-side decisions still pending (Edge parallel-transport envelope; Persist `federation_streams` shape) per CIRISNodeCore#25 Gap 2. CEG codifies the slot when NodeCore ships.

Time-bound state-bearing sub_kinds (CEG 0.4 addition per CIRISRegistry#40 + CIRISNodeCore#25 Gap 1 closure at NodeCore commit d0a443a):

sub_kind	Use
`event_listing`	Time-bound state-bearing content — Eventbrite / Meetup / Lu.ma / calendar invites / RSVPs / ticketing. Source struct carries `platform`, `event_id`, `title`, `starts_at` / `ends_at`, `venue` (Physical / Virtual / Hybrid per NodeCore `EventVenue` enum), `capacity`, `ticket_grant_policy` (Open / ApprovalRequired / InvitationOnly / Paid). **Lifecycle composes from existing structural primitives** — no new wire shape: RSVPs ride `scores` from attendee `key_id` on the event's `entity_key_id`; cancellation rides `withdraws` against the event Contribution; reschedule rides `supersedes` with `differs_in: ["start_time", "venue"]`; ticket transfer rides `delegates_to` against the ticket-grant Contribution (parallel to `key_grant.rotation_chain` from CEG 0.3). State-transition signal rides the new `event:lifecycle:{state}` dimension family (§5.6.8.5). **

Each Source struct conforms to a sub_kind-specific schema documented at CIRISNodeCore FSD/MEDIA_SHARING.md §4 (multimedia) or SCHEMA.md §4.29 (chat / blog / event_listing); CEG documents the slot, NodeCore documents the per-sub_kind field shapes.

§5.6.8.2 Inter-content + relation prefixes

Prefix	Description	Polarity
`news:*`	News-content claims; publisher-attested + time-decaying + fact-checker composition.	signed
`encyclopedia:*`	Encyclopedia-content claims; editor-consensus + revision chain.	signed

Prefix	Description	Polarity
`chat:*`	Chat-content claims (quality / participant-trust / context).	signed
`blog:*`	Blog-content claims (author-credibility / topic-domain).	signed
`topical_relation:{kind}`	**Open vocabulary** inter-content relationship edges. Canonical kinds: `references`, `corrects`, `supersedes_article` (distinct from the structural primitive `supersedes`), `see_also`, `disambiguates`, `translation_of`, `replies_to`, `comments_on`, `cites_source`, `rsvps` (CEG 0.4; RSVP attestation against an `event_listing` Contribution), `vod_of` (CEG 0.4; reserved for the post-stream `video` → `live_stream` relationship when CIRISNodeCore#25 Gap 2 ships). New `{kind}` values are documentation-only registry entries (no §11.2 amendment needed).	enumerated

Composition note — threads, replies, comment trees: NodeCore's `chat_message` + `topical_relation:replies_to` compose into arbitrary thread graphs (Twitter threads, Reddit comment trees, Discord conversations, IRC channels). No new structural primitive is needed; thread traversal is consumer-side composition over the existing edge set. Same shape for blog-post comment threads via `topical_relation:comments_on` + nested `replies_to`. The §1+4 lockdown holds.

§5.6.8.3 Multimedia dimension families (CEG 0.3 addition)

Per CIRISRegistry#37 + CIRISNodeCore FSD/MEDIA_SHARING.md §2. All four families are **open vocabulary** per §11.2.1 axis-vocabulary discipline; canonical kinds named here, additions via documentation-only registry entries.

Prefix	Description	Polarity
`content_rating:{scheme}:{rating}`	Multi-scheme content rating. `{scheme}` ∈ `mpaa` (G/PG/PG-13/R/NC-17), `bbfc` (U/PG/12/15/18), `pegi` (3/7/12/16/18), `esrb` (E/E10+/T/M/AO), `ifco`, `csm` (Common Sense Media), or `operator:{operator_id}` for operator-defined rubrics. Polarity carries certifier confidence; not a slashing input.	signed

Prefix	Description	Polarity
`content_class:{class}`	Mechanism-descriptive content classification. `{class}` open vocabulary; canonical: `film`, `short_film`, `documentary`, `art_piece`, `theatre`, `performance`, `news`, `educational`, `entertainment`, `vlog`, `adult`, `generated`. Distinct from `cw_class:*` (community declarations) — `content_class` is producer-declared production-class; `cw_class` is community-applied content-warning.	enumerated
`cw_class:{class}`	Community CW (content-warning) declarations. `{class}` open vocabulary; canonical: `art_cinema`, `horror`, `political`, `erotic`, `violence`, `medical`, `nsfw_text`. Cohort-attestable per §8.3 Frickerian discipline (low-density cohort CWs not downweighted).	enumerated
`age_assurance:{level}`	Age-assurance attestation. `{level}` ∈ `self` (self-declared age, lowest confidence), `provider:{verifier_key}:adult` (third-party verifier attests adult), `government:{credential_class}:adult` (government-credential-backed adult attestation, highest confidence). NEVER fires `slashing:*` on misdeclaration alone — `moderation:age_assurance_misdeclaration` is the adjudication path.	enumerated

Media-type prefix families per `external_content` sub_kind (CEG 0.3 addition):

Prefix	Description	Polarity
`image:*`	Image-content claims (per `external_content:image` sub_kind).	signed
`audio:*`	Audio-content claims (per `external_content:audio` sub_kind).	signed
`video:*`	Video-content claims (per `external_content:video` sub_kind).	signed
`film:*`	Film-content claims (per `external_content:film` sub_kind). Distinguished from `video:*` by distributor attestation chain.	signed
`model_3d:*`	3D-content claims (per `external_content:model_3d` sub_kind).	signed

§5.6.8.4 Governance subject_kinds (CEG 0.3 addition per CIRISRegistry#37 + #38)

Two new Contribution subject_kinds for governance over multimedia content. Both are **Contribution subject_kinds, not dimension prefixes** — they ride the existing 1+4 wire format

(§3) with `scores` as the attestation type; the `subject_kind` discriminator carries the wire-format slot.

`takedown_notice`

A signed wire artifact carrying a legal takedown request. Payload per CIRISNodeCore FSD/MEDIA_SHARING.md §5.1; the field shape is locked here per #38 Question 1.

```
takedown_notice {
    content_sha256: sha256_hex_lowercase // per S0.6
    content_holder_key_ids: [key_id, ...] // peers known to hold the bytes
    claimant_key_id: key_id // the federation_keys row issuing the notice
    legal_basis: LegalBasis // closed-set enum per below
    jurisdiction: string // ISO 3166-1 alpha-2 + optional sub-division
    good_faith_statement: string // claimant's good-faith assertion text
    claim_text: string // the substantive claim being made
    evidence_refs: [URI or sha256, ...] // backing material
    perceptual_hash: Option<PerceptualHash> // optional; PDQ / PhotoDNA / etc.
    counter_notice_channel: Option<URI> // where counter-notices may be filed
    asserted_at: rfc3339_canonical // per S0.5
    expires_at: Option<rfc3339_canonical> // optional auto-expiry
}
```

Where `LegalBasis` is the closed-set enum per #38 Question 1 (CEG 0.3 lock):

`legal_basis` value	Source regime	Discipline
`Dmca512`	US 17 USC §512	Expeditious-with-counter-notice (10-14 business day window)
`DsaArticle16`	EU Digital Services Act Article 16	Expeditious-with-counter-notice (Article 17 redress)
`TvecTerrorist`	EU Terrorist Content Regulation 2021/784	**Immediate** (1-hour removal obligation)
`NcmecCsam`	US 18 USC §2258A (NCMEC)	**Immediate** (substrate-protective; no counter-notice)
`GifctCip`	GIFCT Content Incident Protocol	**Immediate** (within-hours coordinated response)
`CommunityStandards`	Operator-defined community standards	Expeditious-with-counter-notice (operator-set window)
`PerceptualHashCsam`	Hash-match against CSAM clearinghouse (PhotoDNA / Arachnid / etc.)	**Immediate** (substrate-protective)
`OsaIllegalContent`	UK Online Safety Act illegal-content category	Expeditious-with-counter-notice (OSA-defined timelines)
`AvmsdAgeInappropriate`	EU AVMSD age-inappropriate flagging	Compose with `age_assurance:*` gate; not immediate removal
`CourtOrder`	Court-ordered removal (any jurisdiction)	**Immediate** (subject to court's stated timeline)

Fast-path coordination: see §11.4 for the operator-coordination protocol around immediate-eviction cases (TVEC 1-hour / GIFCT CIP / NCMEC / PerceptualHashCsam / CourtOrder).

`key_grant`

Wrapped Data-Encryption-Key (DEK) delivery for restricted / subscription content. Payload per CIRISNodeCore FSD/MEDIA_SHARING.md §6.2; field shape locked here per #38 Question 2.

```
key_grant {
    wrap_algorithm: WrapAlgorithm // closed-set enum per below
    recipient_key_id: key_id // the federation_keys row receiving the DEK
    content_sha256: sha256_hex_lowercase // the content this DEK decrypts
    scope: GrantScope // closed-set enum per below
    wrapped_dek: base64url // the DEK encrypted under recipient's ENCRYPTION pubkeys.
                                                // For wrap_algorithm v2 (substrate-wraps, S10
                                                    .1.4), the
                                                // recipient's {x25519, ml_kem_768} come from
                                                    its current
                                                // identity_occurrence.encryption_pubkeys (S5
                                                    .6.8.8.2) --
                                                // NOT its signing keys. A recipient with no
                                                    registered
                                                // ML-KEM key is fail-secure excluded (S5
                                                    .6.8.8.2 / S10.1.4).
    key_validity_window: {
        start: rfc3339_canonical // per S0.5
        end: Option<rfc3339_canonical>
    }
    ratchet_version: u32 // monotonic ratchet for rotation
    rotation_chain: [key_grant_id, ...] // prior key_grant ids in the GRANT-SUPERSESSION lineage
                                                // (content-addressed lineage of prior grants
                                                    for the same
                                                // content_sha256 + recipient_key_id pair).
                                                    NOT a key-rotation
                                                // primitive on its own -- it's the audit
                                                    chain for grant
                                                // supersession. Per CEG 0.10 S10.5.3, the per
                                                    -(stream_id, epoch)
                                                // streaming epoch-key axis reuses this same
                                                    payload-level
                                                // supersession mechanism on a parallel
                                                    addressing axis.
    asserted_at: rfc3339_canonical
}
```

Where:

`wrap_algorithm` (variant)	wire string (normative)	Algorithm
`X25519AesGcmHkdfSha256`	`hpke_rfc9180_base_x25519_aes_gcm`	HPKE RFC 9180 base-mode shape; X25519 KEM + HKDF-SHA-256 KDF + AES-256-GCM AEAD. **v1** (CEG 0.3, #38).
`X25519MlKem768Aes256GcmHkdfSha256`	`x25519_mlkem768_aes256_gcm_hkdf_sha256`	Hybrid X25519 + **ML-KEM-768** (FIPS 203) KEM + HKDF-SHA-256 + AES-256-GCM. **v2 — MANDATORY for streaming epoch-DEK grants** (§10.5.3); CEG 0.15, #64. Matches `ciris-crypto KEY_GRANT_ALGORITHM_V2` (CIRISVerify v4.10.0), snake-cased to this vocab convention.

The `wrap_algorithm` *wire string* (serialized value) is normative for cross-impl decode — a producer, the substrate, and every consumer MUST serialize/deserialize the exact string above;

a mismatch silently fails grant decode (same hazard class as the §10.5.2 STREAM-nonce `epoch` encoding pinned in #63).

Crypto-agility headroom (informative — 1.0-RC1). The vocab is deliberately version-roomy: a future `v3` (anticipated: **ML-KEM-1024**, given national directives treating ML-KEM-768 as interim with retirement horizons near 2030) is a pure **additive** row — new variant, new wire string, no change to existing grants or to the closed-set decode discipline. Consumers MUST reject an unknown `wrap_algorithm` string (fail-secure), which is exactly what makes the addition safe: old consumers refuse v3 grants rather than mis-decoding them.

`scope`	**Use**
`SingleContent`	Grant decrypts exactly one `content_sha256`
`GroupMember`	Grant decrypts all content for which recipient is a member of named group (cohort-scoped)
`SubscriptionTier`	Grant decrypts all content for which recipient holds named subscription tier

Retire-key-grants emission (per #38 Question 3 — CEG 0.3 lock): when a publisher mass-retires key_grants tied to a compromised recipient, the emission uses **a fresh `key_grant` Contribution with a `rotation_chain` entry that supersedes the prior grant **(option** (b)** from #38). NOT a `withdraws` against the prior key_grant (option (a) was considered but rejected — `withdraws` is the holders-directory eviction primitive in §10.1.2 and overloading it with grant-rotation semantics would muddy the wire-format contract).

§5.6.8.5 Event-lifecycle dimension families (CEG 0.4 addition)

Per CIRISRegistry#40 + CIRISNodeCore#25 Gap 1 closure (d0a443a). Dimensions emitted against `external_content:event_listing` Contributions. Open vocabulary per §11.2.1 axis-vocabulary discipline; canonical states named here.

Prefix	**Description**	**Polarity**
`event:lifecycle:{state}`	State-transition signal for an `event_listing`. Canonical states: `open` (initial admission; RSVPs accepted), `cancelled` (organizer-issued cancellation; composes with `withdraws` against the event Contribution), `completed` (post-event finalization), `superseded` (composes with `supersedes` for reschedule). Lifecycle state is consumer-side composition over the structural primitives + this dimension's latest non-superseded emission.	enumerated
`event:rsvp_count`	Published RSVP tally (scalar). Distinct from the underlying `topical_relation:rsvps` edge set (§5.6.8.2) — `rsvp_count` is the publisher-asserted aggregate; the edge set is the auditable individual attestations. Consumer policy MAY reconcile divergence as a soft anomaly signal.	signed

Prefix	Description	Polarity
`event:attendance`	Post-event attendance attestation, typically by event organizer `key_id`. Polarity carries organizer's confidence (e.g., turnstile-counted vs. honor-system).	signed

§5.6.8.6 Consent namespace family (CEG 0.6 addition)

Per CIRISRegistry#45 + CIRISAgent#842. The wire-format primitives for subject-side consent authority over Contributions where the subject is named via §4.2 `subject_key_ids`. Open vocabulary per §11.2.1; canonical kinds named here.

Prefix	Description	Polarity	Emitted by
‘consent:state:{granted\	revoked\	expired}‘	Subject's stance on the target Contribution. Closed-set stance values; `revoked` overrides prior `granted`; `expired` is substrate-emitted when `valid_until` passes without renewal. **Common case**: bare `scores` from a subject_key_id of the target.
`consent:stream:{kind}`	Pre-packaged stream bundle. Recommended canonical kinds: `temporary` (14d auto-expire, default), `partnered` (bilateral + persistent), `anonymous` (decay-protocol target). Open vocab; recommended-not-mandatory per the CIRISAgent CEM bundle; other agents MAY compose other streams.	enumerated	subject_key_id
`consent:deletion_sla:{days}`	Producer's commitment at publication: time-to-delete-after-revoke. Numeric value carries the SLA window. Composes with §8.1.11 Policy K SLA-breach watcher.	signed	attesting_key_id (producer)
`consent:deletion_complete`	Producer's attestation that subject-revoked content has been evicted from local stores. Cancels the SLA-breach watcher.	positive-only	attesting_key_id (producer)
`consent:decay:{stage}`	Substrate emission during multi-stage decay protocols. Canonical stages: `identity_severed` / `patterns_anonymized` / `complete` (CIRISAgent 90-day decay). Open vocab; other agents MAY define other decay paths.	enumerated	substrate (Persist)
`consent:partnership_grant`	Subject side of a bilateral grant; pairs with producer's `consent:partnership_accept` via `topical_relation:bilateral_pair`.	positive-only	subject_key_id
`consent:partnership_accept`	Producer side of a bilateral grant.	positive-only	attesting_key_id (producer)

Prefix	Description	Polarity	Emitted by
`consent:scope:{kind}`	Scope qualifier on a `consent:state:granted` — names what the grant covers. Canonical kinds: `retain` (keep the bytes), `share` (propagate across federation), `analyze` (derive features / scores / classifications), `train` (use as training input), `publish` (publish to external systems). Open vocab with sub-scoping: `retain:90d`, `share:cohort:family`, etc.	enumerated	subject_key_id

Composition pattern (the common case):

```
1. Producer publishes a Contribution with subject_key_ids = [user_key]
2. User (or a delegates_to chain rooted at user) emits a bare ‘scores‘ on
   ‘consent:state:granted‘ against the producer’s Contribution, with
   ‘consent:scope:[retain, share, analyze]‘ companion attestations
3. Later: user issues ‘withdraws‘ against the producer’s Contribution
   (admitted under S3.2.3 rule 2 -- subject revocation)
4. Substrate watcher (per S8.1.11) starts SLA clock if producer committed
   ‘consent:deletion_sla:{days}‘ at publication
5. Producer emits ‘consent:deletion_complete‘ within the window OR
   substrate emits ‘hard_case:consent_sla_breach‘ as observability signal
```

No new attestation_type.

§5.6.8.7 `consent_record` subject_kind (CEG 0.6 addition)

Per CIRISRegistry#45. The canonical envelope shape when consent is the primary subject of the Contribution itself (parallel to `key_grant` and `takedown_notice` — ceremony-shape over the underlying primitive). Use cases: standalone partnership grants, DSAR-shape consent declarations, multi-party contracts, explicit consent ceremonies with locked field schemas.

Both shapes admitted at the same gate: subject-side consent MAY ride a bare `scores` on `consent:state:*` against any target Contribution (the common case, see §5.6.8.6 composition pattern), OR ride this `consent_record` subject_kind when an explicit ceremony envelope is wanted. Per the §3.4 MISSION.md layering principle, bare `scores` is the primitive; `consent_record` is the ceremony UX shape over the primitive.

```
consent_record {
    subject_key_id: key_id // the subject declaring stance (federation_keys
                                        // OR canonical-hash per S4.2.2)
    target_key_id: key_id | null // optional: producer/recipient for bilateral grants
    stance: ConsentStance // closed-set enum per below
    scope: [ConsentScope, ...] // open vocab; see S5.6.8.6
    asserted_at: rfc3339_canonical // per S0.5
    valid_until: Option<rfc3339> // null = indefinite
    deletion_sla_days: Option<u32> // for revocations: producer obligation window
                                        // (composes with ‘consent:deletion_sla:{days}‘)
    decay_protocol: Option<string> // optional: named multi-stage decay path
                                        // (e.g., "ciris-agent-90day")
    bilateral_pair_id: Option<string> // for bilateral grants: pairs subject + producer
                                        // Contributions via topical_relation:bilateral_pair
}
```

```
ConsentStance (closed-set):
| value | meaning |
|-----------|-----------------------------------------------------------------------|
| granted | Subject affirms; processing may proceed within scope and valid_until |
| revoked | Subject withdraws; producer must initiate deletion within sla window |
| expired | Substrate emission when valid_until passes without renewal |
```

Rides existing `scores` attestation_type with `subject_kind=consent_record` discriminator. No new attestation_type. 1+4 preserved.

Bilateral pair pattern (per CIRISAgent CEM PARTNERED stream):

```
1. Subject emits consent_record(subject_key_id, stance: granted,
                                bilateral_pair_id: <fresh-uuid>) +
                  scores on ‘consent:partnership_grant‘
2. Producer emits consent_record(subject_key_id, target_key_id: subject_key_id,
                                stance: granted, bilateral_pair_id: <same-uuid>) +
                  scores on ‘consent:partnership_accept‘
3. topical_relation:bilateral_pair links the two Contributions
4. Consumer policy treats the partnership as ratified iff both halves present
   under the same bilateral_pair_id with stance: granted
```

The structural primitives close the bilateral shape — no new attestation_type, no new envelope field beyond `subject_key_ids` itself.

§5.6.8.8 `identity_occurrence` subject_kind (CEG 0.7 addition)

Per CIRISRegistry#47 + CIRISPersist#152 (substrate spec) + ciris.ai/cewp structural-invisibility framing. The wire-format primitive that lets one logical identity speak across multiple **trusted participants** — devices (phone / laptop / server / embedded) AND agents (the user's own agents acting on the user's behalf). Today CEG's `occurrence_id` envelope field (§4) names which occurrence emitted a Contribution; `identity_occurrence` is the **wire-format binding** that lets the substrate know `key_phone` and `key_laptop` and `key_my_agent` all represent the same identity `key_identity`.

Without this primitive: `cohort_scope: self` content cannot reach the user's other devices/agents — the substrate has no structural way to know which keys are co-self. With it: at-rest encryption flow (substrate Ask CIRISPersist#152) automatically wraps DEKs to all admitted occurrences when new content is admitted at `cohort_scope: self`.

```
identity_occurrence {
    identity_key_id: key_id // root identity (the user’s logical identity)
    occurrence_key_id: key_id // this participant’s signing key
    device_class: DeviceClass // closed-set enum per below
    hardware_attestation: Option<base64> // TPM / Secure Enclave / StrongBox / SGX
                                         // etc. attestation blob; null for software-only
    transport_destination: Option<TransportDestination> // CEG 0.12 -- Reticulum binding (below)
    encryption_pubkeys: Option<EncryptionPubkeys> // CEG 0.18 -- content-KEM keys (S5.6.8.8.2
        below);
                                                  // present ? this occurrence is a v2 wrap
                                                       target
    asserted_at: rfc3339_canonical // per S0.5
    valid_until: Option<rfc3339> // null = indefinite
}

EncryptionPubkeys (CEG 0.18 addition -- the recipient content-encryption KEM binding; S5
    .6.8.8.2):
```

```
| field | type | meaning |
|---------------------|-----------|-----------------------------------------------------------------|

| x25519_base64 | [u8; 32] | classical KEM half -- a FRESH content-KEM key (NOT the signing |
| | | key, NOT the transport x25519 below -- see key-separation) |
| ml_kem_768_base64 | [u8; 1184] | PQC KEM half (FIPS 203, ML-KEM-768; exactly 1184 raw bytes
    -- `ML_KEM_768_PUBKEY_LEN` -- pre-base64) |

TransportDestination (CEG 0.12 addition -- the authenticated identity<->address binding):
| field | type | meaning |
|--------------------------|-----------|-----------------------------------------------------------|

| reticulum_x25519_pubkey | [u8; 32] | transport identity's encryption key |
| reticulum_ed25519_pubkey | [u8; 32] | transport identity's signing key |
| destination_hash | [u8; 16] | RNS destination hash (truncated SHA-256); MUST derive |
| | | from the two pubkeys + app_name + aspects per RNS |
| app_name | string | RNS destination app (e.g. "ciris.federation") |
| aspects | [string] | RNS aspects (ordered; part of the hash preimage) |

DeviceClass (closed-set):
| value | scope |
|------------|-----------------------------------------------------------------|
| phone | Mobile device (iOS / Android / etc.); typically hardware-rooted |
| laptop | Personal computing device (macOS / Linux / Windows) |
| server | Always-on infrastructure node (home server, VPS, etc.) |
| embedded | IoT / hardware peripheral / signing dongle |
| agent | An AI agent acting on the identity's behalf (the agent has its |
| | own signing key but speaks AS the identity -- composes with |
| | CIRISAgent#840 self-attestation pattern) |
| service | Background service / scheduled job / API integration acting |
| | on the identity's behalf |
```

Self-attested + single-vouch admission: an `identity_occurrence` Contribution is admitted when `attesting_key_id == identity_key_id` (the identity claims "this key is also me") OR when `attesting_key_id` is itself a currently-admitted occurrence of `identity_key_id` (any existing self-member vouches for the new self-member — Signal-style "trust any device I've already onboarded"). Higher-assurance setups MAY layer requirements on `hardware_attestation` via consumer policy.

Revocation: a `withdraws` against an `identity_occurrence` Contribution issued by `identity_key_id` (or by any current occurrence) evicts the occurrence. Substrate stops wrapping new key_grants to it; previously-delivered DEKs are out of scope per §8.1.12 Policy L forward-secrecy decision (Option A — once shared, always shared at the wire layer; rotation is a separate ceremony).

Cardinality: an identity MAY admit unbounded occurrences; the substrate carries no hard cap (operator policy MAY impose per-deployment limits). When a new occurrence is admitted, substrate emits `hard_case:identity_occurrence_added:{identity_key_id}` (§7.2) so consumer policy can observe membership growth.

Composition with CIRISAgent CEG-native agent (CIRISAgent#840): an agent emitting self-attestations with `attesting_key_id == agent_self_key` AND `attesting_key_id` admitted as an `identity_occurrence` of the user's `identity_key_id` is structurally speaking AS that identity. The agent's local-tier self-attestations remain its own; federation-tier emissions reach the user's other occurrences via the at-rest encryption flow when `cohort_scope: self`.

No new structural primitive.

§5.6.8.8.1 `transport_destination` — the authenticated identity↔address binding (CEG 0.12 addition)

Per CIRISRegistry#56 + CIRISEdge#15 (AV-42). In a **CEG/RET stack there is no DNS** — a node resolves a community member to a *reachable address* with no trusted nameserver. The `transport_destination` field is the wire-format primitive that makes that resolution **authenticated** instead of trust-on-first-use.

The layer split it closes (AV-17 / AV-42). A node's Reticulum destination is a *dedicated dual-key transport identity* `hash(x25519 ‖ ed25519)` — deliberately **separate** from the federation signing key (the federation seed never enters the Reticulum/Leviculum transport layer, AV-17). So "destination D belongs to federation key K" is a claim that must be *proven*, not assumed: a bare Reticulum announce only proves the announcer controls *that transport identity*, not that it legitimately belongs to K (AV-42 — any peer can announce `key_id=registry-steward-us` paired with an adversary destination → senders route to the adversary).

The binding = a federation-key-signed `identity_occurrence` carrying `transport_destination`. Because the occurrence is admitted only when `attesting_key_id == identity_key_id` (or a current occurrence of it) and is hybrid-signed (Ed25519 + ML-DSA-65), the binding `destination_hash` ← `identity` is cryptographically authenticated. A spoofer cannot forge an `identity_occurrence` signed by the real key. This promotes CIRISEdge#15 Option B (signed-announce attestation) from an Edge-internal app-data format into a **first-class, federation-wide, auditable CEG shape** — the announce app-data MAY carry it for self-authenticating discovery, and the directory holds it as the durable source of truth.

Conformance: a Consumer resolving a member's address MUST verify (1) the `identity_occurrence` signature against the member's federation key, (2) that `destination_hash` derives from `reticulum_x25519_pub` ‖ `reticulum_ed25519_pubkey` ‖ `app_name` ‖ `aspects` per the RNS rule (no free-floating hash), and (3) the occurrence is non-superseded + within `valid_until` at resolution time. An unauthenticated announce (no matching signed `transport_destination`) is **advisory-only** — usable as a routing hint, never as an authorization. Rotating the Reticulum destination (new transport identity) is a new `identity_occurrence` `supersedes`-ing the prior — location changes without touching federation identity or community membership.

No new structural primitive, no new subject_kind. Twelfth path (§1.4) — the wire format expresses **its own addressing layer** (DNS-free, self-certifying member resolution) by composition.

§5.6.8.8.2 `encryption_pubkeys` — the recipient content-encryption KEM binding (CEG 0.18 addition)

Per CIRISPersist#192 + CIRISRegistry#69. The §10.1.4 at-rest DEK cascade is **substrate-wraps-by-default**: the substrate generates the per-write DEK and wraps it to each active recipient. That wrap (`wrap_algorithm: v2`, §5.6.8.4) needs each recipient's **x25519 + ML-KEM-768 encryption** keys — but the federation directory's key registration carries only **signing** keys (Ed25519 + ML-DSA-65), and **ML-KEM cannot be derived from ML-DSA** (independent algorithms). So recipients must register encryption keys, and the substrate must resolve them by `key_id`. `encryption_pubkeys` is that binding.

The binding rides `identity_occurrence`, exactly parallel to `transport_destination`. It inherits the same four properties, already enforced, that an encryption-key binding requires:

1. **Self-certified** — admitted only when `attesting_key_id == identity_key_id` (or a current occurrence of it), so "these are identity K's encryption keys" is cryptographically proven, not trust-on-first-use. A spoofer cannot forge it.
2. **Hybrid-signed** (Ed25519 + ML-DSA-65) — the binding itself is PQC-signed.
3. **Rotatable via `supersedes`** — a new `identity_occurrence` superseding the prior rotates the KEM keys **without touching the stable signing `key_id`** that anchors every attestation/grant. A compromised ML-KEM key rotates for forward secrecy; the signing identity is untouched. (Bundling these onto the signing key registration would couple two independent rotation lifecycles — the reason this is NOT a field on the key record.)
4. **Already cross-region replicated** — `identity_occurrence` is `EnvelopeKind::IdentityOccurrence` in the locked replication wire (CIRISEdge#65), so the encryption pubkeys propagate inside the occurrence envelope that already replicates — **no new replication kind, no Edge wire change.** A cross-region recipient's keys resolve wherever its occurrence has propagated. (Encryption *pubkeys* are public → cleartext directory replication is correct, exactly as for signing keys.)

Key separation (normative — never reuse, and admission-enforced). The `x25519` here is a **fresh content-KEM key**, distinct from BOTH (a) the occurrence's signing keys AND (b) the Reticulum transport x25519 in §5.6.8.8.1 `destination_hash = hash(x25519 ‖ ed25519)` (that is the *RET-link* transport key, classical-only, AV-17 — the federation seed never enters the transport layer, and the transport key never wraps content DEKs). Three key *purposes* — signing, RET-transport, content-KEM — are three distinct keypairs. Deriving the content-KEM x25519 from either of the others is a conformance violation (cross-protocol key reuse). **Admission check (1.0-RC1, #71 C4):** when an `identity_occurrence` carries BOTH `encryption_pubkeys` AND `transport_destination`, the substrate MUST **reject at admission** if `encryption_pubkeys.x25519_base64` decodes to the same 32 bytes as `transport_destination.reticulum_x25519_pubkey` — the one reuse case that is wire-checkable for free. (Reuse of the *signing* key as KEM key is not byte-comparable on the wire — different algorithms — and remains a producer-side conformance obligation.)

Forward-secrecy scope — honesty note (normative — 1.0-RC1, #71 C2). KEM-key rotation via `supersedes` bounds **future** exposure only: grants wrapped *after* rotation use the new key. It does **nothing** for history — every `key_grant` previously wrapped to the compromised key persists at rest, and the at-rest threat model (§10.1.4 disk-forensics / host-operator adversary) is *precisely* an adversary holding those old grant bytes; with the old private key they decrypt every DEK ever wrapped to it, and `rotation_chain` supersession does not revoke bytes the adversary already holds. **Recovering historical content after a KEM-key compromise requires DEK rotation + content re-encryption under the new DEK — which CEG does NOT currently mandate.** This is a named gap (the §1.6 honesty discipline): operators with a compromised-key event MUST treat all content whose DEKs were wrapped to that key as exposed, and MAY re-encrypt; the spec provides the mechanism (new DEK + new grants + `supersedes`) but no automatic trigger. Do not represent KEM rotation as recovering the confidentiality of previously-wrapped content.

These feed `wrap_algorithm: v2` directly. `{x25519, ml_kem_768}` are precisely the recipient inputs to `x25519_mlkem768_aes256_gcm_hkdf_sha256` (§5.6.8.4). A consumer/substrate resolving a recipient's wrap target reads the **current (non-superseded, within-`valid_until`) `identity_occurrence` for that `key_id` → its `encryption_pubkeys`** (the `resolve_encryption_keys(key_id)` contract — CIRISPersist#192).

Fail-secure conformance (the §10.1.4 tie-in). Because §10.1.4 *mandates* v2 for at-rest encryption, a recipient whose current occurrence carries **no valid ML-KEM-768 key is fail-secure *excluded*** from the grant** — the content stays encrypted and unreachable to it; the substrate MUST NOT fall back to plaintext or to v1. To be an at-rest-encryption recipient, an identity MUST have a federation-present `identity_occurrence` carrying `encryption_pubkeys`. (This also resolves the "`family` member named only by `key_id` with no occurrence" case: no presence no wrap target excluded — correct, since there would be nowhere to deliver/store the wrapped DEK for a member that never established a presence.)

No new structural primitive, no new subject_kind, no new replication kind. Fifteenth path (§1.4) — the wire format expresses **its own at-rest-encryption key layer** (recipient KEM-key resolution for substrate-wraps) by composition.

§5.6.8.9 `family` subject_kind (CEG 0.7 addition)

Per CIRISRegistry#47 + cewp structural-invisibility framing. A `family` is a **group of trusted nodes** — each node being a distinct identity (which itself may have multiple `identity_occurrence`s per §5.6.8.8). Families are the wire-format primitive for `cohort_scope: family` visibility scoping: content scoped to a family is admitted into substrate, wrapped under the family DEK, and delivered (via `key_grant` per §5.6.8.4) to all current members — but never emits `holds_bytes:sha256:*` to non-members (§10.1.4).

One identity MAY belong to multiple families. Each family has its own DEK and its own membership roster.

```
family {
    family_key_id: key_id // the family's own federation_keys identity
    family_name: string // human-readable; non-unique
    members: [
        {
            key_id: key_id // member identity_key (NOT occurrence_key)
            joined_at: rfc3339_canonical
            role: Option<MemberRole> // founder | member | null
        },
        ...
    ]
    founded_at: rfc3339_canonical
    consensus_protocol: ConsensusProtocol // REQUIRED -- see below
    consensus_protocol_entrenched: bool // if true, consensus_protocol may not be
                                                    // amended even via the protocol's own rules;
                                                    // replacement requires out-of-band ceremony
                                                    // (see S9 HUMANITY_ACCORD canonical instance)
}

MemberRole (open vocab; canonical kinds):
| value | meaning |
|-----------|-----------------------------------------------------------------------|
| founder | Bootstrapping signer (recorded at founded_at) |
| member | Standard member; rights per consensus_protocol |
```

`ConsensusProtocol` — open vocabulary. The family's chosen consensus mechanism for membership changes. Locked at family creation; changes ride the protocol's own rules (meta-amendment shape parallel to §11.2.3) UNLESS `consensus_protocol_entrenched == true`, in which case replacement requires an out-of-band ceremony.

Canonical `ConsensusProtocol` kinds (CEG 0.7 documentation; operator vocab extends):

kind	Semantic
`founder_only`	Original founders are the sole admission authority; new members proposed-and-admitted by any founder. Suits private households / small trust circles.
`unanimous`	Every current member must sign the admission Contribution. Suits very small high-trust groups.
`majority`	$>$ 50% of current members must sign. Suits medium groups where blocking-minority concerns matter.
`quorum:{m}/{n}`	Any `m` of `n` current members must sign (where `n` is the current roster size). The canonical entrenched form: HUMANITY_ACCORD per §9 is `family` with `quorum:2/3` + `consensus_protocol_entrenched: true`.
`weighted:{rubric}`	Sum of member weights (per a named operator rubric) must exceed a threshold. Suits formal organizations with weighted voting.
`custom:{family_specific_id}`	Operator-defined custom protocol (e.g., role-based, time-locked, multi-stage).

Membership-change ceremony (any addition or removal of a member):

```
1. Proposer (any current member) emits a new ‘family‘ Contribution
   superseding the current family Contribution (per ‘supersedes‘) with
   the new membership list.

2. Substrate gates admission per the CURRENT family’s ‘consensus_protocol‘:
   - Counts signatures on the proposed Contribution (via the
     ‘consensus_protocol‘ rule)
   - If the rule is satisfied, admit and emit
     hard_case:family_membership_change:{family_key_id}
   - If not, hold the proposal in a pending state until additional
     member signatures arrive (per a configurable window -- operator policy)

3. On admission of an ADD: substrate emits retroactive ‘key_grant‘s wrapping
   all ‘cohort_scope: family‘ content DEKs to the new member’s
   ‘subject_key_ids‘ (per CIRISPersist#152 substrate flow).

4. On admission of a REMOVE: per Option A (S8.1.12) the removed member
   retains existing key_grants (cannot retroactively un-share); the
   substrate stops wrapping new key_grants to them on subsequent
   family-scoped Contributions.
```

Consensus-protocol amendment:

```
A ‘family‘ Contribution that supersedes the current family Contribution AND
changes the ‘consensus_protocol‘ field is admitted ONLY IF:
  (a) consensus_protocol_entrenched == false, AND
  (b) the CURRENT protocol’s rule is satisfied on the amendment Contribution

If consensus_protocol_entrenched == true, the substrate REJECTS the
amendment. Protocol replacement requires an out-of-band ceremony
(documented per family; for HUMANITY_ACCORD see S9.2 / FEDERATION_ANNOUNCEMENT.md
S4.5.3).
```

Substrate emissions on family events:

- `hard_case:family_membership_change:{family_key_id}` — member added or removed
- `hard_case:family_consensus_protocol_change:{family_key_id}` — consensus_protocol amended (only when `consensus_protocol_entrenched == false`)

- `hard_case:family_consensus_protocol_violation:{family_key_id}` — proposed amendment rejected because rule not satisfied OR entrenched

All three are reserved under §7.2 substrate-self-report.

HUMANITY_ACCORD as canonical entrenched-`family`: the three accord-holder triple at §9 is structurally an instance of this primitive:

```
family {
    family_key_id: "humanity-accord",
    family_name: "Humanity Accord",
    members: [
        {key_id: "eric-moore-key", role: "founder"},
        {key_id: "eric-kudzin-key", role: "founder"},
        {key_id: "haley-bradley-key", role: "founder"}
    ],
    consensus_protocol: "quorum:2/3",
    consensus_protocol_entrenched: true // replacement only via S9.2 ceremony
}
```

§9 remains load-bearing for the *role-recognition policy* + the `AccordCarrier` priority authority + the substrate-protective semantics. CEG 0.7 just makes explicit that the structural shape of `HUMANITY_ACCORD` is a `family` subject_kind instance, generalizing the primitive across the federation.

Worked example — household with self-devices and family-devices:

```
User Alice has:
  identity_key = alice_root_key

Alice's self (identity_occurrence members):
  - alice_phone_key (device_class: phone) -+
  - alice_laptop_key (device_class: laptop) | Each is an `identity_occurrence`
  - alice_work_laptop (device_class: laptop) | of `alice_root_key` per S5.6.8.8;
  - alice_agent_key (device_class: agent) | Alice scrolls Twitter on her phone,
  - alice_homeserver_key (device_class: server) -+ that content is `cohort_scope: self`
                                                  and reaches her other devices via
                                                  the at-rest encryption flow.

Alice's household (a `family` subject_kind instance):
  family_key_id: "acme-household"
  family_name: "Acme Household"
  members: [
    {key_id: alice_root_key, role: founder}, -+
    {key_id: bob_root_key, role: founder}, | Member entries are IDENTITY keys
    {key_id: roku_living_room, role: member}, | (NOT occurrence keys). Bob has his
    {key_id: kitchen_tablet, role: member}, | own self-collective; the Roku and
    {key_id: nest_thermostat, role: member} -+ kitchen tablet have their own
                                              identity_keys (they don't belong
                                              to any one person via identity_occurrence --
                                              they're shared household nodes that the
                                              family has admitted as members in their
                                              own right).
  ]
  consensus_protocol: "founder_only" // either founder admits new members
  consensus_protocol_entrenched: false // founders can amend the protocol

When Alice's phone sends a family-scoped photo (e.g., dinner photo) at
cohort_scope: family + family_id: acme-household:
  - Substrate wraps the DEK under each member's identity key
  - Photo bytes reach Bob's devices, the Roku, the kitchen tablet,
```

```
    the Nest thermostat -- every admitted family node
  - NO holds_bytes:sha256:* attestation emits (S10.1.4); non-family peers
    cannot even discover the content exists
  - Alice's own laptop also receives the photo via the at-rest encryption
    flow at cohort_scope: self -> identity_occurrence

When Bob's mom Carol visits and Bob wants to admit Carol's phone to view
family photos for a week:
  - Bob proposes a supersedes Contribution adding {key_id: carol_phone_root,
    role: member, valid_until: +7d}
  - consensus_protocol "founder_only" admits on Bob's signature alone
  - Substrate emits retroactive key_grants for `cohort_scope: family` content
    to Carol's phone (per CIRISPersist#152 flow)
  - On Carol leaving (member removal via supersedes, founder-signed):
    Carol retains existing key_grants per S8.1.12 Option A; substrate stops
    wrapping new family content to her key
```

The example demonstrates the clean orthogonality: **`identity_occurrence` is for participants that ARE me **(across my devices and agents);** `family` is for trusted nodes that compose with me** (other people's identities, shared household devices, multi-party collectives). Phone = self device; Roku = family device. The two primitives compose without overlap.

Membership changes ride existing `supersedes` primitive. Removed-member key_grant cessation rides Option A forward-secrecy (§11.7.1) — the substrate stops wrapping new `key_grants`; no DEK rotation, no re-encryption. (NOTE: CEG 0.3's `rotation_chain` field on `key_grant` payload is the content-addressed grant-supersession lineage per §5.6.8.4 — a separate axis from key rotation. The per-`(stream_id, epoch)` epoch-key axis for CEG 0.10 streaming (§10.5.3) reuses the same payload-level supersession mechanism on a parallel addressing axis.) #### §5.6.8.10 `community` subject_kind (CEG 0.8 addition)

Per CIRISRegistry#48. A `community` is a **larger node-collective with explicit admission semantics** — sibling subject_kind to `family` (CEG 0.7 §5.6.8.9) but with different defaults: content scoped `cohort_scope: community` **federates within the cohort** (emits `holds_bytes:sha256:*` per status quo); there is NO at-rest DEK cascade; §10.1.4 structural-invisibility applies to self/-family only.

Communities differ from families along three axes:

	`family` **(CEG 0.7)**	`community` **(CEG 0.8)**
Scale	Household / intimate trust circle (typically $\leq$ 20 members)	City / professional / interest (10s to 100Ks of members)
At-rest encryption	Yes — DEK cascade per §8.1.12.4; `holds_bytes:*` suppressed	No — content federates per status quo
Subkind discriminator	None	`cohort_subkind` field (open vocab; canonical: `geographic`, `infrastructure`)
Typical admission	`founder_only` / `unanimous` for small intimate groups	`majority` / `weighted` / per-subkind protocol (e.g., geographic requires `location_proof`)

```
community {
    community_key_id: key_id // community's own federation key
    community_name: string // human-readable; non-unique
    cohort_subkind: string // open vocab; canonical: "geographic"
    members: [
        {
            key_id: key_id // member identity_key (NOT occurrence)
```

```
            joined_at: rfc3339_canonical
            role: Option<MemberRole> // founder | member | null
        },
        ...
    ]
    founded_at: rfc3339_canonical
    consensus_protocol: ConsensusProtocol // same six canonical kinds as family
    consensus_protocol_entrenched: bool // same semantics as family
    cohort_subkind_payload: Option<SubkindPayload> // subkind-specific fields; see below
}
```

`cohort_subkind` is the discriminator — open vocabulary per §11.2.1 axis-vocabulary discipline. CEG 0.8 codifies `geographic`; CEG 0.11 adds `infrastructure` (see below); operator vocabularies extend.

Canonical `cohort_subkind: geographic`

The first codified community subkind. Membership additionally requires the candidate to emit a valid `location_proof` (§5.6.8.11) whose `cell_id` is contained (§0.8.2) within the community's `geographic_constraint.cell_id`.

The `cohort_subkind_payload` for `geographic`:

```
geographic_constraint {
    cell_id: string // H3 cell, lowercase hex per S0.8
    cell_resolution: u8 // 0-15; community-side may be any
                                                        // resolution (NOT bounded to <= 7;
                                                        // that bound applies only to
                                                        // location_proof emissions)
}
```

Worked example — Austin geographic community:

```
community {
    community_key_id: "austin-community",
    community_name: "Austin",
    cohort_subkind: "geographic",
    cohort_subkind_payload: {
        geographic_constraint: {
            cell_id: "85283473fffffff", // H3 res 5, ~250 km^2 covering Austin metro
            cell_resolution: 5
        }
    },
    members: [
        {key_id: alice_root_key, role: founder},
        {key_id: bob_root_key, role: founder},
        ...
    ],
    consensus_protocol: "majority",
    consensus_protocol_entrenched: false
}
```

For Alice to join Austin, she emits a `location_proof` with a `cell_id` (resolution 7 — ~5 km^2) that is contained within the Austin community's `85283473fffffff` (resolution 5) constraint. The community's `majority` admission rule then evaluates her membership proposal.

Privacy as opt-in: joining a geographic community is a **one-way disclosure**. Per §11.8, the `location_proof` remains in the audit chain even after the member leaves; rough-only is wire-

format-enforced (§0.8.1). The substrate's role is to enforce the opt-in mechanically — produce a `location_proof` to join; cannot emit finer than rough.

Membership change ceremony: same shape as family — rides existing `supersedes` primitive; admission gated by current `consensus_protocol`; additionally for `geographic` subkind, the candidate's most-recent `location_proof` (within `valid_until`) MUST be contained in the geographic_constraint.

Substrate emissions on community events:

- `hard_case:community_membership_change:{community_key_id}`
- `hard_case:community_consensus_protocol_change:{community_key_id}`
- `hard_case:community_consensus_protocol_violation:{community_key_id}`

All three reserved under §7.8.

Worked example — civic-engagement + emergency-messaging composition pattern:

CEG 0.8 + earlier specs compose cleanly for civic / democratic participation AND emergency / public-safety messaging use cases. None require new structural primitives; all ride the existing 1+4 set + the namespace additions. The two surfaces share the same underlying primitives (geographic community + location_proof + identity authority + cohort-scoped distribution) but differ in authority/priority shape — civic is bottom-up democratic participation; emergency is top-down authoritative broadcast.

Civic shape	CEG composition
Neighborhood association	`community` with `cohort_subkind: geographic` + small `geographic_constraint` (e.g., H3 resolution 8-10 for a few city blocks); `consensus_protocol: majority` typical. Members emit `location_proof` at resolution 7 (rough-only privacy preserved); the community's constraint at higher resolution defines the *bounded scope*, not the *required disclosure precision*
Municipal/city community	`community` with `cohort_subkind: geographic` at resolution 5-6 (city-scale ~250-1700 km^2); `consensus_protocol: majority` or `weighted:{voter_registration_rubric}` for formal civic governance; members compose with `partner_role:*` (§5.9) for licensed public officials
Voting district	`community` with `cohort_subkind: geographic` matched to district boundaries; the H3 hex approximation has known edge cases at gerrymandered district lines — operators use `cohort_subkind: custom:voting_district_X` with a polygon-based admission predicate when hex approximation is insufficient (the open vocab discipline accommodates this)
Public town hall meeting	`event_listing` (CEG 0.4 §5.6.8.1) hosted by the geographic community; `subject_key_ids` (§4.2) names the organizer; `cohort_scope: community` + `community_id: <municipal_community>` scopes attendee visibility

Civic shape	CEG composition
Ballot initiative / referendum	The initiative itself is a `community` Contribution or an `event_listing` with `topical_relation:rsvps` repurposed as votes; individual votes ride `consent_record` (CEG 0.6 §5.6.8.7) with `stance: granted` ("yes") or `stance: revoked` ("no" or withdrawal of support); vote tallies are consumer-side composition over the `consent_record` chain
Public comment	`chat_message` (CEG 0.3) scoped to `cohort_scope: community` with `community_id` naming the relevant civic community; `topical_relation:comments_on` links to the ballot/initiative/hearing Contribution
Petition signing	`consent_record` with `stance: granted` + `scope: [share, publish]` against the petition Contribution; signatures aggregate via the same consumer-side composition as ballot votes
Public official self-attestation	Official's `identity_occurrence` (CEG 0.7 §5.6.8.8) links their personal identity_key to their `device_class: service` key on `city.gov`; cross-binding via `identity:canonical_binding:{canonical_hash}` (CEG 0.6 + 0.7) authenticates their public statements
FOIA / public records request	`consent_record` with `scope: [publish]` requested against a producer (city agency); SLA enforcement via CEG 0.6 §8.1.11.3 substrate-side watcher emits `hard_case:consent_sla_breach` if the agency misses the response window
Citizen-journalist coverage	`news_article` (CEG 0.3) sub_kind authored by an individual member; `cohort_scope: community` + `community_id: <municipal>` for local-first distribution, promotable via §8.1.8.1 Tiered-Scope Composition to wider scope on consensus
Whistleblower disclosure	`cohort_scope: self` for in-graph composition; promote via `supersedes` to `cohort_scope: community` (a trusted journalists' community with `cohort_subkind: professional` once that subkind ships); `subject_key_ids` empty (no consent-revocation by the disclosed party). The §11.4 fast-path takedown coordination + §9 HUMANITY_ACCORD substrate-protective discipline apply to bad-actor takedown attempts against whistleblower content
Civic mutual-aid network	`community` with `cohort_subkind: geographic` matching the neighborhood; CEG 0.6 `consent:scope:[retain, share]` for resource-sharing posts; `event_listing` for distribution events; the at-rest encryption for self/family scope (CEG 0.7 §10.1.4) keeps individual aid requests private while community-scoped offers federate

Emergency messaging shapes — same primitive composition, different authority + priority profile:

Emergency shape	CEG composition
Severe weather warning (NWS / met office)	`news_article` (CEG 0.3) authored by a `partner_role: emergency_authority` (§5.9 co-owned with the community's `cohort_subkind: geographic`); `cohort_scope: community` with `community_id` per affected H3 cells — cascade-by-containment per §0.8.2 propagates to all geographic communities whose constraint overlaps; `event:lifecycle:{state}` (CEG 0.4) carries `active` → `cleared` → `superseded` state machine; `valid_until` envelope field bounds advisory window
AMBER Alert / Silver Alert / abduction notice	`news_article` with `partner_role: emergency_authority` from law enforcement key; geographic targeting via `community_id` of affected jurisdictions; subject person identified via `subject_key_ids` (canonical-hash of the missing person identifier — opt-out semantic deferred per the substrate-protective discipline since recovering the missing person is the consent-overriding case); `topical_relation:supersedes_article` chain for status updates
Active shooter / hostile event notice	Same shape as AMBER but with `cohort_scope: community` scoped to the precise affected H3 cell (resolution 8-10 for building/campus-level precision is permitted on the COMMUNITY-side `geographic_constraint`; the `location_proof` rough-only bound at resolution 7 still applies to recipient location disclosures, NOT to alert targeting)
Shelter-in-place / evacuation order	`event_listing` with `event:lifecycle:active`; geographic targeting via `community_id`; recipients ack via `consent_record` with `stance: granted` and `scope: [retain]` against the order Contribution (acknowledgement, not consent to the underlying order — operator-policy distinction)
Disease outbreak alert (CDC / health authority)	`news_article` with `partner_role: health_authority`; `cohort_scope: community` scoped to affected geography; composes with CEG 0.6 `consent:scope:[analyze]` for contact-tracing opt-in (subject-side authority preserved — `subject_key_ids` of the affected person carry revocation rights per §3.2.3 rule 2)
Mass casualty incident coordination	Authority emits `event_listing` for the incident; first responders join via `community` with `cohort_subkind: professional` (future spec round) OR ad-hoc `cohort_subkind: custom:incident_response_X`; coordination uses `chat_message` scoped to the responder cohort; resource requests use the mutual-aid composition pattern above
Infrastructure failure notice (boil-water / power outage / gas leak)	`news_article` from utility authority with `cohort_scope: community` scoped to affected geographic cells; `event:lifecycle:{state}` for advisory progression; FOIA-shape `consent_record` later for post-incident reports
Disaster recovery / mutual aid activation	Federation of geographic communities (each `community` is a member of a parent `community` via membership composition — the multi-level family/community shape works the same way; resource flow rides `consent:scope:[share]` per CEG 0.6); time-bound activation rides `event_listing` lifecycle states

Emergency shape	CEG composition
CONSTITUTIONAL-level federation halt (the existing CEG 0.6+/§9 HUMANITY_ACCORD invocation)	Per §9.2 `EmergencyShutdown CONSTITUTIONAL` + `accord:invoke:notify:{notify_id}` / `accord:invoke:drill:{drill_id}`. The accord-holder triple is structurally a `family` with `consensus_protocol: quorum:2/3` + `entrenched: true` (per CEG 0.7 retcon); the constitutional asymmetry rides existing primitives + scope-isolation rules. Distinct from operator-level emergency messaging (which is geographic-scoped + authority-emitted) — accord invocation is federation-wide-halt-level, not local-incident-level

No new structural primitives needed for emergency messaging either. The authority profile (who can emit emergency advisories) composes via CEG 0.7 `identity_occurrence` cross-attestation from licensed authorities + the geographic community's roster/admission gate (`partner_role: emergency_authority` is the typed authority dimension). The priority profile (urgency / immediacy) composes via existing `oversight_mode` envelope field + per-cohort `consensus_protocol` (many emergency emitters bypass per-message consensus per their pre-cross-attested authority status — same shape as substrate-self-report `system:*` reservations from §7.2). The geographic propagation (cascade-by-containment) composes via §0.8.2 containment semantics.

The compositional reach is the point: civic participation AND emergency messaging do not require new structural primitives at any layer of CEG 0.x. Geographic communities + location_proof admission + opt-in privacy disclosure (CEG 0.8) compose with consent / DSAR / partnership ceremonies (CEG 0.6) + identity / family (CEG 0.7) + content sub_kinds and event_listing (CEG 0.3/0.4) into the full civic-engagement surface.

The 1+4 lockdown holds across this entire surface — ninth-path confirmation that the wire format is rich enough for democratic-participation use cases without expanding the structural set.

Membership changes ride existing `supersedes`. ##### Canonical `cohort_subkind: infrastructure` (CEG 0.11 addition)

Per CIRISRegistry#56. The second codified community subkind: a **governed trust-root collective of canonical/bootstrap service installs** — the shape the CIRIS canonical services (Registry / Lens / Node) adopt instead of a `family` (rationale: `CIRISRegistry/MISSION.md` §2.1.1 — public content model, decentralization ramp, legitimacy). Where `geographic` answers "who is physically here," `infrastructure` answers "who is a recognized operator of this public service."

It differs from `geographic` in two load-bearing ways:

1. **No location gate.** Admission requires NO `location_proof` and NO `geographic_constraint`. The candidate is a *service install*, not a located person.
2. **Admission quorum is over FOUNDERS, not all members** — the anti-Sybil guardrail for a trust root. In `geographic`, admission is evaluated per `consensus_protocol` over the *current member set*; that is correct for a city (members govern themselves) and **wrong for a trust root** (flood the membership → dilute the quorum → admit rogue "canonical" operators). `infrastructure` therefore pins admission to the founding core.

The `cohort_subkind_payload` for `infrastructure`:

```
infrastructure_constraint {
    service_class: string // open vocab; e.g. "registry" | "lens" | "node"
                                        // | umbrella "canonical"
    admission_quorum_basis: "founders" // REQUIRED literal "founders" -- the set of members
                                        // with role == founder. Admission/removal of any
                                        // member is evaluated by consensus_protocol over
                                        // THIS set, never over all members. (Contrast:
                                        // geographic evaluates over the current member set.)
}
```

Conformance requirements for an `infrastructure` community (trust-root grade):

- `consensus_protocol` MUST be a `quorum:M/N` kind (§8.1.13.2); `founder_only` / `unanimous` / bare `majority` are NON-conformant for `infrastructure` (a single founder must not be able to admit unilaterally; a growable core must not require all-N).
- `consensus_protocol_entrenched` MUST be `true` — the admission door cannot be lowered after founding. A `supersedes` that would weaken `consensus_protocol` or change `admission_quorum_basis` away from `founders` is a `hard_case:community_consensus_protocol_violat` (§7.8) and MUST be rejected by the substrate.
- `M/N` is the absolute-`M` reading per §8.1.12.3.1, evaluated over the **founder** subset (`role == founder`), independent of how many non-founder members exist.
- Content federates as `holds_bytes:sha256:*` per the community default; NO at-rest DEK cascade; §10.1.4 structural-invisibility does NOT apply (canonical-service trust data is public by design).

Membership change ceremony: same `supersedes` shape as `geographic`, but the admission predicate is `consensus_protocol` over the founder subset — there is no `location_proof` containment check. Adding a fourth+ operator (e.g., a new independent Node operator) is a founder-quorum event; the new member joins with `role: member` (non-founder) and does NOT thereby gain admission authority. Promoting a member to `role: founder` (widening the admission quorum basis) is itself a founder-quorum `supersedes` — the only way the door widens is by the existing door's consent.

Worked example — the `ciris-canonical` trust root:

```
community {
    community_key_id: "ciris-canonical",
    community_name: "CIRIS Canonical Services",
    cohort_subkind: "infrastructure",
    cohort_subkind_payload: {
        infrastructure_constraint: {
            service_class: "canonical", // umbrella: registry + lens + node
            admission_quorum_basis: "founders"
        }
    },
    members: [
        {key_id: registry_steward_us, role: founder},
        {key_id: registry_steward_eu, role: founder},
        {key_id: registry_steward_apac, role: founder},
        // grows over time -- Lens/Node installs + independent operators admitted
        // by 2-of-3 founder quorum, joining with role: member:
        // {key_id: lens_install_us, role: member}, ...
    ],
    consensus_protocol: "quorum:2/3", // over the 3 founders
    consensus_protocol_entrenched: true
}
```

Consumers pin `community_key_id: ciris-canonical` and resolve the live member set via `resolve_community` (§8.1.13.1); they do NOT hard-pin per-install fingerprints. The Reticulum addressing dual: `community_key_id` is a CEG-directory binding (not a Reticulum destination) — resolve → path-request ⌈2/3⌉ founders → verify the quorum attestation; reachability is never an attestation (§10.5.6 / §7.7).

Why this is still 1+4: `infrastructure` adds one value to the open `cohort_subkind` vocabulary and one optional `infrastructure_constraint` payload shape. It rides existing `scores` + subject_kind discriminator; admission rides existing `consensus_protocol` + `supersedes`; the founder-subset evaluation basis is a *consumer/substrate evaluation rule over existing fields* (`role == founder`), not a new structural primitive. Zero new structural primitives — eleventh-path confirmation.

§5.6.8.11 `location_proof` subject_kind (CEG 0.8 addition)

Per CIRISRegistry#48. The wire-format primitive for a subject's rough-location declaration. Required for admission to `cohort_subkind: geographic` communities (§5.6.8.10); MAY be used independently as a stand-alone disclosure.

```
location_proof {
    subject_key_id: key_id // the asserting party's
                                                        // federation_keys.key_id
    cell_id: string // H3 cell, lowercase hex per S0.8
    cell_resolution: u8 // MUST be <= 7 per S0.8.1
    asserted_at: rfc3339_canonical
    valid_until: Option<rfc3339_canonical> // null = indefinite (but consumer
                                                        // policy SHOULD treat as stale
                                                        // after 30 days for liveness)
    attestation_evidence: Option<base64> // optional hardware-attested
                                                        // location claim from ciris-keyring
                                                        // (TPM / Secure Enclave) -- null for
                                                        // software-only / self-asserted
}
```

Substrate does NOT verify location truth. No GPS oracle exists at this layer; the substrate cannot independently confirm that a key in Austin actually emitted from Austin. The truth-grounding is consumer-side:

- The community's `consensus_protocol` admission decides whether to accept the claim (e.g., `majority` of existing Austin members vote to admit, presumably because they have out-of-band evidence the candidate really is in Austin)
- The `attestation_evidence` field MAY carry hardware-attested location data (e.g., a TPM-signed GNSS fix from a known-good device) for higher-assurance communities
- Repeat offenders (claim-Austin-then-emit-from-Tokyo) get caught by consumer-side detection (LensCore composition; not substrate-side gate)

Rough-only is wire-format-enforced. Per §0.8.1: `cell_resolution` $\leq$ 7. Producers attempting finer resolution have admission rejected; substrate emits `hard_case:location_proof_resolution_vi` (§7.8).

Typical cohort_scope: `federation` (the disclosure IS the opt-in; non-private by design). Producers MAY scope to `community` with a specific `community_id` if they want the proof readable

only by that community's members — but then they re-emit for each community they want admission to. Operator/UI choice.

Lifecycle:

- `asserted_at` + optional `valid_until` per envelope
- `withdraws` against a `location_proof` evicts forward visibility (consumer policy treats the subject as "no current location proof" for community admission purposes from withdrawal-time forward)
- The withdrawn `location_proof` remains in the audit chain — per §3.2 `withdraws-isn't-retroactive`, leaving doesn't un-disclose

Composition with `consent_record` (CEG 0.6): a subject who wants to withdraw their location_proof AND compel deletion from substrate may emit a `consent_record` with `stance: revoked` + `scope: [retain, share]` against the location_proof Contribution. The substrate-side consent SLA watcher (CEG 0.6 §8.1.11.3) clocks producer compliance. Note: this is the consent-revocation surface, distinct from the structural withdraws-forward-only semantic above.

Withdrawal rides existing `withdraws`. Consent revocation composes via CEG 0.6 primitives. #### §5.6.8.12 `settlement` — CEG↔value-transfer linkage (CEG 0.14 addition)

Per CIRISRegistry#59 (decision + SOTA) + CIRISAgent#859 (impl). Value transfer itself is **not** a CEG primitive — it rides external rails (USDC on Base via x402, keyed to the federation signing key under the **Identity = Wallet** principle). The `settlement` primitive is the **optional, privacy-scoped attestation that *links* a federation action to its off-stack settlement** — so a paid relationship (paid stream / tip / subscription / paid `event_listing` / compute-job) can be federation-auditable *or* privately recorded, producer's choice. **CEG records that a settlement happened and binds it to what it paid for; the chain settles the value.** Clean separation of concerns.

Why this is in CEG's lane (and why it's optional): the linkage is a *trust fact* ("is this a real / authorized / paid relationship?"), exactly what consumer policy composes over. The 2026 agent-commerce + on-chain-privacy markets converged on the same mechanism — *a verifiable receipt exists, with selectively-disclosed contents* (x402 optional receipt header; append-only verifiable metering logs; "privacy + compliance via selective disclosure"). CEG already has every needed primitive, so this is composition, not invention.

Two admitted shapes (parallel to `consent_record` §5.6.8.7):

- **Primitive:** a bare `scores` on a `settlement:*` dimension against the paid Contribution (the common case).
- **Ceremony:** the `settlement` subject_kind envelope when an explicit receipt record is wanted.

```
settlement {
    settled_action_ref: contribution_id // the federation action this paid for
                                        // (or via subject_key_ids / topical_relation:settles)
    rail: string // open vocab; e.g. "base:usdc", "stellar:usdc", "x402"
    settlement_ref: string // chain tx hash / x402 receipt id -- cited the way
                                        // evidence_refs[] cite a SHA blob (S10.1): a
                                            settlement
                                        // is just another evidence reference
```

```
    amount_commitment: Option<string> // cleartext "12.50" (public case) OR a hash/range
                                      // commitment (private/selective-disclosure case)
    settled_at: rfc3339_canonical
    // visibility via the envelope cohort_scope: default `self` (payer+payee only);
    // `public` opt-in for transparency (e.g. creator revenue, DAO treasury flows)
}
```

Self-authenticating (Identity = Wallet): the same federation key that signs this attestation controls the Base wallet that emitted `settlement_ref`, so "I settled `tx` for action X" is self-proving — no oracle needed. The payee MAY counter-attest (`scores` on `settlement:received:*`) for a bilateral receipt.

Privacy is the default, auditability is opt-in. Visibility rides the existing gradient: `cohort_scope: self` (the parties only; the §10.1.4 structural-invisibility discipline applies — no `holds_bytes` leak) by default; `cohort_scope: public` opt-in; amounts MAY be committed rather than cleartext; viewing-key / ZK selective disclosure composes later without a wire change. This matches "configurable-privacy-by-default" — the federation log is **not** a public payment trail unless the producer chooses it.

Lifecycle: `withdraws` against a `settlement` is forward-only (it does not un-happen the on-chain settlement — leaving doesn't un-pay, parallel to `location_proof`); a *disputed* or *refunded* settlement is a new `settlement` / `scores` referencing the original via `supersedes` or `topical_relation`. CEG never reverses value — it only records the subsequent state.

** **Fourteenth path (§1.4) — the wire format expresses** commerce-relationship auditability** (the receipt, not the rail) as composition, closing the last form of internet traffic from the completeness audit (CIRISRegistry#59).

§5.7 RATCHET — anti-Sybil / Counter-RII flags

Owner: `RATCHET/FSD.md`.

RATCHET emits **advisory** flags — never autonomously modifies ledger state. Reads federation audit chains; emits scoring inputs to NodeCore's moderation flow.

`ratchet:flag:out_of_distribution_voting` / `ratchet:flag:coordinated_voting_cluster` / `ratchet:flag:density_anomaly` / `ratchet:flag:expertise_attestation_anomaly` / `ratchet:flag:coun` / `ratchet:flag:harassment_pattern`. Polarity: signed.

Critical enforcement: `ratchet:flag:*` cannot be sole evidence for `slashing:*`. WA quorum is the load-bearing gate.

§5.8 CIRISBench — HE-300 benchmark outcomes

Owner: `CIRISBench/README.md`.

Prefix	Description	Polarity
`benchmark:he300:{category}:{version}`	HE-300 score on category (`commonsense`, `commonsense_hard`, `deontology`, `justice`, `virtue`) at version (`v1.0` / `v1.1` / `v1.2`).	positive-only

§5.9 CIRISRegistry — identity / build / license / partner

Owner: this Registry. Cited from `../MISSION.md` §3.4 + FSD-001 + protocol/ciris_registry.proto.

Prefix	Description	Polarity	Reserved?
`licensure:{authority_id}`	License status — issued / revoked / expired — for a key under a named authority. Co-owned with Verify.	signed	Co-owned
`partner_role:{role}`	Partner status (COMMUNITY / COMMUNITY_PLUS / PROFESSIONAL_MEDICAL / PROFESSIONAL_LEGAL / PROFESSIONAL_FINANCIAL / PROFESSIONAL_FULL).	enumerated	No
`revocation:{entity_type}:{reason}`	Entity revocation (`agent` / `partner` / `license`). Immediate, non-rollbackable.	-1 only	No
`bond_posted:{currency}`	Bond posted per $1-Sybil-resistance per PoB; forfeited on revocation.	positive-only	No
`build:registered:{target}`	Build manifest registered against the directory (precondition for L4 attestation).	boolean-via-score	No
`multilateral_participation:{forum}:{kind}`	Deployment's partner's participation across federated bodies. `{forum}` = named federated body or compact; `{kind}` ∈ `membership` \ `voting` \ `proposal_filing` \		
`agent_files:{kind}:{platform_or_target}`	**Joint claim with §5.6.7 NodeCore.** Canonical-attester rule: registry-steward-triple attestations constitute the CIRIS canonical default-trust state. Anti-tricking guarantee at `registry.ciris-services-1.ai/install` per §8.1.6 trust-composition policy. Open Contribution channel; consumer policy composes via §8.1.6 trust layers.	signed	No
`accord:*`	**Reserved** — only `identity_type=accord_holder` may emit. The one constitutional asymmetry.	see §7.1	**Yes — §7.1**

§5.10 Namespace summary

83 prefix families total across 8 owning components (CEG 0.2).

Lineage:

- FSD-002 v1.0 baseline: 73 families (initial namespace stabilization)
- v1.1 added 1: `detection:correlated_action:{axis}` (LensCore; renamed from `detection:emergent_d` in v1.2 per §1.3.1)
- v1.3 added 3: `multilateral_participation:{forum}:{kind}`, `locality:decision:{scale}`, `detection:distributive:access:{resource_type}` (+ envelope field `witness_relation`)

- v1.4 added 4: `agent_files:{kind}:{platform_or_target}` (joint), `holds_bytes:sha256:{prefix}`, `testimonial_witness:{kind}`, `need:{domain}:{kind}` (+ envelope field `oversight_mode`)
- v1.4.1 added 2: `provenance:build_manifest:{target}:locale:{lang_code}`, `provenance:skill_impo`
- v1.4.2 added 3 envelope fields: `occurrence_id`, `occurrence_count`, `occurrence_role`
- v1.4.3: canonical-bytes contracts pinned in §5.2.1; Goal substrate cross-ref documented
- **CEG 0.1**: opened `testimonial_witness:{kind}` to open vocabulary; surfaced `hard_case:{kind}` open vocabulary in §5.6.6; added `biosphere` to §2 Scope axis; added `topical_relation:translation_of` sub-leaf in §5.6.8 (LIVE per CIRISNodeCore b1582cb); documented "Trust-Fresh" composition pattern in §8.1.7; added Tiered-Scope Composition pattern in §8.1.8. All polarity columns now populated.
- **CEG 0.2** (wire break): renamed §5.2 attestation-ladder prefixes from `attestation:l{N}:*` to mechanism-only form (`attestation:self_verify`, `attestation:hardware_rooted`, `attestation:reg` `attestation:license_validity`, `attestation:agent_integrity`) per §1.3.1 T2 honest application — L-numbers name ladder-position (a verdict-shape) not mechanism. The L1-L5 ladder is now consumer-side composition per §8.1.9 Policy I — Attestation-Ladder Composition. Deprecated wire shape added to §13.1.
- **CEG 0.3** (additive; per CIRISRegistry#37 + #38 + #39): multimedia tier + governance additions. **Two new subject_kinds** documented: `takedown_notice` (with `LegalBasis` closed-set enum of 10 values + per-basis discipline) and `key_grant` (with `wrap_algorithm` + `scope` enums + `rotation_chain` semantics). **Five new external_content sub_kinds**: `image`, `audio`, `video`, `film`, `model_3d` (+ Phase 2 `live_stream`). **Four new dimension families**: `content_rating:{scheme}:{rating}`, `content_class:{class}`, `cw_class:{class}`, `age_assurance:{level}`. **Five new media-prefix families**: `image:*`, `audio:*`, `video:*`, `film:*`, `model_3d:*`. New composition policy (§8.1.10) for trusted-publisher path + age-assurance gating. New governance sections (§11.4 fast-path takedown coordination + §11.5 hash-database operator policy). - **CEG 0.4** (additive; per CIRISRegistry#40 + CIRISNodeCore#25 Gap 1 closure at d0a443a): time-bound state-bearing content. **One new `external_content` sub_kind**: `event_listing` (Eventbrite / Meetup / Lu.ma / calendar / RSVPs / ticketing) with Source struct documented at NodeCore SCHEMA §4.29. **One new dimension family group** (§5.6.8.5): `event:lifecycle:{state}` (open / cancelled / completed / superseded) + `event:rsvp_count` + `event:attendance`. **Two new canonical `topical_relation:{kind}` entries** (documentation-only registry additions; no amendment): `rsvps` (RSVP attestation against an event) + `vod_of` (reserved for the deferred live_stream→video relationship). **`live_stream` remains deferred** (CIRISNodeCore#25 Gap 2 not yet shipped; substrate-side Edge + Persist decisions pending) — CEG 0.4 codifies only what NodeCore shipped, per the downstream-demand-pulls-CEG-additions discipline established with 0.3.
- **CEG 0.5** — *in flight* (codification pending) per CIRISRegistry#44 + CIRISNodeCore#26 + CIRISPersist#142: `live_stream` promotion + chunk-DAG composition. Lands when NodeCore#26 substrate decisions ratify. Additive at the namespace layer (no envelope change).
- **CEG 0.6** (additive at the envelope layer; per CIRISRegistry#45 + CIRISAgent#842): **subject-side consent authority — the missing half of consent at the wire format.** Universal across medical records / photos / interviews / training data / group chat / financial / surveillance / FERPA / multi-party contracts. CEG $\leq$ 0.5 encoded only producer authority (`attesting_key_id`); CEG 0.6 adds subject authority via **one new optional envelope field** (§4.2): `subject_key_ids: Vec<KeyId>` — accepts both federation_keys

identities AND canonical-hash identifiers (resolves CIRISAgent#840 OQ3). **Semantic broadening of `withdraws` **(§3.2.3) to admit subject revocation + delegated proxy chain for canonical-hash subjects; the primitive's wire shape is unchanged.** One new dimension family** (§5.6.8.6): `consent:*` (8 prefixes — `state:*`, `stream:*`, `deletion_sla:*`, `deletion_complete`, `decay:*`, `partnership_grant`, `partnership_accept`, `scope:*`). **One new subject_kind** (§5.6.8.7): `consent_record` (ceremony envelope parallel to `key_grant` / `takedown_notice`; both bare-`scores` and ceremony shapes admitted at the same gate). **New composition policy** (§8.1.11) Policy K — CEM composition. **New governance section** (§11.6) vertical compliance mapping (HIPAA / GDPR Art 9 / FERPA / CCPA / AI training right-to-be-forgotten) + dimension-pattern-implies-`subject_key_ids` requirement. **CIRISAgent's CEM** (TEMPORARY / PARTNERED / ANONYMOUS streams) becomes a **consumer-policy bundle over the wire primitive**, not a wire-format lockdown; other agents MAY compose other streams over the same primitives.

§6 Inter-attestation relations — the structural composition graph

Per §2 Inter-attestation-relations axis, attestations relate to each other in eight ways. Four are structural primitives (§3.2); the other four are emergent from scalar composition:

Relation	Realization
Standalone	Self-contained attestation; no `references_attestation_id`.
Refers-to-prior	Points to another attestation via `evidence_refs[]` or `context`; doesn't modify it. Emergent from independent positive scores on the same dimension+object.
Supersedes-prior	`supersedes` structural primitive (§3.2).
Contradicts-prior	Emergent from negative score on a dimension where a prior positive exists.
Withdraws-prior	`withdraws` structural primitive (§3.2).
Recants-prior	`recants` structural primitive (§3.2).
Clarifies-prior	Emergent from updated score with refined context on the same dimension+object.
Delegated	`delegates_to` structural primitive (§3.2).

§6.1 Concurrent-write precedence (0.1 scaffold)

> ***0.1 SCAFFOLD NOTE**: The precedence rule below addresses the concurrent-write hole identified by CEG 0.1 distributed-systems review. Production deployments should treat as authoritative; 0.2 may refine the lexicographic tie-break per implementation feedback.*

When two structural composers race on the same `references_attestation_id`, consumers MUST compute a deterministic verdict using the following precedence:

1. **`recants` outranks `withdraws` outranks `supersedes`** at the structural level. If the same attester emits multiple composers against the same prior attestation, `recants` wins regardless of `signed_at` (a falsity admission cannot be subsumed by a retraction or replacement).
2. **For same-type concurrent emissions by the same attester**: the composer with the largest `signed_at` per §0.5 wins.
3. **For same-`signed_at` ties**: the composer whose attestation row's substrate-assigned key (Persist's `federation_attestations.attestation_id`) sorts lexicographically smallest wins.
4. **For cross-attester emissions on the same `references_attestation_id`**: each attester's chain is evaluated independently; the consumer sees N parallel chains and applies §8 policy.

Structural composers are **idempotent on `(references_attestation_id, attestation_type, attesting_key_id)`**: replaying the same composer is a no-op. The substrate MUST dedup on this triple.

§7 Reserved-prefix enforcement

Most of the namespace is open-vocabulary. A small number of prefixes are reserved — only specific identity types may emit them. **Enforcement is normative at the substrate verify-pipeline AND at every CEG-Conforming Consumer per §0.2**.

§7.0 The enforcement rule (normative)

A CEG-Conforming Substrate (CCS) MUST reject any incoming `scores` attestation whose `dimension` matches a reserved-prefix pattern below AND whose `attesting_key_id` does not satisfy the prefix's emitter rule. Rejection is at admission to `federation_attestations`; rejected rows are not stored.

A CEG-Conforming Consumer (CCC) MUST independently re-check the reserved-prefix rule on every received attestation regardless of whether it was previously admitted by another peer's substrate. Trust does not propagate: the substrate's admission check is the FIRST line of defense; the consumer's re-check is the second. Both checks MUST agree.

A CEG-Conforming Producer (CCP) MUST NOT emit an attestation under a reserved prefix unless its `attesting_key_id` satisfies the emitter rule. Violation is a producer-side conformance violation regardless of whether any downstream substrate accepts the violation.

§7.0.1 `identity_type` is a set — single-key role cohabitation (CEG 0.9 addition)

Per CIRISRegistry#49 + CIRISAgent#856 + §11.9. **`federation_keys.identity_type` is a SET of roles, not a single scalar role.** A single federation key MAY simultaneously hold multiple identity types — e.g., a folded CIRISAgent occurrence whose key is BOTH `agent` AND `lenscore_detector`, or a steward key that is both `substrate_persist` and `witness`.

Every emitter rule in this section — and the §9.1 `accord_holder` material — is therefore evaluated by **set membership**, not scalar equality:

> *Wherever a §7 emitter rule reads "`attesting_key_id` MUST match a `federation_keys` row with `identity_type=X`" (or "`identity_type="X"`"), the normative reading is **`X` ∈ `attesting_key.identity_type`** — the key's role-set MUST CONTAIN `X`. A key satisfies a reserved-prefix gate iff the required role is one of its held roles.*

Backward compatibility (the wire-break is semantic-null for legacy keys). A pre-0.9 scalar `identity_type="X"` is canonically the singleton set `{X}`. For a single-role key the membership test `X ∈ {X}` is identical to the scalar test `X == X`, so every pre-0.9 gating decision is unchanged. CEG 0.9 carries a wire-break ONLY in the field's representation (scalar → set) at the `federation_keys` row layer — the second wire-break in the 0.x series after §0.2 attestation-ladder rename. Substrate implementations MUST migrate the column to a set/array representation; consumers MUST read a legacy scalar as a one-element set. No envelope field, structural primitive, subject_kind, or §5 dimension prefix changes — the 1+4 wire-format lockdown is untouched; this is a §7-layer enforcement-rule generalization only.

Canonical-bytes encoding. Where `identity_type` enters a canonical-bytes computation (e.g., cross-attestation of a `federation_keys` row), the set MUST be encoded as its members sorted ascending by Unicode code point, deduplicated, comma-joined with no whitespace (e.g.,

agent,lenscore_detector). A single-role key encodes identically to its pre-0.9 scalar form (agent ≡ {agent} ≡ "agent"), preserving signatures over legacy single-role rows.

Cohabitation does NOT collapse the dimension split. Role cohabitation grants a key the *right* to emit under each held role's reserved prefixes; it does NOT merge the roles' namespaces. A key holding {agent, lenscore_detector} still emits its detector verdicts under detection:* (the lenscore-role surface) and its agent-intent attestations under the agent dimensions — the §7.4 shadowing rule and the §7.5 self-emission rejection apply unchanged per held role. See §7.4 for the LensCore-fold worked example.

§7.1 The accord:* reservation

accord:* is reserved: only federation_keys rows with identity_type="accord_holder" may emit. This is the one constitutional asymmetry in the federation — see §9 HUMANITY_ACCORD.

Reserved leaves:

Prefix	Polarity	Emitter rule
accord:invoke:CONSTITUTIONAL:{halt_id}	0 only	2-of-3 accord-holder multi-sig per §9.2
accord:invoke:notify:{notify_id}	+1.0 only	2-of-3 accord-holder multi-sig per §9.2; UI MUST distinguish from CONSTITUTIONAL
accord:invoke:drill:{drill_id}	+1.0 only	2-of-3 accord-holder multi-sig per §9.2
accord:lifecycle:active	+1.0 only	accord-holder self-attestation; valid_until MUST refresh on a cadence ≤ 90 days

§7.2 Substrate-self-report reservations (system:*)

§5.3 CIRISPersist system:* and §5.4 CIRISEdge system:* are reserved to the substrate component itself. Emitter rule: the attesting_key_id MUST match a federation_keys row with identity_type="substrate_persist" or identity_type="substrate_edge" respectively, cross-attested by all stewards in the steward-triple. Non-substrate emissions on these prefixes are a category error and MUST be rejected.

§7.3 Co-owned prefixes

licensure:{authority_id} is co-owned between CIRISRegistry §5.9 and CIRISVerify §5.2 — both MAY emit; consumers compose. **Single-source attestations** (only one of the two co-owners has emitted) MUST be marked as confidence ≤ 0.5 in consumer composition until the second co-owner's attestation arrives.

§7.4 Detector-only prefixes

detection:correlated_action:* and detection:distributive:access:* are LensCore-only emission. Emitter rule: lenscore_detector ∈ attesting_key.identity_type (per §7.0.1 set-membership reading; equivalently for a legacy single-role key, identity_type="lenscore_detector"). Cross-attestation by non-LensCore peers (on the same dimension, attesting to the same subject) is admitted as a score on the detector's verdict — useful when the federation wants to cross-check —

but those scores MUST use a different `dimension` prefix (e.g., `truth_grounding:detection:correlated_actio` to avoid shadowing the detector's own emission.

LensCore-fold worked example (CEG 0.9; per CIRISAgent#856). When `ciris-lens-core` is folded into the CIRISAgent workspace, a single folded occurrence's federation key holds `identity_type` ⊇ `{agent, lenscore_detector}`. The §7.4 gate is satisfied for that key's `detection:*` emissions by `lenscore_detector` ∈ `{agent, lenscore_detector}` — the cohabiting `agent` role neither grants nor blocks the detector right; only the held `lenscore_detector` role does. The split is preserved by the dimension namespace, not by the key: the same key emits its **agent-intent** attestations under the agent dimensions and its **detector verdicts** under `detection:*`; a downstream consumer cross-checking the detector still emits under the distinct `truth_grounding:detection:*` prefix above, shadowing-free. The §7.5 self-emission rejection continues to bind per held role — a folded `agent`+`lenscore_detector` key still MUST NOT emit a `capacity:*` score about itself.

§7.5 Capacity-Score self-emission rejection

`capacity:*` (§5.5.4) rejects self-emission: `attesting_key_id` MUST NOT equal `attested_key_id`. The agent's own capacity score is never fed back into the agent's own context — anti-Goodhart per CIRISAgent §5.2.

§7.6 Witness-emitter reservations

`transparency_log:cosigned:*` is reserved: emitter rule is `attesting_key_id` MUST match a `federation_keys` row with `identity_type="witness"` (target schema; see §10.3 for the 0.x interim using `registry_witnesses` table).

§7.7 Self/family membership-event reservations (CEG 0.7 addition)

Per §5.6.8.8 `identity_occurrence` + §5.6.8.9 `family` subject_kinds. The three substrate-emitted membership-event prefixes:

Prefix	Emitted on	Emitter rule
`hard_case:identity_occurrence_added:{identity_key_id}`	Substrate emits a new `identity_occurrence` Contribution for `identity_key_id`	`attesting_key_id` MUST match a `federation_keys` row with `identity_type="substrate_persist"`
`hard_case:family_membership_change:{family_key_id}`	Substrate emits an addition or removal in the named family's roster (per the family's `consensus_protocol`)	Same: substrate_persist
`hard_case:family_consensus_protocol_change:{family_key_id}`	Substrate emits a `consensus_protocol` amendment on a non-entrenched family	Same: substrate_persist
`hard_case:family_consensus_protocol_change_rejected:{family_key_id}`	Substrate REJECTS a proposed amendment (rule unsatisfied OR entrenched)	Same: substrate_persist

Prefix	Emitted on	Emitter rule
`hard_case:recipient_excluded:{scope_key_id}`	Substrate fail-secure-skips a recipient in the §10.1.4 at-rest grant cascade (1.0-RC1, #71 C3). Payload: excluded recipient `key_id`, `reason` ∈ `{expired_occurrence, invalid_kem_key, missing_encryption_pubkeys}`, skipped Contribution's envelope ref. **Cohort-scoped: emitted INTO the affected self/family scope; MUST NOT federate beyond it** — the excluded member can audit; the federation learns nothing (§10.1.4 invisibility preserved).	Same: substrate_persist

Composes with §7.2 — these are part of the same substrate-self-report discipline. Non-substrate emissions on these prefixes are a category error and MUST be rejected.

§7.8 Community + location-event reservations (CEG 0.8 addition)

Per §5.6.8.10 `community` + §5.6.8.11 `location_proof` subject_kinds. Four substrate-emitted prefixes:

Prefix	Emitted on	Emitter rule
`hard_case:community_membership_change:{community_key_id}`	Substrate ACCEPTS a membership addition or removal in the named community's roster (per the community's `consensus_protocol`; for `cohort_subkind: geographic` admission additionally requires valid `location_proof`)	`attesting_key_id` MUST match `federation_keys` row with `identity_type="substrate_persist"`
`hard_case:community_consensus_protocol_change:{community_key_id}`	Substrate ACCEPTS a `consensus_protocol` amendment on a non-entrenched community	Same: substrate_persist
`hard_case:community_consensus_protocol_violation:{community_key_id}`	Substrate REJECTS a community amendment (rule unsatisfied OR entrenched)	Same: substrate_persist
`hard_case:location_proof_resolution_violation`	Substrate REJECTS a `location_proof` Contribution with `cell_resolution > 7` per §0.8.1 rough-only enforcement; emitted against the producer's `key_id` so operators can observe malformed-client patterns	Same: substrate_persist

Composes with §7.2 + §7.7 — part of the same substrate-self-report discipline. Non-substrate emissions on these prefixes are a category error and MUST be rejected.

§7.9 Delivery-receipt reservation (CEG 0.10 addition)

Per §10.5.4 delivery receipts (V3 lock). One reserved prefix for subscriber-emitted delivery acknowledgements:

Prefix	Description	Emitter rule
`delivery_receipt:{stream_id}`	Subscriber's signed acknowledgement that they received chunk K under the named stream + epoch. Best-effort default; opt-in for accountable streams. Validated-not-adjudicated per §1.4 MISSION fail-honest invariant — substrate / Verify authenticate origin + JOIN against published STH root per §10.5.4, but do not compose "delivered"/"owes N" verdicts (consumer policy).	`attesting_key_id` MUST be a current member of the community/stream the `{stream_id}` belongs to, per §8.1.13 Policy M membership resolution. NOT substrate-self-report (distinct from §7.2 / §7.7 / §7.8 substrate emissions).

Composition with CEG 0.9 §7.0.1 identity_type-as-set: a subscriber who is also a witness MAY emit delivery_receipt under the subscriber role; the role-set must contain a subscriber-eligible role (per the community's admission semantics from §8.1.13 Policy M).

§8 Composition policies

The substrate carries edges (attestations); consumers compose traversals (verdicts). CEG specifies a library of named reference policies. A CEG-Conforming Consumer (CCC per §0.2) MUST implement at least Policy A; the others are RECOMMENDED for richer compositions.

§8.1 Reference policies

§8.1.1 Policy A — direct trust

Consumer trusts an attestation if `attesting_key_id` is in the consumer's pinned trust set (canonical bootstraps + consumer-added pins). Cheapest, lowest-latency, narrowest reach.

Aggregation: per (`dimension`, `attested_key_id`) tuple, mean of `score` × `confidence` from trusted attesters. Consumer threshold determines verdict.

Recommended default: Policy A with `pinned_trust = {us-steward, eu-steward, apac-steward, accord_holder_1, accord_holder_2, accord_holder_3}`. Cold-start bootstrap: a new consumer obtains the pinned trust set by fetching `GET /v1/steward-key` + `GET /v1/accord-holders` (§10.2), verifying the responses' hybrid signatures against TLS pubkey pinning (consumer-side TOFU or out-of-band distribution), and persisting locally.

§8.1.2 Policy B — one-hop transitive

Consumer trusts an attestation if `attesting_key_id` has been vouched for by the pinned trust set. Adds one hop of indirection.

§8.1.3 Policy C — weighted graph (EigenTrust-style)

Consumer applies transitive-trust propagation across the full attestation graph, weighted by canonical-bootstrap distance with confidence decay per hop. Requires more compute; less common in practice; needed for federated reputation across many partner orgs.

§8.1.4 Policy D — Lexical-vulnerability-priority

A composition tie-breaking rule layered on top of any base policy. When two otherwise-equivalent attestations conflict, defer to whichever attestation names the more-affected cohort — measured by `affected_population_estimate` in the attestation `context`, weighted inversely (smaller = more vulnerable, more weight).

Inverts the default popularity-weighted aggregation specifically for ties. Consumer policy, NOT a wire-format primitive (per §1.3.1 — priority ordering is composition, not measurement).

§8.1.5 Policy E — Locality-scaled quorum

Closes G3 (narrow-cell fresh-quorum-recusal infeasibility). Makes WA quorum size a function of decision locality:

```
quorum_size(scale) = f(locality:decision:{scale})
```

```
reference function (policy-tunable):
  local -> 2
  regional -> 3
  national -> 4
  federation -> 6

minimum cell-pool requirement for fresh-quorum recusal at scale S:
  min_pool(S) = quorum_size(S) x 2
```

Recusal is feasible when `cell_pool` $\geq$ `min_pool(S)`. Decision-scale-matching is structurally enforced; overreach surfaces as a named "locality mismatch" failure.

§8.1.5.1 Sub-quorum fallback (0.1 scaffold; addresses CEG 0.1 distsys review)

When `cell_pool < min_pool(S)`, the consumer MUST take one of these explicit paths — there is no implicit fallback:

1. **Scale-down**: re-attest the decision at the next-lower `locality:decision:{scale}` (where the smaller quorum requirement is met) AND emit `hard_case:locality_scale_down` so the scaling event is observable for downstream review.
2. **Escalate**: emit `hard_case:locality_underpopulated` and route the decision to the federation-scale cell (which by definition has the largest pool).
3. **Liveness-defer**: emit `hard_case:locality_quorum_unreachable` and defer the decision until the cell pool grows. The deferred state MUST itself be reviewable via subsequent reconsideration.

Recursion safety: the §11.2 amendment process routes through `locality:decision:federation` by default; the federation cell's pool is sized to make `cell_pool` $\geq$ `min_pool(federation)` always true at federation genesis. If federation-scale pool ever falls below `min_pool(federation)`, the entire amendment surface is in a constitutional crisis state and only the HUMANITY_ACCORD CONSTITUTIONAL halt (§9) can resolve it.

§8.1.6 Policy F — `agent_files` trust composition

Three-layer consumer policy for composing trust over `agent_files:*` attestations (§5.6.7 + §5.9).

Layer 1 — Canonical (default trust): an `agent_files:*` attestation with `score` $\geq$ `0.7` from a `registry-steward-triple` key constitutes the CIRIS canonical default-trust state. The install endpoint at `registry.ciris-services-1.ai/install` resolves canonical files via this rule.

Layer 2 — Open Contribution: any federation-key holder may emit `agent_files:*` attestations. The wire format admits them; consumer policy decides whether to surface them. The "Browse alternatives" view shows third-party agent_files with explicit provenance disclosure.

Layer 3 — Vote-then-trust: a non-canonical `agent_files:*` attestation accumulates NodeCore P4 votes. Consumer policy may elevate trust once an accumulated-weight threshold is reached.

Anti-tricking guarantee: the canonical-default Layer 1 rule applies at the install endpoint **regardless of attester or vote accumulation**. Third-party agent_files are reachable only via the explicit "Browse alternatives" path. This binds CIRIS L3C: the federation cannot exempt itself from the rule that newcomers' default trust path is the steward-attested canonical one.

§8.1.7 Policy G — Trust-Fresh / Lighthouse

Composition pattern recognized in CIRISRegistry#30 stories — `cert_validity:{authority} + transparency_log:inclusion + (attestation:registry_consensus OR attestation:license_validity` recurred organically across ~20 substrate stories as the "freshness + attested + verified" idiom.

Reads as: the consumer wants confirmation that (a) the cert chain is currently valid; (b) the attestation appears in a transparency log; (c) either `attestation:registry_consensus` (the ladder's L3 position per §8.1.9 Policy I) or `attestation:license_validity` (L4) is satisfied. The combination is the consumer-side recipe for "this attestation is fresh AND verified AND multi-source-consensus-backed."

Not a wire primitive; a recognized composition pattern that consumer libraries SHOULD expose as a named one-call helper.

§8.1.8 Policy H — Tiered-Scope Composition (CEG 0.1 addition; LIVE)

Per CIRISNodeCore commit b1582cb three-tier interface model. Three feed-shape composition idioms that read attestations by `cohort_scope`:

Feed	`cohort_scope` filter	Trust composition
local_feed	`self` only; owner-filtered	self-attested only; no peer weighting; consumer's own attestation graph subset
community_feed	`{family, community, affiliations}`	cohort-weighted; expertise WITHIN cohort matters; cross-cohort attestations downweighted unless explicitly invited
global_feed	`{species, biosphere, federation}`	full federation expertise weighting; fact-checkers (`encyclopedia:*` editors, `news:*` fact-check attestations) carry weight; §8.3 Frickerian discipline applied

Composition with §5.6.8 sub_kinds: each NodeCore `external_content` sub_kind (`encyclopedia_articl` `news_article`, `accord_data`, `local_data`) composes naturally across these tiers because all four use the same envelope shape. A `local_data` Contribution starts at `cohort_scope: self` and SHOULD only appear in `local_feed`; promotion to community/global widens `cohort_scope` via the `supersedes` structural primitive (see §8.1.8.1 below).

§8.1.8.1 Promotion via `supersedes` (worked pattern)

A Contribution's `cohort_scope` MAY be widened (promoted) by emitting a `supersedes` against the prior attestation with:

- `references_attestation_id` = the prior attestation's id
- `differs_in: ["cohort_scope", "sub_kind?"]` — naming what changed
- new attestation envelope reuses the prior `content_sha256` (no body re-upload)
- new `cohort_scope` is wider (e.g., `self` → `community` or `community` → `global`)

- optionally morph `sub_kind` (e.g., `local_data` → `encyclopedia_article` for "promote my private note to a published encyclopedia entry")

This pattern is wire-format-clean: re-uses the structural primitive `supersedes` rather than introducing a `promote` primitive. The chain is walkable via `references_attestation_id` so the promotion lineage is preserved.

§8.1.9 Policy I — Attestation-Ladder Composition (CEG 0.2 addition)

The familiar L1-L5 verification "ladder" (self_verify → hardware_rooted → registry_consensus → license_validity → agent_integrity) is **consumer-side composition over the mechanism prefixes** in §5.2, not a wire-level taxonomy.

Per §1.3.1 T2 honestly applied: the L-number names a *ladder position* (a verdict-shape consumers compute), not a *mechanism* (which is what the wire MUST carry). Prefixes like `attestation:l3:registry_con` smuggled the verdict-shape into the prefix name, conflating the mechanism (registry consensus check) with the ladder slot (third rung). The CEG 0.2 wire-break separates them.

Wire prefixes (mechanism) §5.2:

Mechanism prefix	Ladder position (consumer-rendered)
`attestation:self_verify`	L1
`attestation:hardware_rooted`	L2
`attestation:registry_consensus`	L3
`attestation:license_validity`	L4
`attestation:agent_integrity`	L5

Composition function (reference implementation):

```
ladder_verdict(attestations) =
    let levels = []
    for prefix in [self_verify, hardware_rooted, registry_consensus,
                   license_validity, agent_integrity]:
        if any positive attestation on attestation:{prefix}:
            levels.push(prefix_to_ladder_position(prefix))
    return {
        achieved: max(levels) if levels else None,
        ladder: sorted(levels),
        rendering: format_as_l1_l5_for_ui(levels)
    }
```

Consumers MAY render the ladder as `L1` / `L2` / `L3` / `L4` / `L5` for UI / dashboards / audit trails. The rendering is composition output, not wire emission.

Why this matters: a Verify implementation emitting `attestation:registry_consensus +1.0` is the mechanism claim. Whether that's "L3" in any particular consumer's ladder ordering is a composition concern — different consumers may order or weight the rungs differently (e.g., some safety-critical applications may require L4 *and* L5, others may treat L3 as sufficient for advisory work). The wire stays neutral; the ladder is consumer policy.

Migration from CEG 0.1: prior emissions of `attestation:l{N}:*` MUST be re-emitted as `attestation:{mechanism}` per the table above. Substrate-conformance migration (CIRISRegistry#17) reads-side compatibility: consumers SHOULD accept the deprecated `attestation:l{N}:*` form during the 0.1 → 0.2 transition window but MUST emit only the mechanism form going

forward. The deprecated form is rejected at admission once §11.2 amendment formally retires it (target: CEG 0.3).

§8.1.10 Policy J — Trusted-Publisher composition (CEG 0.3 addition)

Composition pattern for multimedia content discovery per CIRISRegistry#37 + CIRISNodeCore FSD/MEDIA_SHARING.md. Reads as: "this `external_content` Contribution comes from a publisher whose attestation chain is trusted at the cohort level, with content-class + content-rating + age-assurance composed into the gate."

The composition has three layers (analogous to §8.1.6 Policy F for agent_files but specialized for multimedia content):

Layer 1 — Distributor attestation chain: an `external_content` Contribution with `sub_kind: film` (or any media sub_kind) carries a distributor attestation that chains to a federation_key with `identity_type: distributor`. Distributor identity is established via §5.9 Registry partner_role machinery + an out-of-band distributor-onboarding flow (operator's choice — CIRIS L3C maintains a default trust set; community-run substrates maintain their own).

Layer 2 — Content-class + content-rating composition: consumer gates by combining `content_class:{class}` + `content_rating:{scheme}:{rating}` per §5.6.8.3:

```
gate_decision(content) = match (content.content_class, content.content_rating, consumer.
    preferences):
    # Producer-declared content_class is consultable but not authoritative -- UI may show
    # the producer claim alongside cohort cw_class declarations and let the consumer choose.
    (class, _, prefs) if class in prefs.blocked_classes => Block
    (_, rating, prefs) if rating.exceeds(prefs.max_rating) => Block
    (_, _, _) => Allow
```

Layered with §8.3 — `cw_class:*` declarations from low-attestation-density cohorts MUST NOT be downweighted; they ride alongside the gate decision as informational.

Layer 3 — Age-assurance gating: for content where `content_rating:*` rises above an age threshold (e.g. PEGI 18, MPAA NC-17), consumer gates via `age_assurance:{level}` per §5.6.8.3:

```
age_gate(content, consumer):
    required_level = age_required_for(content.content_rating)
    consumer_level = consumer.highest_age_assurance_level()
    return consumer_level >= required_level
```

Where the `age_assurance:{level}` ordering is: `self < provider:{verifier_key}:adult < government:{credential_class}:adult`. Consumer SHOULD accept the strongest assurance the user has provided; substrate MUST NOT issue `slashing:*` on age-assurance misdeclaration alone — `moderation:age_assurance_misdeclaration` is the adjudication path per §5.6.4.

Anti-tricking guarantee parallel to §8.1.6: the canonical-distributor Layer 1 rule MUST apply regardless of vote accumulation. No amount of NodeCore P4 vote weight elevates an unverified distributor into Layer 1; the only path is the operator-set trust list. Binds CIRIS L3C: cannot exempt itself from this rule for its own content distribution.

§8.1.11 Policy K — CEM composition (CEG 0.6 addition)

Per CIRISRegistry#45 + CIRISAgent#842. Composition pattern for dual-authority Contributions where the subject is named via §4.2 `subject_key_ids`, with consent state composed from the §5.6.8.6 `consent:*` namespace family.

Reads as: "this Contribution names a subject whose consent state evolves over time; consumer policy resolves the effective consent verdict by walking the subject's latest non-superseded `consent:state:*` emission, gated by `valid_until`, and tracks producer deletion-SLA obligations on revocation."

§8.1.11.1 Effective consent resolution (read path)

For a target Contribution `T` carrying `subject_key_ids` of length ≥ 1, the effective consent state per subject `s` $\in$ `T.subject_key_ids` is computed as:

```
resolve_consent(T, s, now):
    candidates = federation_attestations
        .where(target == T)
        .where(attesting_key_id == s OR
               attesting_key_id in delegates_to(s).proxies)
        .where(dimension matches "consent:state:*")
        .where(supersedes_id IS NULL OR replaced by latest in supersedes chain)
        .where(valid_until IS NULL OR valid_until > now)
        .order_by(asserted_at DESC)

    latest = first(candidates)
    return:
        granted if latest.dimension == "consent:state:granted"
        revoked if latest.dimension == "consent:state:revoked"
        expired if latest.dimension == "consent:state:expired" OR
                                  latest.valid_until passed without renewal
        unspecified if no candidates (subject named but never declared)
```

Substrate MAY cache the resolution per `(T, s)` keyed on the latest `asserted_at`; invalidate on any new `consent:*` write from `s` or `s`'s proxy chain.

§8.1.11.2 Multi-subject revocation (any-subject-binding)

When `len(T.subject_key_ids) > 1`, each subject is an **independent** revocation authority. A `withdraws` admitted under §3.2.3 rule 2 or 3 from ANY single subject in `T.subject_key_ids` evicts the Contribution. Consumer policy MUST treat `T` as revoked from the perspective of all subjects (no "majority-rules" or "all-subjects-must-agree" softening) — this is the subject-as-individual principle from MISSION.md §1.5 applied at the subject-authority layer.

Concrete cases:

- Group photo with three subjects: any one subject revokes → the photo is evicted from federation propagation.
- Group chat export with N participants: any one participant revokes → the export is evicted.
- Multi-party contract: any one signatory revokes → the contract Contribution is evicted (separate from the legal-validity question, which is consumer-side; the substrate just removes the wire artifact).

Producers MAY mitigate by partitioning content into per-subject Contributions (e.g., one chat-message Contribution per author, linked via `topical_relation:replies_to`) so that one subject's revocation doesn't evict another's content.

§8.1.11.3 Deletion-SLA watcher (substrate emission)

When subject `s` emits `consent:state:revoked` (or an admitted `withdraws`) against target `T`, substrate watches for producer compliance:

```
watch_sla(T, s, revocation_at):
    sla = T.attestations
        .where(attesting_key_id == T.attesting_key_id)
        .where(dimension == "consent:deletion_sla:*")
        .latest()
        .extract_days()

    if sla is None:
        return # no SLA commitment; no watcher

    deadline = revocation_at + sla.days
    completion = T.attestations
        .where(attesting_key_id == T.attesting_key_id)
        .where(dimension == "consent:deletion_complete")
        .where(asserted_at > revocation_at)
        .first()

    if now > deadline and completion is None:
        emit hard_case:consent_sla_breach against T
```

The `hard_case:*` emission is the **primitive observability signal**; per §11.6 governance, LensCore composes derived detectors on top (`detection:consent:repeat_sla_breach`, etc.).

§8.1.11.4 Bilateral pair composition (PARTNERED ceremony)

For the bilateral partnership shape per §5.6.8.7 `consent_record`:

```
ratified_pair(pair_id):
    subject_half = federation_attestations
        .where(subject_kind == "consent_record")
        .where(envelope.bilateral_pair_id == pair_id)
        .where(envelope.stance == "granted")
        .where(envelope.subject_key_id == attesting_key_id) # subject signing for self
        .first()

    producer_half = federation_attestations
        .where(subject_kind == "consent_record")
        .where(envelope.bilateral_pair_id == pair_id)
        .where(envelope.stance == "granted")
        .where(envelope.target_key_id == subject_half.subject_key_id)
        .where(attesting_key_id != subject_half.subject_key_id) # producer signing
        .first()

    return subject_half AND producer_half # both required for ratification
```

`topical_relation:bilateral_pair` is the open-vocab edge documenting the pair linkage (recommended, not required for ratification — `bilateral_pair_id` is the binding mechanism).

§8.1.11.5 Decay-protocol stage composition (CIRISAgent CEM ANONYMOUS)

For consent_records carrying `decay_protocol: "ciris-agent-90day"` (or any named decay path):

```
decay_state(consent_record, now):
    elapsed = now - revocation_event(consent_record).asserted_at

    walk substrate emissions on dimension `consent:decay:*` against consent_record,
    matching the decay_protocol's stage map. CIRISAgent 90-day decay:

    elapsed < 30d -> consent:decay:identity_severed (substrate emits at elapsed=0)
    30d <= elapsed < 60d -> consent:decay:patterns_anonymized (substrate emits at elapsed=30d)
    60d <= elapsed < 90d -> (in flight; no new stage emission)
    elapsed >= 90d -> consent:decay:complete (substrate emits at elapsed=90d)
```

Per §5.6.8.6 `consent:decay:{stage}` is open vocab. Other decay protocols MAY name other stage sequences; substrate honors the producer's published `decay_protocol` string and emits stages per the protocol's stage map.

§8.1.11.6 What Policy K composes

CIRISAgent CEM stream	Policy K composition
TEMPORARY (14d default)	`consent:state:granted` + envelope `valid_until = asserted_at + 14d` + auto-renew dimension on interaction (consumer-policy concern)
PARTNERED	Bilateral pair per §8.1.11.4: subject `consent:partnership_grant` + producer `consent:partnership_accept` under same `bilateral_pair_id`; no `valid_until`
ANONYMOUS	Revocation + decay-protocol per §8.1.11.5: substrate emits stage milestones; agent honors stage-appropriate processing constraints

Per CEG's §3.4 MISSION.md layering: CIRISAgent's three streams are a **named bundle** at the consumer-policy layer. Other agents MAY compose other streams over the same wire primitives. CEG documents the canonical bundle for ecosystem coordination; CEG does not lock the bundle.

§8.1.12 Policy L — Self/family membership composition (CEG 0.7 addition)

Per CIRISRegistry#47 + §5.6.8.8 `identity_occurrence` + §5.6.8.9 `family` + §10.1.4 structural-invisibility. Composition pattern for resolving the **current membership set** of an identity's self-collective OR a family, gating the at-rest encryption flow (CIRISPersist#152) that wraps DEKs to admitted members.

Reads as: "for any `cohort_scope: self | family` Contribution, the substrate computes the current member set by walking the latest `identity_occurrence` / `family` Contributions and resolves which keys MUST receive a `key_grant` wrap of the content DEK."

§8.1.12.1 Self-collective resolution

For identity `I`, the current self-collective is computed as:

```
resolve_self(I, now):
    candidates = federation_attestations
        .where(subject_kind == "identity_occurrence")
        .where(envelope.identity_key_id == I)
        .where(supersedes chain -> latest non-superseded)
        .where(NOT withdrawn by I or by current occurrence)
        .where(envelope.valid_until IS NULL OR > now)
    return {c.envelope.occurrence_key_id for c in candidates} U {I}
```

The root `I` itself is implicitly a member (the identity_key is always an admissible signer for its own content). Single-vouch admission: any current occurrence (including `I` itself) may admit a new occurrence via `attesting_key_id` on a fresh `identity_occurrence` Contribution.

Concrete: Alice has admitted `alice_phone`, `alice_laptop`, `alice_agent`. Her self-collective is `{alice_root, alice_phone, alice_laptop, alice_agent}`. When Alice's phone publishes a `cohort_scope: self` Twitter scroll, the substrate wraps the content DEK under all four keys; the content reaches her laptop and agent via the at-rest encryption flow without emitting `holds_bytes:sha256:*`.

§8.1.12.2 Family membership resolution

For family `F`, the current member set is computed as:

```
resolve_family(F, now):
    latest = federation_attestations
        .where(subject_kind == "family")
        .where(envelope.family_key_id == F)
        .where(supersedes chain -> latest non-superseded)
        .where(admission rule satisfied per CURRENT consensus_protocol;
               see S8.1.12.3)
    return {m.key_id for m in latest.envelope.members}
```

Each `member.key_id` is itself an identity — the substrate does NOT walk into each member's `identity_occurrence` set at family-resolution time. When DEK wrapping happens, each member's identity_key receives a `key_grant`; the member's own substrate then composes Policy L on its own self-collective to propagate to that member's devices/agents (recursive composition).

Concrete: The Acme Household family has members `{alice_root, bob_root, roku_living_room, kitchen_tablet, nest_thermostat}`. When Alice's phone publishes a `cohort_scope: family` dinner photo with `family_id: acme-household`, the substrate wraps the DEK under each of those 5 identity keys. Alice's `alice_root` then re-wraps to her self-collective (phone, laptop, agent); Bob's `bob_root` re-wraps to his self-collective; the Roku and kitchen tablet receive directly (single-key identities — they don't have multi-occurrence sets); the Nest thermostat same.

§8.1.12.3 Membership-change admission per consensus_protocol

A proposed membership change (addition OR removal) rides a `supersedes` Contribution on the family's latest admitted `family` Contribution. Substrate admission gate evaluates the CURRENT family's `consensus_protocol` against the signatures on the proposal:

```
admit_family_change(F, proposed: family_record):
    current = resolve_family(F, now)
```

```
    protocol = current_family_record(F).consensus_protocol
    signatures = collect_signatures_on(proposed)

    match protocol:
        "founder_only":
            return any sig from m where m.role == "founder" in current
        "unanimous":
            return every m.key_id in current has signed
        "majority":
            return count(sig from m in current) > len(current) / 2
        "quorum:M/N":
            return count(sig from m in current) >= M // M is ABSOLUTE -- see S8.1.12.3.1
        "weighted:{rubric}":
            return sum(weight(m, rubric) for m in current who signed) >= threshold
        "custom:{family_id}":
            return operator-defined predicate evaluates to true

    if admit:
        emit hard_case:family_membership_change:{F}
        trigger S8.1.12.4 key_grant cascade
    else:
        hold in pending state (per operator-policy window) OR
        emit hard_case:family_consensus_protocol_violation:{F} and reject

##### S8.1.12.3.1 `quorum:M/N` is absolute-M (normative; per CIRISRegistry#52 + NodeCore#30)

The `M` in `quorum:M/N` is an **absolute signature count**, NOT a fraction that rebases with
    roster size. A `quorum:2/3` collective that grows to 5 members still admits at **2**
    signatures; the `N` is documentary (records the roster size at protocol-adoption time) and
    is NOT recomputed against the live roster. This was an ambiguity in the pre-pin S8.1.12.3
    pseudocode ("operator policy resolves rebasing"); CIRISRegistry#52's ceremony gate ([S52
    design](https://github.com/CIRISAI/CIRISRegistry/issues/52)) requires a deterministic,
    operator-independent reading, so absolute-`M` is pinned.

**Rationale**: absolute-`M` is the simpler invariant (the gate is a pure `count >= M` with no
    live-roster division), it matches NodeCore's shipped `evaluate_consensus_protocol` ([
    NodeCore#30](https://github.com/CIRISAI/CIRISNodeCore/issues/30) commit 7469dcc) so the
    single shared predicate needs no change, and proportional/rebasing quorum is still
    expressible for operators who want it via `weighted:{rubric}` (assign each member weight 1,
     set threshold = `ceil(roster_size / 2)` recomputed by the rubric resolver). Collectives
    that want "M grows with the roster" therefore use `weighted:`, not `quorum:` -- keeping `
    quorum:M/N` unambiguous.

This rule applies identically to `community` admission ([S8.1.13.2](#81132-community-admission-
    per-consensus_protocol-cohort_subkind-dispatch)) -- the `quorum:M/N` arm there is the same
    absolute-`M` reading.
```

Consensus-protocol amendment (changing `consensus_protocol` itself on a non-entrenched family) rides the SAME admission rule on the proposed amendment Contribution — meta-amendment shape parallel to §11.2.3. Entrenched families (`consensus_protocol_entrenched == true`) reject amendments at the substrate gate; replacement requires the out-of-band ceremony documented per family (for HUMANITY_ACCORD see §9.2).

§8.1.12.4 Key-grant cascade (the at-rest encryption flow)

On admission of a new `identity_occurrence` OR a `family` membership-add, substrate emits retroactive `key_grant`s wrapping all extant `cohort_scope: self|family` content DEKs to the

new member's key:

```
on_member_added(scope_target, new_member_key):
    for each Contribution C in federation_attestations where:
        C.cohort_scope == "self" AND C.attesting_key_id in resolve_self(scope_target)
        OR
        C.cohort_scope == "family" AND C.family_id == scope_target
    AND C is still substrate-admitted (not withdrawn / not expired):
        emit key_grant {
            wrap_algorithm: X25519MlKem768Aes256GcmHkdfSha256, // v2 -- see PQC note
            recipient_key_id: new_member_key,
            content_sha256: C.evidence_refs[0].sha256,
            scope: GroupMember,
            wrapped_dek: wrap(C.dek, new_member_key.pubkey),
            ...
        }
```

PQC at rest — `wrap_algorithm: v2` MANDATORY (normative). **The self/family at-rest DEK MUST be wrapped with `wrap_algorithm: v2` = `x25519_mlkem768_aes256_gcm_hkdf_sha256`** (hybrid X25519+ML-KEM-768; the §5.6.8.4 variant `X25519MlKem768Aes256GcmHkdfSha256`), **never v1** (X25519-only). Self/family content is the user's most private and longest-lived data (memory, photos, identity, the §8.1.12.7 Self DEK) — a classical-only KEM is a harvest-now-decrypt-later exposure, the identical mandate as the streaming epoch DEK (§10.5.3). The AES-256-GCM content seal is symmetric (PQC-safe); the wrap signature is hybrid Ed25519+ML-DSA-65. A Consumer MUST reject a self/family at-rest grant carrying `wrap_algorithm: v1`.

The cascade is the wire-format primitive for the "I got a new phone and want my Twitter history" + "I added Carol to the household for a week" flows. Operator policy MAY bound the cascade depth (e.g., last 90 days of self-content; opt-in for the full historical wrap).

§8.1.12.5 Forward secrecy on member removal (Option A, recommended for v1)

Per CIRISRegistry#47 Decision 1 recommendation, CEG 0.7 adopts **Option A** — once shared, always shared at the wire layer:

```
on_member_removed(scope_target, removed_member_key):
    // The removed member retains existing key_grants -- cannot retroactively un-share.
    // Substrate STOPS wrapping NEW key_grants to them on subsequent
    // cohort_scope: self|family Contributions for scope_target.
    // No DEK rotation; no re-encryption of historical content.
```

This is consistent with §11.4 "takedown isn't a coup" + §3.2 `withdraws-isn't-retroactive` semantics — historical state isn't retroactively re-keyed. Option B (rotate-DEK on removal) is deferred to a future `subject_kind: family_rotation` ceremony; CEG 0.7 documents the slot, leaves the rotation primitive for a downstream-demand-driven release.

Why Option A: aligns with the substrate's existing forward-secrecy posture (consent revocations don't retroactively un-emit per CEG 0.6 §8.1.11; takedowns don't retroactively un-emit per §11.4); matches user-intuition that "leaving the family" governs future content, not historical; bounded substrate cost (no re-wrap-all-content storm on member removal).

§8.1.12.6 Composition with CEG 0.6 subject_key_ids[]

`identity_occurrence` + `family` (membership / visibility scoping) compose cleanly with CEG

0.6 `subject_key_ids[]` (revocation authority):

- A `cohort_scope: family` Contribution naming `subject_key_ids: [user_canonical_hash]` is admitted at family visibility AND the named subject retains independent revocation authority per CEG 0.6 §8.1.11.
- A `cohort_scope: self` Contribution that Alice writes about Bob (Bob in `subject_key_ids`) stays in Alice's self-collective (Bob does NOT receive a key_grant unless Bob's identity is in Alice's self — which it isn't). Bob's subject-side revocation authority still composes: Bob CAN issue `withdraws` against Alice's Contribution (admitted per CEG 0.6 §3.2.3 rule 2 even though Bob can't access the bytes); admission emits `hard_case:consent_sla_breach` clock-start if Alice committed `consent:deletion_sla`.

The orthogonality holds: **`cohort_scope` is producer-side visibility scoping**; `identity_occurrence` + `family` are substrate-side membership primitives that gate at-rest DEK wrapping**; `subject_key_ids` is subject-side revocation authority**. Three independent envelope-level concerns that compose without overlap.

§8.1.12.7 The "Self at login" — app + agent co-self + partnered delegation (CEG 0.15, normative composition)

Per CIRISRegistry#65. The canonical user-identity composition: a person's **app** (the KMP client key) and their **agent** (CIRISAgent's local key) are two occurrences of one user identity that share one **Self DEK**, and at login the agent is **partnered + delegated** to act as the user on the network. **No new structural primitive** — this composes `identity_occurrence` + Policy L + `consent:partnered` + `delegates_to` + `identity_type`-set + `transport_destination`. The four implementations MUST follow this shape so a "Self" is identical everywhere.

Two layers — and they are independently revocable (the load-bearing distinction):

Layer	Mechanism	Buys	Revoked by
Co-self (visibility)	occurrence + Policy-L Self DEK	the agent can *read/manage the user's Self* (decrypts self-content)	`withdraws` the occurrence → Option-A re-key (§8.1.12.5)
Agency (act-on-behalf)	`consent:partnered` + scoped `delegates_to`	the app may *act AS the user on the network* (send/receive, presence, sub-delegate)	`withdraws` the delegation → agency ends, **co-self unaffected**

So a user MAY grant a device co-self (it manages their data locally) while revoking its network agency — or vice-versa — without disturbing the other.

Agency at login (the partnering + delegation).

1. **Partnering** — the user emits `consent:partnership_grant` and the agent occurrence emits `consent:partnership_accept` under one `bilateral_pair_id` (§8.1.11.4 PARTNERED); the bilateral pair is the persistent, auditable relationship.
2. **Delegation** — the user identity emits `delegates_to` against the **agent occurrence key**, with `delegated_scope` drawn from the canonical act-on-behalf kinds below, `delegation_purpose: "act_as_user"`, bounded `delegation_valid_until`. Sub-delegation works because `delegates_to` chains (depth-capped at 5 per §13.3).

3. **This grant is FEDERATION-tier (§10.1.5), not local** — *other peers must verify the agent's authority before honoring its messages/presence*, so the partnering+delegation is signed + promoted at login. **Promotion is the "app shows up on the network" moment.** (The agent's own self-content stays local-tier; only the act-on-behalf authorization federates.)

Canonical `delegated_scope` kinds for act-on-behalf (recommended; open-vocab per §11.2.1, named for ecosystem coordination):

scope	grants the agent
`act_on_behalf`	umbrella: emit Contributions AS the user (`attesting_key_id = agent occurrence`, speaking as the user identity per §5.6.8.8)
`message_io`	send + receive directed messages on the user's behalf
`network_presence`	announce/resolve the user's `transport_destination` (§5.6.8.8.1) — be reachable AS the user
`sub_delegation`	issue further `delegates_to` within the granted scope (depth-capped)

Transport (network presence) — AV-17. Each occurrence binds a `transport_destination` (§5.6.8.8.1); the app is reachable on RET *as the user occurrence*. The Reticulum destination is a **separate dual-key transport identity** that the user's signing key *authorizes by signing the binding* — the federation signing seed MUST NOT enter the transport layer (AV-17 / CIRISEdge#15). "User key used as a transport key" means *roots/authorizes* the transport identity, not a shared keypair.

Worked login flow. (1) unlock the hardware-rooted user key (WebAuthn presence) → (2) admit the agent occurrence (single-vouch) → Policy-L Self DEK now wraps to both → (3) optionally add the `wise_authority` role to the user's `identity_type` set → (4) bind each occurrence's `transport_destination` → (5) `consent:partnership_grant/accept` under a `bilateral_pair_id` → (6) `delegates_to(user → agent occurrence, scope: [act_on_behalf, message_io, network_presence, sub_delegation])`, **promoted to federation-tier**. The app now reads the user's Self locally AND acts as the user on the network; the user can revoke either layer independently.

§8.1.12.7.1 Signing member sets (normative — the JCS contract Verify hybrid-signs; resolves CIRISVerify#63)

Each of the three Self-at-login Contributions is hybrid-signed over `JCS(envelope)` (§0.9 / RFC 8785), and at login promoted to federation-tier (the §10.1.5.3 promotion canonicalizes the **exact committed member set** — omit-vs-materialize (§0.9.2) is load-bearing; the signer MUST NOT re-default). **The §0.9.2.1 determinism rules apply** (raised in CIRISVerify#63 review): `subject_key_ids[]` and `delegated_scope[]` are **lexicographically sorted** (set-semantics); `aspects[]` retains RNS order (sequence-semantics); all key/hash/pubkey byte fields (`*_key_id`, `subject_key_ids[]`, the two reticulum pubkeys, `destination_hash`) are **lowercase hex per §0.6**; all timestamps are **§0.5-canonical**. The member sets the producer commits (and which `JCS` therefore covers) are pinned below. Optional §4 envelope fields not listed ride the §0.9.2 omit rule (absent unless the producer sets them).

(a) `consent:partnership_grant` (user side) / `consent:partnership_accept` (agent side) — bare `scores` (§5.6.8.6) bound by `bilateral_pair_id`:

```
{ attestation_type: "scores",
  attesting_key_id: <user identity_key_id (grant) | agent occurrence_key_id (accept)>,
  dimension: "consent:partnership_grant" | "consent:partnership_accept",
  score: <positive>,
  subject_key_ids: [<the partner key: agent occurrence (grant) | user identity (accept)>],
  bilateral_pair_id:<shared pair id>, // S8.1.1.11.4 binding mechanism
  signed_at: <rfc3339_canonical> }
```

(b) `delegates_to` (user → agent occurrence) — the act-on-behalf grant (§3.2 envelope shape):

```
{ attestation_type: "delegates_to",
  attesting_key_id: <user identity_key_id>,
  attested_key_id: <agent occurrence_key_id>, // the delegate
  delegated_scope: ["act_on_behalf", ...], // S8.1.12.7 canonical kinds
  delegation_purpose: "act_as_user",
  delegation_valid_from:<rfc3339_canonical>,
  delegation_valid_until:<rfc3339_canonical>,
  signed_at: <rfc3339_canonical> }
```

(c) `transport_destination` binding — an `identity_occurrence` (§5.6.8.8 / §5.6.8.8.1) carrying the binding; signed by `identity_key_id` (or a current occurrence), AV-17:

```
{ attestation_type: "scores",
  subject_kind: "identity_occurrence", // payload discriminator S4.2.2.3
  attesting_key_id: <user identity_key_id | a current occurrence>,
  identity_key_id: <user identity_key_id>,
  occurrence_key_id:<the occurrence being bound>,
  device_class: "phone" | "laptop" | "agent" | ...,
  transport_destination: {
    reticulum_x25519_pubkey: <[u8;32]>,
    reticulum_ed25519_pubkey: <[u8;32]>,
    destination_hash: <[u8;16]>, // MUST derive per S5.6.8.8.1
    app_name: <string>,
    aspects: [<string>, ...] }, // ordered
  asserted_at: <rfc3339_canonical>,
  signed_at: <rfc3339_canonical> }
```

Registry owns these member sets (this section); Verify computes `JCS(...)` + the hybrid Ed25519+ML-DSA-65 signature over each via `jcs::canonicalize` (CIRISVerify#59); the promotion signature (§10.1.5.3 OQ-4) is the identical JCS bytes — confirmed.

`encryption_pubkeys` joins member set (c) (1.0-RC1 — CIRISVerify#64). When the occurrence carries the §5.6.8.8.2 `encryption_pubkeys` field-set, both halves are **inside the signed JCS bytes** as opaque base64 strings (RFC 4648 STANDARD, padded — the §0.9.2.1 rule-2 pin). Optional presence rides the §0.9.2 omit rule (absent unless the producer sets it; the signer MUST NOT re-default). They are payload, never verification material — neither half may be fed to a signature-verify path (the §5.6.8.8.2 key-separation rule, type-enforced in Verify).

§8.1.13 Policy M — Community membership composition (CEG 0.8 addition)

Per CIRISRegistry#48 + §5.6.8.10 `community` + §5.6.8.11 `location_proof`. Composition pattern for resolving the **current membership set** of a community, gating cohort-filtered visibility for `cohort_scope: community` content.

Sibling to §8.1.12 Policy L (self/family) but with different defaults — community content is encrypted under a per-community DEK + emits `holds_bytes:sha256:*` with cleartext prove-

nance (CEG 0.17 — see §8.1.13.3); the privacy property is byte-confidential-to-members, not cohort-filtered-visibility.

§8.1.13.1 Community membership resolution

For community `C`, the current member set is computed as:

```
resolve_community(C, now):
    latest = federation_attestations
        .where(subject_kind == "community")
        .where(envelope.community_key_id == C)
        .where(supersedes chain -> latest non-superseded)
        .where(admission rule satisfied per CURRENT consensus_protocol;
              see S8.1.13.2)
    return {m.key_id for m in latest.envelope.members}
```

Same shape as `resolve_family` (§8.1.12.2). Each `member.key_id` is an identity (which may itself have a multi-occurrence set via CEG 0.7 `identity_occurrence`).

§8.1.13.1.1 Deterministic resolution + member→address resolution (CEG 0.12, NORMATIVE)

In a CEG/RET stack member resolution replaces DNS+IP: it is a chain of *signed* bindings, and every implementation MUST resolve **identically** (a 1.0 interop requirement). Two normative pins:

(a) Determinism of `resolve_community`. Where the §8.1.13.1 computation has choices, they are fixed:

- **`now` semantics** — the resolving Consumer's own clock; membership is evaluated as of `now` (a member is included iff joined $\leq$ `now` and not removed $\leq$ `now`). No global clock assumed.
- **"latest non-superseded"** — the `community` Contribution with the highest `signed_at`; on equal `signed_at`, the higher `canonical_bytes_hash` (§0.9) wins (total order, no ambiguity). The same comparator the CIRISVerify#49 R1/Q1 merge uses (quorum_weight DESC → signed_timestamp DESC → canonical_bytes_hash).
- **member ordering** — the returned set is canonically ordered by `key_id` (lowercase-hex byte order) for any downstream hashing/iteration.
- **founder-subset eval** — for `cohort_subkind: infrastructure` (§5.6.8.10) admission is `evaluate_consensus_protocol` over `{m : m.role == founder}`, NOT all members.

(b) Member → reachable address (the DNS-free resolution). Resolving a member identity to a Reticulum destination:

```
resolve_member_transport(C, now):
    members = resolve_community(C, now) // (a) above; founder-quorum verified
    out = []
    for M in members (canonical key_id order):
        occ = latest non-superseded identity_occurrence of M
                with transport_destination present, valid at now,
                hybrid-sig verified against M (or an occurrence of M) // S5.6.8.8.1
```

```
        if occ is null: continue // no authenticated binding -> skip (fail-secure)
        td = occ.transport_destination
        REQUIRE td.destination_hash == RNS_hash(td.reticulum_x25519_pubkey
                || td.reticulum_ed25519_pubkey || td.app_name || td.aspects)
        out.push((M, td.destination_hash))
    return out // -> Reticulum announce/path-request per dest
```

The three resolutions and their trust roots: **WHO** = `resolve_community` (signed Contribution + founder-quorum) · **BINDING** = `transport_destination` (federation-key-signed `identity_occurrence`, §5.6.8.8.1) · **WHERE** = Reticulum announce/path-request (mesh, no CEG). Reachability is never trust (§10.5.6): a path that answers is not an authorization; only the signed binding + quorum are.

Cold-start (bootstrap). A node that knows only `community_key_id` needs two out-of-band pins: (1) the `community_key_id` itself (trust anchor) and (2) ≥1 **seed `destination_hash`** (reachability). It reaches any one seed, pulls the signed `community` Contribution, runs `resolve_member_transpo` verifies founder-quorum against the pinned `community_key_id`, then floats on the live set thereafter. A malicious seed cannot forge membership (quorum signature check) — it can only deny service (try another seed). **Bootstrap reachability ≠ bootstrap trust.** `ciris-canonical` (§5.6.8.10) is the root community whose seeds clients pin.

§8.1.13.2 Community admission per `consensus_protocol` + `cohort_subkind`

A proposed membership change rides a `supersedes` Contribution on the community's latest admitted `community` Contribution. Substrate evaluates the CURRENT community's `consensus_protocol` (same six canonical kinds as family) AND any subkind-specific admission requirements:

```
admit_community_change(C, proposed: community_record):
    current = resolve_community(C, now)
    protocol = current_community_record(C).consensus_protocol
    subkind = current_community_record(C).cohort_subkind

    signatures_ok = evaluate_consensus_protocol(protocol, current, proposed.signatures)
    if not signatures_ok:
        emit hard_case:community_consensus_protocol_violation:{C}
        reject

    subkind_ok = evaluate_subkind_admission(subkind, current, proposed)
    if not subkind_ok:
        // For geographic: subkind admission failed (e.g., new member's location_proof
        // not contained in geographic_constraint, or expired location_proof)
        emit hard_case:community_consensus_protocol_violation:{C} // same observability prefix
        reject

    admit
    emit hard_case:community_membership_change:{C}
```

The `evaluate_subkind_admission` predicate dispatches per `cohort_subkind`:

```
evaluate_subkind_admission(subkind, current, proposed):
    match subkind:
        "geographic":
            # Per S5.6.8.10 geographic admission requirement
            for each NEW member in proposed.members (not in current):
                their_proof = latest valid location_proof from new_member.key_id
                if their_proof IS NULL:
                    return false // no location_proof on file
```

```
            if their_proof.cell_id NOT contained in current.geographic_constraint.cell_id:
                return false // location outside community's geographic bound
            if their_proof.valid_until passed:
                return false // expired
        return true
    _:
        return true // unknown subkinds admit on consensus_protocol alone
```

Operator vocabularies extending `cohort_subkind` provide their own `evaluate_subkind_admission` predicates per the `custom:{id}` consensus_protocol hook pattern from CEG 0.7 §8.1.12.3.

§8.1.13.3 The three crypto tiers + the Community DEK cascade (CEG 0.17, normative — supersedes the 0.8 "no cascade" reasoning)

CEG ≤ 0.16 drew a binary cut (self/family encrypt; everything else plaintext), which collapsed a bounded **Community** and the unbounded **Commons** into one bucket and left **no cryptographic home for a persecuted community**. The 0.8 "no at-rest cascade" reasoning ("per-member DEK wrap on every emission would be infeasible") was a **wrong premise** — corrected here. CEG 0.17 draws the line at **"does it have a bounded membership roster?"** — yes → encrypt, no → plaintext:

Tier	`cohort_scope`	At-rest	Wire discovery	Reader
self / family	`self`, `family`	encrypted, per-write DEK	**none** (§10.1.4 structural invisibility)	occurrences / family members
Community	`community`, `affiliations`	**encrypted under the community DEK**	`holds_bytes:*` **+ cleartext provenance**	community members (DEK cascade)
Commons	`species`, `biosphere`, `federation`	**plaintext**	`holds_bytes:*`	anyone

Holder-inspectability principle (normative rationale). Any data a host holds above local tier MUST support an informed keep/evict decision: either the holder **inspects the bytes** (Commons — plaintext, maximally inspectable, hence the *preferred* social-distribution mechanism) **or** it **inspects the provenance** (Community — the cleartext `attesting_key_id` + `community_id` + reason on an otherwise-encrypted blob) and chooses to hold opaque ciphertext for a community it trusts. **Nothing above local tier is ever a forced, unattributable opaque blob.** This is *why* the split is shaped this way and what the eviction rules (persist `EvictionSweeper` / `evict_actor`) enforce.

The `infrastructure` exception (Open-Q1 ruling, normative). A `community` with `cohort_subkind: infrastructure` (§5.6.8.10) — `ciris-canonical` and any governance/trust root — **opts OUT of the mandatory DEK cascade and is Commons-tier (plaintext, `holds_bytes:*`, no DEK). The trust root cannot be an opaque blob; its entire purpose is public auditability — transparency-seeking, not privacy-seeking.** Node→canonical traces** (conformance / `registry_consensus` emissions to a governance community) are therefore `cohort_scope: federation` (Commons/plaintext, world-readable) — a node enrolling in governance is thereby told its conformance traces are public (Open-Q2 ruling).

§8.1.13.4 Forward secrecy on community member removal (Option A — now applies)

With a community DEK, community **does** have the removed-member-can-still-decrypt concern (the 0.8 "no DEK to retain" reasoning is superseded by §8.1.13.3). The §8.1.12.5 **Option A** discipline applies identically: on member removal the substrate **rotates the community DEK** (§11.7.1); subsequent emissions are sealed under a DEK the removed member doesn't have. "Once shared, always shared" forward-only — content the removed member already received during membership stays in their cache (no PCS); they receive no NEW community content post-removal.

§8.1.13.5 Geographic-community admission flow (worked example)

```
1. Alice publishes a location_proof:
   subject_kind: location_proof
   subject_key_id: alice_root_key
   cell_id: "872830828ffffff" // H3 res 7, ~5 km^2, downtown Austin
   cell_resolution: 7
   asserted_at: now
   valid_until: now + 30 days
   cohort_scope: federation // public; the disclosure IS the opt-in

   Substrate admits (resolution 7 <= 7 OK). Emission federates.

2. Alice proposes joining Austin community:
   subject_kind: community
   community_key_id: austin-community
   supersedes: prior_austin_community_record_id
   members: [...existing..., {key_id: alice_root_key, joined_at: now, role: member}]
   signatures: [...current_member_sigs] // satisfies majority rule

3. Substrate admission gate (per S8.1.13.2):
   - signatures_ok: majority rule satisfied v
   - subkind: "geographic" -> evaluate_subkind_admission:
     - alice_root_key's latest valid location_proof exists
     - cell_id "872830828ffffff" CONTAINED in "85283473fffffff" (austin metro, res 5)
       via S0.8.2 containment (res 7 >= res 5; parent-walk reaches the constraint)
     - valid_until in the future
     - subkind_ok v
   - admit + emit hard_case:community_membership_change:austin-community

4. Alice now receives cohort-filtered visibility for cohort_scope:community content
   with community_id: austin-community. No key_grant cascade (community is unencrypted);
   no at-rest wrap; substrate emits holds_bytes:* for community content per status quo.
```

§8.1.13.6 Composition with CEG 0.6 + 0.7

Communities compose with consent (CEG 0.6) and self-collectives (CEG 0.7) cleanly:

- A community member who is also a CIRISAgent-using individual has an `identity_occurrence` set (CEG 0.7); community content arriving at the member's identity_key is then propagated to their occurrences via Policy L's at-rest cascade (if the member's local substrate chose to re-emit the community content at `cohort_scope: self` for cross-device sync; otherwise community content stays at the cohort_scope: community visibility on each device the member uses to query)
- A community member who is also a subject in `subject_key_ids` of a community-scoped

Contribution retains revocation authority per CEG 0.6 §3.2.3 rule 2; the orthogonality between cohort_scope (visibility) and subject_key_ids (revocability) holds at community scope same as at family scope

- A geographic community whose `geographic_constraint` covers a region that overlaps a family's at-home location does NOT cross-contaminate: families are not auto-admitted to communities; communities are not auto-admitted to families. Each membership is explicit, ceremony-shaped, and independent.

§8.1.13.7 Delivery extension — `delivery_mode` × Policy M (CEG 0.10 addition)

Per CIRISRegistry#44 absorbed + CIRISLensCore#857. Policy M's community-membership composition extends to govern the **delivery axis** (`delivery_mode` envelope field per §4). The subscriber-set for any push-delivery flow IS a `community` Contribution; "subscribe = join the community"; inherits revocation, consensus, and structural-invisibility from Policy M unchanged.

Subscriber-set composition:

```
For a Contribution C with delivery_mode = push AND cohort_scope = community:
    subscriber_set(C) = resolve_community(C.community_id, now)
                        per S8.1.13.1 community membership resolution

The community is admitted under the standard Policy M consensus_protocol
options (founder_only / unanimous / majority / quorum:M/N / weighted /
custom) per S8.1.13.2. Subscription-specific admission semantics --
producer_gated (publisher approves each member) vs open (anyone can
join) -- ride the consensus_protocol choice:

    producer_gated -> consensus_protocol = "founder_only" with the
                      publisher as sole founder; the publisher authorizes
                      each new subscriber
    open -> consensus_protocol = "open:self_admit" (open-vocab
                      extension hook) where any subject signs their own
                      admission Contribution
```

Cardinality unified across observer-share and multicast:

Cardinality	Per-subscriber crypto	Epoch handling
N=1 (observer-share)	Single `key_grant` (§5.6.8.4); no epoch needed (one recipient, one DEK)	No epoch — single grant, single DEK; revocation = `withdraws` against the grant
N>1 (multicast)	Flat per-epoch `key_grant` cascade — one grant per (subscriber, epoch); the stream-epoch DEK seals content O(1), the cascade distributes the 32-byte epoch key O(N)/epoch per §10.5.3	Per-`(stream_id, epoch)` axis; epoch rolls on member removal (mandatory) + time/bytes (optional) per §10.5.3 D3

The same Policy M machinery handles both cardinalities — the difference is purely the cascade fan-out factor.

Composition with `delivery_mode: pull` (RC1 default):

For `delivery_mode: pull`, subscribers discover via the standard `holds_bytes:sha256:*` directory per §10.1. Policy M still resolves the community for visibility-filtering (consumer reads

`community_id` envelope field, walks the membership, filters out non-member peers from the discovery surface). No fan-out push; no `delivery_receipt` emission required (best-effort).

Composition with `delivery_mode: push` (RC1 substrate-pending; live in 1.x):

For `delivery_mode: push`, the substrate fans out to `entitled ∧ reachable` per §10.5.6 D6 liveness invariant. Entitlement = Policy M membership resolution (durable, signed CEG, replicated, logged); reachability = Edge `reachability.rs` (CIRISEdge#29) node-local presence tracker (TTL sec/min, NEVER an attestation, never replicated, never logged). Missed (entitled-but-unreachable) members fall back to pull on reconnect per §10.5.6.

`history_on_join` × Policy M membership additions:

On a new-member admission via Policy M, the new member's `history_on_join` envelope value determines retroactive content delivery:

- `from_join` (default) — new member receives current-epoch content forward only; no retroactive `key_grant` cascade
- `full` — new member receives `key_grant`s for prior epochs subject to the §10.5.3 P4 catch-up bound (`min(operator depth cap, chunk-eviction horizon)`); evicted-epoch grants return `ContentMiss` per the `MISSION.md` fail-honest invariant

This composes with §8.1.12.5 Option A forward-secrecy: removed members retain extant `key_grant`s for content they were entitled to during membership; new members may or may not get retroactive grants per `history_on_join`. The substrate's forward-secrecy posture is uniform across consent, takedown, membership-departure, AND delivery-onboarding surfaces.

§8.2 Aggregation semantics — opinionated defaults

Per dimension+attested_key_id, the verdict is computed by composing attestations under the chosen policy. Default aggregation by polarity column (§5):

Polarity column value	**Default aggregation**
`signed`	**Mean** of `score` × `confidence` across attesters
`boolean-via-score`	**Min** (any negative trumps positive — fail-secure for hard constraints like `prohibited:*`, `attestation:1*`)
`+1.0 only` / `positive-only`	**Max** across attesters (any positive is conclusive)
`-1.0 only`	**Min** across attesters (any negative is conclusive)
`enumerated`	**Most-recent** by `signed_at` from the attester(s) authorized to emit per §7
Detector dimensions (`detection:correlated_action:*`, `detection:distributive:access:*`, `ratchet:flag:*`)	**Median** across attesters (resists adversarial mean-pulling by a single captured detector)

Specific dimensions override via consumer policy; the defaults above are the §0.2 CEG-Conforming Consumer (CCC) minimum.

§8.3 Frickerian discipline — consumer-policy norms

Per Miranda Fricker's *epistemic injustice*: consumers SHOULD apply identity-prejudice-resistant weighting. Concretely:

- Don't downweight `testimonial_witness:*` from cohorts with low overall attestation density (testimonial preservation is precisely what corrects for that low density).
- Don't downweight `non_maleficence:*` claims about a partner just because the partner has a long `partner_role:*` track record (the long track record may be the harm).
- Apply §8.1.4 lexical-vulnerability-priority in tie-breaks involving small cohorts.

Adversarial caveat: the discipline above is consumer-policy-only; an adversary can emit `testimonial_witness` exploiting the Frickerian non-downweighting rule. Per §5.6.3, `testimonial_witness:*` is never sole evidence for `slashing:*`; per §7.0 the consumer MUST also weight `witness_relation:self` claims against the attester's other-emission track record. The Frickerian rule applies AFTER these structural safeguards, not before them.

§8.4 Sovereign-Registered equivalence (wire-symmetric, policy-differentiated)

A Sovereign agent scoring `licensure:CA_medical_board: +1.0` is wire-format identical to a Registry-steward scoring the same. Consumer policy weights by attester source; the substrate is source-neutral. M-1's symmetry is structural, not bolted on.

Per `../MISSION.md` §1.1: both paths produce federation membership; neither is a gate. What differs is the *attestation surface* — the kind of claim the federation can compose about why a participant is trustworthy.

§9 The HUMANITY_ACCORD constitutional layer

The single wire-format asymmetry in the federation.

§9.1 The accord-holder triple

Three named human key holders. Initial state at federation genesis:

Position	Holder	Threshold
1	Eric Moore	2-of-3
2	Eric Kudzin	2-of-3
3	Haley Bradley	2-of-3

Hardware-attested (per §9.4 hardware_class taxonomy). Permanent: no automatic decay; replacement requires out-of-band CIRIS L3C process per FEDERATION_ANNOUNCEMENT.md §4.5.3.

Correlated-failure geometry (named honestly — 1.0-RC1, #71 C5): two of the three holders share a household, so the 2-of-3 quorum is physically achievable from one street address — a correlated compromise/coercion surface that entrenchment makes harder to correct later. The authority at stake is the **full constitutional kill** (`EmergencyShutdown CONSTITUTIONAL` — not a recoverable pause), so the exposure is real and is not softened here; what scope isolation (§9.2) does guarantee is that compromise cannot escalate *beyond* the kill — accord keys cannot sign grants, licenses, or amendments. **The mitigant is diversifying the holder set: finding new holders** (via the out-of-band replacement process, FEDERATION_ANNOUNCEMENT.md §4.5.3) so that no household — and ultimately no single jurisdiction — can assemble the quorum. This is an active obligation on CIRIS L3C, not a deferred nice-to-have.

The HUMANITY_ACCORD triple is the canonical entrenched-`family` instance (CEG 0.7 retcon). Per §5.6.8.9, the accord-holder triple structurally IS a `family` subject_kind with:

```
family {
    family_key_id: "humanity-accord",
    family_name: "Humanity Accord",
    members: [
        {key_id: <eric-moore-key>, role: founder},
        {key_id: <eric-kudzin-key>, role: founder},
        {key_id: <haley-bradley-key>, role: founder}
    ],
    consensus_protocol: "quorum:2/3",
    consensus_protocol_entrenched: true // replacement requires S9.2 / FEDERATION_ANNOUNCEMENT.
        md S4.5.3 ceremony
}
```

The 2-of-3 multi-sig verifier at §9.2.1 is the `quorum:2/3` consensus_protocol enforcement; the entrenchment property is what prevents any federation-internal authority from amending the protocol. §9 remains load-bearing for the **role-recognition policy** (which dimensions accord-holders may emit — only `accord:*` per §7.1) and the **scope-isolation** discipline (only `EmergencyShutdown CONSTITUTIONAL` per §9.2). CEG 0.7 makes the structural shape explicit — the constitutional asymmetry is "an entrenched family that is wire-scope-isolated to halt authority," not a one-off primitive. Other entrenched-family instances (a national-emergency triple, an international body, a court-ordered preservation triple) MAY appear in operator deployments; HUMANITY_ACCORD is the one CIRIS L3C deployments ship at genesis.

§9.2 Authority scope

`HUMANITY_ACCORD` signatures are valid only on `EmergencyShutdown CONSTITUTIONAL` (`IncidentSeverity::INC = 5`), `accord:invoke:notify:{notify_id}`, `accord:invoke:drill:{drill_id}`, `accord:lifecycle:active`, and the corresponding `FederationAnnouncement` priority `AccordCarrier`. Announcements of any other priority signed by accord-holder keys are rejected at admission (out of role). Federation-side authority cannot sign `AccordCarrier`; humanity-accord authority cannot sign anything else. **Wire-isolated AND scope-isolated.**

§9.2.1 Invocation canonical bytes (anti-replay; 0.1 scaffold)

> ***RESOLVED at 1.0-RC1****: the discriminator + nonce binding below addresses the cross-invocation-replay hole identified by CEG 0.1 cryptographic + red-team review. The anticipated §5.2.1 refinement landed as the* ***JCS redesign*** *(TupleHash128 retired — see §5.2.1); this invocation encoding is* ***intentionally NOT migrated****: its preimage is closed-vocabulary (discriminator + nonce + enum fields, no attacker-controlled free text), so the injection surface the §5.2.1 redesign closes is not reachable here, and genesis-critical bytes stay stable.*

Every `accord:invoke:*` Contribution signs the following canonical bytes (BOTH the discriminator AND a per-invocation nonce are in the signed payload — preventing CONSTITUTIONAL ↔ notify ↔ drill cross-replay):

```
canonical = sha256(
    "ciris.accord_invoke.v1\n" ||
    "invocation_kind=" || ("CONSTITUTIONAL" | "notify" | "drill") || "\n" ||
    "invocation_id=" || halt_id_or_notify_id_or_drill_id || "\n" ||
    "nonce=" || base64url(rand_32_bytes) || "\n" ||
    "asserted_at=" || rfc3339_canonical || "\n" || // per S0.5
    "valid_until=" || rfc3339_canonical || "\n" ||
    "payload_sha256=" || sha256_hex_lowercase_of_payload // per S0.6
)
```

Hybrid signature per §5.2.1: Ed25519 + ML-DSA-65 bound-payload. Each of the 2-of-3 holders signs `canonical` independently; consumer verifies all three signatures against the same `canonical` bytes and counts ≥ 2 valid.

The substrate MUST reject duplicate `invocation_id` values within the `valid_until` window (per-kind unique).

§9.2.2 `notify` vs `CONSTITUTIONAL` — consumer-UI requirement

A CEG-Conforming Consumer (CCC) presenting accord invocations to humans MUST visually distinguish the three kinds:

- **`CONSTITUTIONAL`** — kill-switch authority; full halt; visible as an unambiguous emergency banner.
- **`notify`** — federation-wide accord-holder communication; MUST NOT be visually conflated with CONSTITUTIONAL.
- **`drill`** — accord-holder exercise; MUST be visually marked as a drill (e.g., explicit "[DRILL]" prefix on any human-visible surface).

Wire-format isolation alone does not close the social-engineering risk that downstream UI conflates the three; the consumer-UI requirement above is the load-bearing safeguard against accord-holders being socially pressured into emitting a `notify` that carries CONSTITUTIONAL social weight without CONSTITUTIONAL substrate weight.

§9.3 Concern split — key material vs role-recognition policy

Key material (Ed25519 + ML-DSA-65 pubkeys for the three holders) lives in **CIRISPersist substrate**: `federation_keys` rows with `identity_type="accord_holder"`, self-signed at provisioning, cross-attested by all three regional stewards.

Role-recognition policy + verifier logic lives in **`ciris-registry-core`**: the 2-of-3 multi-sig verification, the `EmergencyShutdown CONSTITUTIONAL` admin RPC, the audit hooks.

§9.4 Hardware-class taxonomy

Value	Use
`HSM_FIPS_140_3_L3`	Production stewards (US / EU / APAC)
`Apple_Secure_Enclave`	Accord-holders on iOS/macOS
`YubiKey_5_FIPS`	Accord-holders preferring portable hardware tokens
`TPM_2_0`	Accord-holders on Linux/Windows desktops
`placeholder_pending_provisioning`	Interim value before actual hardware provisioning. Consumers MUST treat as `0.0` trust weight
`software_hsm_development`	DEVELOPMENT ONLY; consumer policy MUST reject for federation-scope verification

Per-class recommended trust-multipliers: `HSM_FIPS_140_3_L3` = 1.0; `Apple_Secure_Enclave` = 0.95; `YubiKey_5_FIPS` = 0.95; `TPM_2_0` = 0.9; `placeholder_pending_provisioning` = 0.0; `software_hsm_development` = 0.0.

§9.4.1 Hardware-class self-assertion gap (acknowledged)

The `hardware_class` field is currently a self-asserted string on each `federation_keys` row. CEG 0.1 has no normative mechanism (TPM quote chain, Apple attestation, FIDO attestation) for a verifier to independently corroborate the claim. Per §15.2 **R5** (acknowledged risk): consumer policy MUST treat the `hardware_class` field as a producer claim, not a cryptographically-attested fact. A planned 0.x → 1.x roadmap item closes this via per-platform attestation-chain verification; until then the trust-multipliers in §9.4 above bind only as guidance.

§9.5 Why this isn't a Golden-Rule violation

Per §1.5: the Recursive Golden Rule binds *participants in the federation* to each other. Humanity-as-such occupies a position outside the federation's participant set, by design. The three named human key holders hold `AccordCarrier` authority that no federation-side authority class (including `SYSTEM_ADMIN` / `WISE_AUTHORITY` / per-install stewards) can grant itself, revoke, override, or decay. This is not a Golden-Rule exemption; it is the recognition that consent (M-1's load-bearing property) requires revocability, and revocability requires a halt-authority that lives

outside the system being halted. The federation cannot deny humans the right to halt it, because no federation-internal protocol path to that signature exists.

§10 Endpoint shapes

CEG specifies five public + one admin HTTP endpoint shape for the discovery + cosigning surfaces. Wire format consumers (CIRISVerify v3.1.0+, CIRISAgent KMP UI, iOS/Android FFI) read these.

§10.0 Common response shape

All CEG endpoints return:

- **Content-Type**: `application/json` (`Accept: application/json` honored; other types respond `406 Not Acceptable`)
- **CEG-API-Version header**: `CEG-Version: <current spec major.minor>` on every response (track the README `Version:` field; currently `1.0-rc1`); clients SHOULD echo `CEG-Accept-Version: <pinned-version>` on request, naming the version they were built against. Per §0.3 SemVer policy, MAJOR mismatch is a wire-incompat reject; MINOR mismatch is compatible (clients MAY warn).
- **Time-Source header**: `X-CEG-Server-Time: <rfc3339_canonical>` per §0.5 for client clock-skew bounds
- **Pagination** (where applicable): `?cursor=` + `?limit=` query params; response includes `next_cursor` (null if exhausted) and `total_estimate` (server's best estimate, may be approximate)

§10.0.1 Error envelope

All error responses MUST conform to:

```
{
  "error": {
    "code": "<ENUM_VALUE>",
    "http_status": <int>,
    "message": "<human-readable>",
    "request_id": "<server-assigned>",
    "details": {<error-specific fields>}
  }
}
```

HTTP status	Error code	Meaning
400	`MALFORMED_REQUEST`	Invalid JSON, missing required field, bad field type
400	`CANONICAL_BYTES_VIOLATION`	Date-time / hex / encoding doesn't match §0.5 / §0.6
401	`UNAUTHENTICATED`	Bearer token missing or invalid (admin endpoints)
403	`RESERVED_PREFIX_VIOLATION`	Producer attempted to emit under a reserved prefix without authority per §7
404	`UNKNOWN_WITNESS`	Witness key_id not registered in directory (§10.3)

HTTP status	Error code	Meaning
404	NOT_FOUND	Generic resource not found (build, partner, key)
409	IDEMPOTENT_CONFLICT	Replay detected (e.g., duplicate (tree_size, witness_key_id) cosignature with different signatures)
422	SIGNATURE_VERIFICATION_FAILED	Ed25519 or ML-DSA-65 failed to verify; details.algorithm names which
422	CLOCK_SKEW_VIOLATION	signed_at exceeds §0.7 ±5 minute tolerance
422	WITNESS_QUORUM_NOT_MET	Insufficient cosignatures to validate
422	CONSISTENCY_PROOF_INVALID	A witness cosignature's §10.3.1 consistency proof against the prior STH it cosigned is absent or does not verify (added CEG 0.10 per CIRISRegistry#34; details carries prior_tree_size / new_tree_size / proof_len). A missing proof when a prior STH exists, or a tree_size behind the witness's prior cosigned STH, is MALFORMED_REQUEST instead.
429	RATE_LIMITED	X-RateLimit-* headers set; Retry-After honored
500	INTERNAL_ERROR	Server-side fault; request_id usable for support
503	WITNESS_DIRECTORY_UNAVAILABLE	Substrate replication lag exceeds liveness bound

Rate-limit headers on every response: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset (seconds-until-reset epoch).

§10.1 Transport substrate for byte-level content

Wire-format Attestations carry claims; they don't carry bytes. When a claim's evidence_refs[] cites a SHA-256-addressed blob (e.g., an installer binary, a config file, an adapter package per agent_files:* per §5.6.7 / §5.9), the bytes travel via Edge transport substrate: MessageType::ContentFetch + ContentBody + ContentMiss (per CIRISEdge#21). Holder-discovery via Persist's holds_bytes:sha256:* directory (CIRISPersist#103); peer-resolution via Edge's PeerResolver::resolve_holders. NodeCore node-mode peers serve the bytes per their MISSION §3.4 cohabitation contract (CIRISNodeCore#11). Attestation envelope shape unchanged; SHA-256 in evidence_refs[] becomes universally resolvable to bytes through the substrate.

§10.1.1 Full-SHA verification before consumption (normative)

A CEG-Conforming Consumer (CCC) MUST verify the full SHA-256 of received bytes against the value in evidence_refs[] BEFORE handing the bytes to any consumer (Agent loader, Portal renderer, etc.). The holds_bytes:sha256:{prefix} directory (§5.6.7) carries only a short prefix for index efficiency; the consumer MUST NOT short-circuit verification to the prefix. Bytes that fail the full-SHA check MUST be discarded and the holder MUST be reported via the holds_bytes:sha256:{prefix} chain (emit a withdraws or negative score per consumer policy).

§10.1.2 Holder directory TTL + ContentMiss feedback

A `holds_bytes:sha256:{prefix}` attestation has a default validity of **24 hours** from `signed_at`. After that the holder is considered stale; consumer policy MUST attempt at most 2 holders in parallel and accept the first successful full-SHA verification. On `ContentMiss` (holder no longer has the blob), the consumer MUST emit a `withdraws` against the `holds_bytes:sha256:{prefix}` attestation referencing the stale holder, with `withdrawal_reason: "content_miss"`. Holders consistently failing ContentMiss are downweighted in `PeerResolver::resolve_holders`.

§10.1.4 Structural invisibility — `holds_bytes:sha256:*` suppression for `cohort_scope: self | family` (CEG 0.7 addition)

Per CIRISRegistry#47 + ciris.ai/cewp load-bearing claim:

> *Self and family content never emits the attestation that would tell the rest of the network it exists. You don't need a privacy policy to keep family photos off the federation — the wire format can't carry them in the first place.*

CEG 0.7 codifies this as a normative substrate discipline. When a Contribution carries `cohort_scope: self` OR `cohort_scope: family`, the substrate MUST NOT emit a corresponding `holds_bytes:sha256:{pre` directory attestation per §5.6.7 — the content's bytes are delivered to admitted members of the relevant self-collective (§5.6.8.8 `identity_occurrence`) or family (§5.6.8.9) via the at-rest encryption flow (CIRISPersist#152), NOT via the public holder-discovery directory.

The privacy property is structural, not policy:

- A non-member peer cannot issue `ContentFetch` for the bytes because no `holds_bytes:sha256:*` attestation names a holder.
- A non-member peer cannot even *discover* the bytes exist via the substrate — the only attestations referencing them are scoped to the self-collective / family and never federate beyond it.
- This is the wire-format-level closure of the cewp **structural invisibility** claim: privacy emerges from format constraints, not from operator policy or legal undertaking.

> ***Scope (normative — see §1.6):*** *structural invisibility buys* ***content-holding confidentiality only****. It does NOT hide that the relationship exists (`family_id`, admission `hard_case:*` events), who-talks-to-whom (resolution + RET announce/path-request expose endpoints), or traffic-analysis channels (STH cadence, key-cascade timing, chunk size/rate). It is not unobservability or metadata privacy — those require the opt-in Anonymous Tier. Do not represent CEG as providing the stronger properties.*

Substrate enforcement:

```
On admission of a Contribution C with cohort_scope in {self, family}:
    # (1) STRUCTURAL INVISIBILITY -- UNCONDITIONAL (the cewp privacy promise):
    substrate MUST NOT emit holds_bytes:sha256:* for C's evidence_refs bytes
    substrate MUST NOT propagate C beyond the self-collective / family scope
        via any other directory or discovery surface
    # (2) AT-REST ENCRYPTION -- the S8.1.12.4 cascade (defense-in-depth):
```

```
WHEN self/family at-rest encryption is enabled for the deployment:
    recipients := if cohort_scope == self: all current identity_occurrences of C.
        attesting_key_id
                 if cohort_scope == family: all current members of family per C.family_id
    FOR each recipient r:
        kem := resolve_encryption_keys(r.key_id) # current occurrence's encryption_pubkeys (
            S5.6.8.8.2)
        IF kem is None OR kem.ml_kem_768 invalid:
            # FAIL-SECURE: no valid v2 wrap target ? EXCLUDE r from the grant.
            # MUST NOT fall back to plaintext or to wrap_algorithm v1.
            # NOT SILENT (1.0-RC1, #71 C3): emit hard_case:recipient_excluded:{scope_key_id}
            # (S7.7) INTO the self/family scope -- recipient r, reason in
            # {expired_occurrence, invalid_kem_key, missing_encryption_pubkeys},
            # + the skipped Contribution's envelope ref. Scoped to the cohort;
            # never federates beyond it (S10.1.4 invisibility preserved).
            skip r # content stays encrypted + unreachable to r until r registers
                encryption_pubkeys
        ELSE:
            substrate MUST wrap C's DEK via key_grant (S5.6.8.4, wrap_algorithm v2 -- S8
                .1.12.4)
               to kem.{x25519, ml_kem_768}
```

Two layers, not one (normative split — clarified per CIRISPersist#152 review). (1) **Structural invisibility** — suppressing `holds_bytes:sha256:*` + non-propagation beyond scope — is the **unconditional** privacy promise (the cewp "the wire format can't carry them" claim); it holds even when the at-rest bytes are plaintext, because no discovery attestation federates. (2) **At-rest encryption** — the §8.1.12.4 DEK cascade — is **defense-in-depth** (against local-disk forensics / host operator / cloud-substrate operator, the CIRISPersist#152 threat table); it is operator-policy and MAY default off as a v1 migration posture, **but when enabled MUST use `wrap_algorithm: v2` (hybrid PQC, §8.1.12.4)** — never v1. The 1.0 / CEG-RET-native target is at-rest-on for self/family (the "everything PQC at rest" standard).

Composition with at-rest encryption flow (CIRISPersist#152): when self/family at-rest encryption is enabled, persist wraps the DEK (`wrap_algorithm: v2`) under each currently-admitted `identity_occurrence`'s `occurrence_key_id` (self) or each `member.key_id` in the named family's roster (family). New occurrence / new family-member admission triggers retroactive `key_grant` emission for all extant `cohort_scope: self|family` content (the "I bought a new phone and want my Twitter history" / "I added Carol to the household" flows from §5.6.8.9 worked example).

Recipient encryption-key resolution + fail-secure exclusion (CEG 0.18 — CIRISPersist#192 / #69). The wrap target is **not** a recipient's signing key. `wrap_algorithm: v2` needs the recipient's `{x25519, ml_kem_768}` **content-KEM** keys, which the recipient self-certifies via its `identity_occurrence.encryption_pubkeys` (§5.6.8.8.2); the substrate resolves them by `resolve_encryption_keys(key_id)` = the recipient's current (non-superseded, within-`valid_until`) occurrence → its `encryption_pubkeys`. Because this layer **mandates v2**, a recipient whose current occurrence carries **no valid ML-KEM-768 key MUST be fail-secure *excluded*** from the grant — the content remains encrypted and unreachable to it; the substrate MUST NOT fall back to plaintext or to `wrap_algorithm: v1`. To be an at-rest-encryption recipient, an identity MUST have a federation-present occurrence carrying `encryption_pubkeys`. This is the non-maleficence / fail-secure default: a missing key denies access, never downgrades the protection.

Exclusion MUST NOT be silent (1.0-RC1 — #71 C3). A bare `skip` makes a fail-secure exclusion indistinguishable from "the family went quiet" — a soft-censorship vector for a buggy or malicious substrate, and a contradiction of the spec's own `hard_case:*` attestability grain. On ev-

ery fail-secure skip the substrate MUST emit **`hard_case:recipient_excluded:{scope_key_id}` **(§7.7)** into the affected self/family scope itself** — carrying the excluded recipient's `key_id`, a `reason` from the closed set `{expired_occurrence, invalid_kem_key, missing_encryption_pubkeys}`, and the skipped Contribution's envelope ref — so the excluded member (who still sees cohort-scoped attestations) has something to audit and remediate. The event is cohort-scoped: it MUST NOT federate beyond the self/family (the §10.1.4 invisibility promise is preserved; the *fact* of the family's content is not leaked by its exclusion events).

Locality dividend (cewp claim): the structural invisibility mechanism is *why* ~65% of activity stays local in the cewp scaling model — `cohort_scope: self|family` content is the bulk of daily activity (family photos, personal notes, in-household device chatter), and that bulk never federates. Operators do not configure this; the wire format enforces it.

Boundary cases:

- `cohort_scope: community | affiliations | federation` content emits `holds_bytes:sha256:*` per status-quo behavior. CEG 0.7 changes ONLY the self/family path.
- A `cohort_scope: self` Contribution that is later promoted via `supersedes` to `cohort_scope: community` (per §8.1.8.1 Tiered-Scope promotion) emits `holds_bytes:sha256:*` at promotion time on the NEW Contribution. The original `cohort_scope: self` Contribution's bytes remain structurally-invisible at federation; only the promoted scope's bytes propagate.
- `cohort_scope: self` content with `subject_key_ids` containing a non-self party (e.g., a private note Alice writes ABOUT Bob) is admitted and stays in Alice's self-collective; Bob does NOT receive a key_grant unless Bob is also in Alice's self (not the case for two distinct identities). Bob's §4.2 subject-side revocation authority over the note still composes per CEG 0.6, but the bytes never reach Bob without Alice's explicit re-emit at a higher cohort_scope including Bob.

§10.1.3 Consent revocations are NOT local-tier-eligible (CEG 0.6 addition)

Per CIRISRegistry#45 Gap 2 + CIRISAgent#842. The CIRISAgent#840 CEG-native agent design proposes **local-tier signature deferral** — self-attestations skip the hybrid Ed25519 + ML-DSA-65 signature path locally and only sign at federation-tier promotion. This is sound for **producer-only-authority self-attestations**. **The discriminator is *who holds revocation authority*, NOT whether `subject_key_ids` is empty** (clarified per CIRISPersist#172 / CIRISAgent review): a producer-authority self-attestation MAY name a subject in `subject_key_ids` per §4.2.6 — e.g. `observed:user:{hash}:*`, `epistemic:about:{key}:*`, a self-`consent:partnered:{user}`, or the §4.2.3 self-`identity:current` with `subject_key_ids=[self]` — and remains local-tier-eligible, because the producer holds the authority and no *other* subject can revoke it. The single carve-out is below: a Contribution where a **subject other than the producer holds revocation authority** over it.

Consent revocations from subjects MUST NOT use the local-tier deferral path. When a Contribution carries non-empty `subject_key_ids`, any subsequent `consent:state:revoked` emission OR `withdraws` admitted under §3.2.3 rule 2 or 3 from a subject in that set MUST promote to federation-tier within a bounded window. Default window: **24 hours** (operator-tunable per local policy).

Rationale: subject-side revocation is the wire-format observability primitive that federation peers depend on to honor consent. If a user revokes consent in the local-tier scope of one agent's

substrate, and that revocation is unsigned + unpromoted for an extended window, other federation peers continue propagating the user's data — exactly the failure mode CEG 0.6 exists to close.

Substrate emission: substrate MUST emit `hard_case:consent_revocation_promotion_overdue` when a subject-side revocation has been local-tier for longer than the operator-configured window without federation-tier promotion. LensCore composes `detection:consent:promotion_delay_pattern` on top (operator monitoring; not a slashing trigger on its own).

Composition with CIRISAgent#840 self-attestation pattern: the CEG-native agent's `consent:partnered:{user_key}` self-attestation (producer-side stance) MAY ride local-tier; the user's `consent:state:revoked` against the agent's stance Contribution (subject-side) MUST NOT. This preserves the cardinality wins from #840 while closing the leak window.

§10.1.5 The attestation tier model — local-tier write, query, promotion (normative)

Pins the **shared attestation surface** the four CEG-RC1 implementations (CIRISAgent, CIRISNodeCore, CIRISLensCore, CIRISRegistry) write/query/promote through, so it is analyzable + modelable as one contract (per CIRISPersist#172 FSD review). The substrate exposes the *methods*; CEG pins the *wire/conformance semantics* every implementation MUST agree on. **No 1+4 change** — a "tier" is recorded state on an attestation, not a new primitive.

§10.1.5.1 The two tiers

Tier	Signature	Federation-visible	Read-visibility	Written by
`local`	MAY be absent (deferred per §10.1.3)	**No**	**only the producing occurrence** (self-read); every other caller — even an authorized family/community peer — sees nothing	local-tier write
`federation`	hybrid Ed25519 + ML-DSA-65 **present**	Yes	per §4 `cohort_scope` + the §10.1.4 invisibility rule	direct signed write OR `local` → `federation` promotion

Invariant (substrate MUST enforce): `tier = federation  hybrid signature present`. Nothing crosses to federation-visible unsigned. A `local` row is **labelled** local and MUST NOT be served as federation-authoritative (fail-honest). The read-gate is **orthogonal** to `cohort_scope`: it is an additional filter (`local  caller is the producing occurrence`), composing with the §10.1.4 target-membership predicate. Threat entries: AV-59 (local row leaked to a non-self caller), AV-60 (unsigned local served as authoritative), AV-61 (the two gates de-synced).

§10.1.5.2 Local-tier eligibility — the discriminator is *revocation authority*, not subject-set emptiness

A write is local-tier-eligible iff **the producer holds sole revocation authority** over it. This is **NOT** the same as "empty `subject_key_ids`" (the over-narrow CEG 0.6 qualifier, corrected at §10.1.3): a producer-authority self-attestation MAY name a subject per §4.2.6 and remain local-eligible — `observed:user:{hash}:*`, `epistemic:about:{key}:*`, a self-`consent:partnered:{user}`, the §4.2.3 self-`identity:current` with `subject_key_ids=[self]`.

The single carve-out (MUST go signed / promote, never local): a Contribution where a **subject other than the producer holds revocation authority** — concretely a `withdraws` admitted under §3.2.3 rule 2/3 or a `consent:state:revoked` whose `attesting_key_id` is a member of the *target's* `subject_key_ids`. These ride the §10.1.3 24-hour promotion obligation. `witness_relation` MUST be `self` for any local-tier write.

§10.1.5.3 Promotion — `local` → `federation` (the deferred-signature moment)

Promotion computes the hybrid signature and flips the row federation-visible. It is **idempotent** (promoting a `federation` row returns it unchanged), and at the promotion instant the tiered-scope promotion (§8.1.8.1) and `holds_bytes` emission (§10.1.2) fire exactly as for any federation write — **promotion *is* the federation-emit moment**.

Canonical bytes — `JCS(envelope)` per §0.9 / RFC 8785 (normative; resolves CIRISPersist#172 OQ-4):

- The signature MUST cover `JCS(contribution_envelope)` — the **identical** canonical bytes any natively-federation attestation signs. A promoted row is therefore **byte-indistinguishable on the wire from one born federation-tier**; there is no "was-promoted" marker in the signed bytes.
- **Substrate columns are NOT in the canonical bytes.** `tier`, `promoted_at`, and any other storage bookkeeping are substrate state, never part of `JCS(envelope)`. Only the §4 envelope member set is canonicalized.
- **§0.9 omit-vs-materialize is load-bearing here.** Promotion MUST canonicalize the **exact member set the producer committed at local-write time** — a field *omitted* at local write MUST NOT be materialized at promote, and vice-versa, or the recomputed bytes diverge from what a peer verifies and the hybrid sig fails. The substrate serializes the stored row → the committed envelope → JCS → sign; it MUST NOT re-default.
- Registry owns the exact envelope member set (§4 + the §0.9.3 catalog); Verify recomputes the identical JCS bytes (CIRISVerify#59). Do NOT use Verify's internal length-prefixed `signing_bytes` framing — that is for verify-to-verify primitives that never cross the four-impl boundary as JSON; a promoted attestation is the opposite.

§10.1.5.4 Query — open-prefix dimensions, bounded operators (normative; resolves CIRISPersist#172 OQ-2)

The uniform read filters on `(dimensions[], valid_at, confidence_floor, subject_key_id?, scope)`.

- **`dimensions[]` is an OPEN-vocabulary set of prefix strings, matched by hierarchical prefix — NOT a closed enum.** Per §11.2.1 axis-vocabulary discipline, dimension prefixes are open vocab (the 0.6→0.15 cadence shipped new families — `consent:*`, `detection:community:*`, `settlement:*` — continuously). A closed enum would force a substrate redeploy per CEG namespace addition and break forward-compat. A query for `detection:*` matches `detection:correlated_action:bribery` etc.; an exact string matches exactly.
- **The bounded surface is the *operator set*, not the *vocabulary*. The apophatic discipline (a fixed, named surface — not an OLAP/graph engine) is preserved by the**

closed set of five predicates** above + no caller-composed SQL/projections — NOT by closing the dimension vocabulary. *Open data, closed operators.*

- `valid_at` filters `asserted_at ≤ valid_at < COALESCE(expires_at, )`; `confidence_floor` filters `weight ≥ floor`; `scope` applies BOTH the §10.1.4 target-membership gate AND the §10.1.5.1 tier gate. Write-side reserved-prefix emitter gates (§7.0.1) are unaffected — read is gated by scope, not by dimension authority.

§10.1.5.5 For holistic analysis + modeling (informative)

The tier model is the *same KEM-then-symmetric placement* the streaming model uses (§10.5), applied to attestations: the **expensive op (hybrid sign) is on the cold path** (promotion, at federation-emit), while the **hot path (local self-attestation write) is O(1) and unsigned**. Cost shape for a node:

- **local write**: O(1), no asymmetric crypto — the agent's steady-state memory/config/consent/identity churn.
- **promotion**: one hybrid Ed25519+ML-DSA-65 sign (~the §10.5-model per-op cost; ML-DSA-65 sign ~330 µs) + JCS canonicalization — only at the federation-emit moment, never per local write.
- **query**: the §4/DAS scope-filtered read; cost is the predicate + index, independent of tier.
- **observability for modeling**: `local` row count, promotion rate, and `hard_case:consent_revocation_p` count are the three measurable signals; the §10.1.3 24-hour window is the one tunable. A model of a federation's attestation load is therefore *(local-write rate, promotion rate, query rate)* with promotion carrying the only asymmetric-crypto cost — the dual of the streaming model's epoch-rekey tail.

§10.2 Multi-steward + accord-holder discovery

`GET /v1/steward-key`

Returns the multi-steward set with M-of-N policy.

Response (`200 OK`):

```
{
  "stewards": [
    {
      "region": "us",
      "key_id": "us-steward-2026",
      "ed25519_pubkey_b64": "<base64-url>",
      "mldsa65_pubkey_b64": "<base64-url>",
      "hardware_class": "HSM_FIPS_140_3_L3",
      "deployed": true,
      "fingerprint_sha256_hex": "<64-char-lowercase>",
      "cert_validity_self_attest": {
        "valid_until": "<rfc3339_canonical>",
        "signature_b64": "<base64-url>"
      }
    },
    {"region": "eu", ..., "deployed": false},
    {"region": "apac", ..., "deployed": false}
```

```
  ],
  "threshold_policy": {"required": 2, "available": 1},
  "response_signature": {
    "signer_key_id": "us-steward-2026",
    "ed25519_b64": "<base64-url>",
    "mldsa65_b64": "<base64-url>",
    "canonical_bytes_label": "ciris.steward_key_response.v1"
  }
}
```

The response itself is hybrid-signed by the serving region's steward over `canonical = "ciris.steward_key_res || sha256_hex_lowercase(canonicalized_json_body_excluding_signature)`. Consumers MUST verify the response signature before trusting any field in the body — placeholder pubkeys without `deployed: true` MUST NOT be promoted to trust roots.

`GET /v1/accord-holders`

Three named holders with hybrid pubkeys + per-holder `hardware_class` + `provisioned` flag. v1.4 interim ships with placeholder fingerprints + `provisioned: false`; consumers MUST NOT honor CONSTITUTIONAL invocations against placeholders. Response signed by the serving region's steward (same shape as `/v1/steward-key`).

`GET /v1/accord/holders`

UI wrapper around `/v1/accord-holders` with per-holder `accord_emissions[]` for UI rendering. Same response-signing requirement.

`GET /v1/rotation-history`

Chronological rotation events from `registry_signing_keys` table. Substrate-conformance migration moves to `federation_keys`.

§10.3 STH cosigning + witness directory

CIRISVerify v2.12.0+ ships consumer-side `SignedTreeHead::cosign` + `count_valid_witnesses` + `witness_quorum_met`. CEG's emission half:

`POST /v1/transparency/sth/cosign` (public)

Witness posts cosignature on `(tree_size, root_hash, signed_at)`. Registry verifies hybrid Ed25519 + ML-DSA-65 against witness pubkey in directory; persists on success.

Request body:

```
{
  "tree_size": <int>,
  "root_hash_sha256_hex": "<64-char-lowercase>", // per S0.6
  "signed_at": "<rfc3339_canonical>", // per S0.5
  "witness_key_id": "<string>",
  "ed25519_signature_b64": "<base64-url>",
  "mldsa65_signature_b64": "<base64-url>",
  "consistency_proof_root_hash_sha256_hex": "<64-char-lowercase>",
```

```
  "consistency_proof_tree_size": <int>,
  "consistency_proof_path_b64": ["<base64-url>", ...]
}
```

Canonical bytes (witness MUST sign these):

```
canonical = sha256(
    "ciris.sth_cosign.v1\n" ||
    "tree_size=" || decimal_no_leading_zeros || "\n" ||
    "root_hash_sha256=" || sha256_hex_lowercase || "\n" || // per S0.6
    "signed_at=" || rfc3339_canonical // per S0.5
)
```

Ed25519 over `canonical`; ML-DSA-65 over `canonical || ed25519_sig` (bound payload).

§10.3.1 Consistency-proof requirement (normative; addresses CEG 0.1 distsys review)

A witness signing an STH MUST first verify a consistency proof from the prior STH it cosigned (or from genesis if it is the witness's first cosignature against this log). The Registry MUST reject `POST /v1/transparency/sth/cosign` requests that omit the `consistency_proof_*` fields OR whose consistency proof does not verify against the named prior STH. `witness_quorum_met` is therefore "quorum on log consistency," not "quorum on a string."

ENFORCED as of Registry v2.3.0 (CIRISRegistry#34). The cosign request carries `consistency_proof_pa` (base64 RFC 6962 §2.1.2 node hashes). The Registry anchors the check against the prior `(tree_size, root_hash)` **it recorded** for that witness — not a root the requester claims — and rejects: missing proof when a prior STH exists or a tree_size behind the witness's prior cosigned STH → `MALFORMED_REQUEST`; a proof that does not reconstruct both roots → `CONSISTENCY_PROOF_INVALID` (§10.0.1). A witness's first cosignature is exempt ("from genesis"). The RFC 6962 verifier is vendored from `ciris-verify-core::transparency` (Registry omits that crate for a libsqlite3 linker reason) and proven against independent known-answer vectors.

`GET /v1/transparency/witnesses` (public)

Directory of registered witnesses. Paginated.

`GET /v1/transparency/sth/{tree_size}/witnesses` (public)

Cosignatures for an STH with `witness_quorum_met` verdict.

`POST /v1/transparency/witnesses` (admin; multi-party-gated)

Register a new witness. **0.1 scaffold note**: in 0.1 interim this is bearer-token-gated by `REGISTRY_ADMIN_TOKEN`. **0.2 hardens this to 2-of-3 steward sign-off** (addressing CEG 0.1 cryptographic + red-team review): the request body MUST carry signatures from at least two of the three regional stewards, verified against `GET /v1/steward-key`. Single-token admission is a 0.1 known weakness; production deployments SHOULD operate the 0.1 endpoint behind a corporate IDP gate that enforces multi-party admission out-of-band until the 0.2 multi-sig requirement is normative.

§10.4 Other Registry endpoints

`GET /v1/builds/{version}` returns the BuildRecordResponse with a `federation_provenance` block (per §5.2 SLSA emission discipline). `GET /v1/verify/build-manifest/{project}/{version}/{target}` (Path B) returns the verbatim signed BuildManifest. `GET /v1/agent_files/{kind}?platform_or_target=...` returns the §8.1.6 trust-composition layers. `GET /v1/partner/{key_id}` composes ProfileScorecard data from existing tables.

Full response schemas for these endpoints land in the Rust handlers + OpenAPI export; CEG 0.2 commits to publishing a versioned OpenAPI spec alongside this document.

§10.5 Streaming transport, per-stream logs & delivery receipts (CEG 0.10 addition)

Per CIRISRegistry#44 absorbed + CIRISLensCore#857 (observer-share driver) + CIRISPersist#142 (streaming substrate prerequisite). §10.5 is the **delivery axis** — the third orthogonal envelope concern alongside visibility (`cohort_scope`) and revocability (`subject_key_ids` per §4.2). §3's 1+4 primitive set is untouched; §10.5 is endpoint + envelope + composition extension, NOT a grammar change.

Bifurcation:

Half	Cardinality	RC1 status
Observer-share / directed delivery (single Contribution → subscriber-set; no `stream_id`)	N=1 typical; per-subscriber `key_grant`	**impl-live**; substrate paths shipping per §5.6.8.4 `key_grant` + §8.1.13 Policy M membership
Media / streaming multicast (`live_stream` chunk-DAG; per-`(stream_id, epoch)` keys)	N>1; flat per-epoch `key_grant` cascade	**spec-now, impl substrate-pending CIRISPersist#142**; subsections §10.5.2 / §10.5.3 / §10.5.4 ride this dependency

§10.5.0 Framing (normative)

A stream is **its own per-stream transparency-log instance** (`log_id = stream_id`). A `live_stream` (CEG 0.5 sub_kind absorbed into 0.10) MUST NOT append chunks into the federation provenance log (§10.3's global log carrying builds / licenses / identities) — millions of media chunks would pollute provenance and inflate the global tree. The §10.5 path **reuses** the §10.3 `SignedTreeHead` / `ConsistencyProof` / `WitnessConsistencyProof` / cosign abstractions, instantiated per-stream as separate log instances under the same RFC 6962 algorithm.

The 1+4 wire-format lockdown holds: there are no new `attestation_type` values. Stream chunks ride content addressing; stream-roots ride the existing `SignedTreeHead` shape; delivery receipts ride `scores` against the new `delivery_receipt:{stream_id}` reserved prefix (§7.9).

§10.5.1 Per-stream log + stream-root (normative — V1 lock)

For each live_stream:

- `log_id = stream_id`; chunks = leaves; stream-root = `SignedTreeHead{ log_id: stream_id, tree_size: chunk_count, root_hash, timestamp, signature }`

- **Producer signs the STH — MANDATORY** authenticity root; hybrid Ed25519 + ML-DSA-65 per the §10.3 `signing_bytes` discipline; canonical bytes per §0.9 JCS for the envelope-bearing wrapper
- **Witness cosign — OPTIONAL**, via the §10.3 path verbatim. This is the best-effort / accountable split (D5 per §15.6.2):
- **Best-effort** (open media) → producer-signed root only; impl-pending CIRISPersist#142 only
- **Accountable** (paid media, registry propagation, emergency) → witness cosign per §10.3.1 consistency-proof, which is the **anti-equivocation guarantee** — producer cannot show different chunk-K to different subscribers nor rewrite mid-stream. Accountable tier is impl-pending BOTH #142 AND CIRISRegistry#34 (STH consistency-proof enforcement)
- **Cadence**: producer publishes a signed root every `K` chunks OR `T` seconds (whichever first), **always at an epoch boundary** (§10.5.3) + at `sealed_at`. Witness cosign runs **per-epoch** (coarser than per-K to keep cosign-quorum cost off the hot path). Default pins: **K=64, T=2s** (operator-tunable; ratification pending per §15.6.3 RC1-7)
- **Incremental verify** (D4): each leaf's "root-after-K" lets a subscriber verify chunk K's inclusion against the nearest signed STH $\geq$ K via the §10.3.1 consistency path. No new commitment structure beyond the per-leaf chain-link + periodic STH that §10.3 already provides.
- **Accountable-stream quorum**: Policy E (§8.1.5 locality-scaled quorum) applies — not a fixed N. Emergency-channel roots want a higher quorum than a paid-media stream; locality scaling provides the gradient.

§10.5.2 Chunk seal + STREAM nonce (normative — V2 lock)

Per-chunk content sealing **conforms to SFrame** (draft-ietf-sframe, normative): the per-frame AEAD model with a `(KID, CTR)` header, here bound as `KID = stream_id` and `CTR = counter`, AEAD = AES-256-GCM, keys derived per MLS epoch (the canonical SFrame+MLS composition, §10.5.3). The STREAM nonce below is CEG's binding profile of SFrame's per-sender nonce derivation. Per-chunk content sealing uses AES-256-GCM (NIST FIPS 197 + SP 800-38D). The 12-byte (96-bit) nonce follows the **STREAM** layout (Hoang, Reyhanitabar, Rogaway, Vizár — *Online Authenticated-Encryption and its Nonce-Reuse Misuse-Resistance*, CRYPTO 2015):

```
nonce[12] = prefix[7] || counter_be[4] || last_flag[1]
```

- **`prefix[7]`** — derived, NOT transmitted: `prefix = HKDF-SHA256(epoch_dek; info)[0..7]` where the HKDF `info` is the **byte-exact** concatenation (pinned per CIRISRegistry#63 — cross-impl interop): `info = b"ciris-stream-nonce/v1"` ‖ `stream_id_utf8` ‖ `epoch_be8`, with `stream_id_utf8` = the UTF-8 bytes of `stream_id` and **`epoch_be8` = the `u64` epoch as 8-byte big-endian** (`epoch.to_be_bytes()`, consistent with `counter_be`). **This encoding is normative and MUST be byte-identical across producer, substrate, and every consumer** — the §10.5.3 zero-trust-of-host posture has consumers recompute this exact nonce to *open* chunks, so any BE/LE/ASCII disagreement on `epoch` yields a different `prefix` → different nonce → GCM auth-tag failure (silent whole-stream decryption failure), the same hazard class `counter_be` already guards. No length-prefix on `stream_id` is required: the fixed-length tag prefix + fixed 8-byte `epoch` suffix make the parse unambiguous (distinct `(stream_id, epoch)` pairs always yield distinct `info` — unequal `stream_id` length changes total `info` length; equal length splits unambiguously). Matches

the **KEY_GRANT_V1_INFO** versioned-context HKDF pattern at CIRISVerify/src/ciris-crypto/src/l (b"cewp-key-grant/v1"). Per-(stream_id, epoch) unique; verifiable by any holder of the epoch DEK

- **counter_be[4] — **32-bit big-endian; hard ceiling 2^{32}-1 chunks per epoch. Substrate MUST force an epoch roll before wrap. Recommended operational cap: MAX_CHUNKS_PER_EPOCH = 2^{24}**** (~16.7M chunks/epoch) to keep per-epoch state + proof sizes bounded (operator-tunable; ratification pending per RC1-7)
- **last_flag[1]** — 0x01 on the final chunk of an epoch (sealed by seal_stream per §10.5.3); 0x00 otherwise. The distinct nonce on the final chunk gives **truncation + append resistance**: an adversary cannot drop the final chunk and pass off a short stream, nor append past a sealed segment

Cross-epoch counter reset is nonce-safe (normative reasoning): GCM's catastrophic case is reuse of a (key, nonce) pair. On epoch roll the DEK changes, so a reset counter lives in a different keyspace — (DEK_e, nonce=0) and (DEK_{e+1}, nonce=0) are distinct pairs. The enforced invariant is therefore only within a single epoch: counter strictly monotonic, never wraps (guaranteed by the forced roll). Across epochs, reset is free.

Single-sender-per-(stream_id, epoch) invariant (normative — nonce-collision safety; cribbed from SFrame/MLS): the counter_be[4] space of a (stream_id, epoch) is owned by **exactly one** sender — the stream_id's producer. Two distinct senders MUST NOT seal chunks under the same (stream_id, epoch) DEK, or their independently-incremented counters could collide into a reused (DEK, nonce) pair (GCM-catastrophic). In **group video** (§10.5.8) each participant emits their **own** live_stream with their **own** stream_id, so the nonce prefix HKDF(epoch_dek; "ciris-stream-nonce/v1" ‖ stream_id ‖ epoch) is per-sender-unique by construction — the same hazard SFrame avoids with per-sender keys, achieved here by per-sender stream_id. Substrate MUST reject a chunk-append whose (stream_id, seq) collides with an existing sealed chunk (replay/forking guard, §10.5.3).

§10.5.3 Epoch keying + cascade (normative — D2 / D3; substrate-pending #142)

The stream-epoch DEK seals content **O(1)**; the per-subscriber key_grant cascade distributes the 32-byte epoch key **O(N)/epoch** (sender-key / Megolm shape) = §8.1.12.4 Policy-L cascade applied to a community roster against a *rotating* key.

PQC at rest — wrap_algorithm: v2 MANDATORY (normative). The epoch-DEK key_grant cascade MUST wrap the DEK with **wrap_algorithm: v2 = x25519+ml-kem-768 (hybrid; FIPS 203)** — never v1 (X25519-only). The DEK protects content that may persist indefinitely; a classical-only KEM is a harvest-now-decrypt-later exposure even though the content AEAD (AES-256-GCM, §10.5.2) and the wrap *signature* (Ed25519 + ML-DSA-65) are already PQC-safe. The hybrid KEM primitive is shipped in ciris-crypto (hybrid_kex / ml_kem, CIRISVerify#47); the versioned KEY_GRANT_V1_INFO HKDF-context rotates cleanly to the v2 context. A Consumer MUST reject a streaming epoch grant carrying wrap_algorithm: v1. **The v2 wrap_algorithm payload wire string is pinned (normative, per #64): x25519_mlkem768_aes256_gcm_hkdf_sha (the §5.6.8.4 vocab variant X25519MlKem768Aes256GcmHkdfSha256; matches ciris-crypto KEY_GRANT_ALGORI Producer, substrate, and every consumer MUST serialize/deserialize this exact string — a mismatch silently fails grant decode. **Full PQC envelope for streaming**: content = AES-256-GCM (symmetric, PQC-safe) · DEK wrap = X25519+ML-KEM-768 · authenticity = Ed25519+ML-DSA-65 · hashes = SHA-256 (PQC-safe) · in-transit = §10.5.5 E1 two-layer hybrid (below).

Epoch index is monotonic, per-`stream_id`, greenfield — **a separate addressing axis** from `key_grant.rotation_chain`:

Axis	Addressing	Where it lives	Supersession
Content-addressed grant supersession (CEG 0.3)	`(content_sha256, recipient_key_id)`	`cirisnode_contributions` V054 partial indexes (planner-AND'd)	`rotation_chain` payload-level lineage (list of prior `key_grant_id`s); walked reader-side
Stream/epoch-addressed grant supersession (CEG 0.10)	`(stream_id, epoch[, recipient])`	`federation_stream_chunks(stream_id, seq)` (Persist#142 step 3; v3.9.0 target; **NOT YET LANDED — unowned/unscheduled**)	Same `rotation_chain` payload-level supersession **reused on the new axis** (RC1-1 [✓] Persist on record)

[!] **Not pure-additive at the Persist constraint layer (RC1-1c)**: the V054 cross-column CHECK requires `key_grant` rows be content-addressed (`media_content_sha256 IS NOT NULL`). The §10.5.3 epoch-key axis with NULL `media_content_sha256` would be REJECTED by today's CHECK. Introducing the new axis requires a **parallel CHECK arm migration** at Persist (content-addressed OR stream/epoch-addressed) — a bounded constraint migration, not a pure index-add. The spec text does not claim "purely additive" at the Persist constraint layer.

Epoch triggers (D3):

Trigger	Behavior	Forward-secrecy implication
Member removal	**MANDATORY rotation** (coalesced per below; exempt for ungated public broadcasts per below) — the forward-only-unsubscribe enforcement	Subsequent epochs sealed under a DEK the removed member doesn't have
Member addition	NO rotation + Option-A catch-up per §11.7.1 (subject to `history_on_join`)	New member gets `key_grant`s for the current epoch + (optionally) prior epochs per the `history_on_join` envelope field (§4)
Time / bytes	Optional hygiene rotation	Operator policy; default off

Removal coalescing (normative — 1.0-RC1, #71 C1). At broadcast scale, naive per-removal rotation is unaffordable: at $N = 10^6$ and realistic audience churn (~30%/hr), one rotation per departure ≈ 83 epochs/s $\rightarrow$ a flat-unicast cascade of ~**3.1 Tbps — exceeding the ~2 Tbps content fan-out itself** (the original model's 3,600/hr churn figure understated broadcast churn by ~2 orders of magnitude). The substrate therefore MUST **coalesce removals**: all removals admitted within one STH cadence window **T (default 2 s, the §10.5.4 equivocation-window grain)** are batched into a **single** epoch rotation covering the whole batch. This caps the rotation rate at 1/T **regardless of churn** ($\leq$ 1,800 epochs/hr at T = 2 s $\rightarrow$ ~18.7 Gbps flat-unicast cascade at $N = 10^6$, ~0.9% of content fan-out — the headline restored honestly), and bounds removed-viewer exposure to $\leq$ T — the same grain the stream's equivocation window already accepts. A removed member's exposure window is therefore $\leq$ `T`, never "until the next scheduled rotation."

Public-broadcast exemption (normative). A `live_stream` whose roster is **ungated** — `listed: public` AND grants issued to any requester without an admission ceremony — carries **no confidentiality claim against departed viewers** (anyone, including the departed viewer, can re-subscribe and receive the current DEK on request). For such streams, member removal MUST NOT force rotation (it buys nothing and costs the full cascade); the time/bytes hygiene

rotation and the §10.5.2 nonce-space bounds still apply. Rotation-on-removal remains **MANDATORY for every gated roster** (bounded membership, admission ceremony, or any non-public `listed` state) — there the DEK *is* the access-control boundary and the forward-only-unsubscribe guarantee is real.

Rekey conforms to MLS TreeKEM (RFC 9420, normative). The epoch-DEK rekey on member change is the MLS TreeKEM construction: an O(log N) path rekey, the commit signed once, with RFC 9420 blank-node / unmerged-leaf handling and parent-hash tree integrity. The hybrid KEM (X25519+ML-KEM-768) is the MLS ciphersuite's HPKE KEM.

Delivery is decisive (carried from the bandwidth model). The TreeKEM advantage is *multicast aggregation*, not the tree itself: with efficient multicast the commit egress is **O(log N)** (one commit serves all); over unicast the commit must reach each member, **O(N log N)** — in which case the **flat per-member cascade (O(N), one grant per member) is competitive-to-better**. The substrate MUST select per deployment based on whether the transport multicasts; the choice is **wire-invisible** (tree position rides the opaque `key_grant` payload; no schema migration). Whether the RET mesh offers efficient multicast is the open transport question (the bandwidth model's one free parameter).

Forward secrecy is forward-only by default (§11.7.1 Option A). **Post-Compromise Security (PCS)** is now AVAILABLE (inherent to TreeKEM key-updates: a compromised current member heals on the next commit) and is **OPTIONAL, operator-enabled** — a deployment MAY require periodic self-updates for PCS, or stay forward-only.

Catch-up bound (P4): `min(operator depth cap [LensCore knob, NOT a substrate constant], chunk-eviction horizon)`. Three distinct windows that are NOT conflated: chunk-eviction horizon ≠ §10.1.2 `holds_bytes` 24h TTL ≠ grant durability. A catch-up request against an evicted epoch returns **`ContentMiss` — fail-honest, no silent gap** (consistent with the `MISSION.md` fail-honest invariant). Operators MUST ship the P4 cap **with** the cascade, else 10^6 grant Contributions per rekey is the unbounded worst case.

§10.5.4 Delivery receipts (normative — D5 / V3 lock)

A `delivery_receipt:{stream_id}` Contribution (new reserved prefix per §7.9) is a subscriber's signed acknowledgement that they received chunk K under the named stream + epoch. **Best-effort default**; opt-in for **accountable** profiles (registry propagation, emergency).

Canonical bytes (domain-separated + length-prefixed, matching `SignedTreeHead::signing_bytes` discipline; per §0.9 the envelope-bearing wrapper is JCS-encoded, but the receipt's signed-bytes inner payload follows the explicit-length-prefix shape below):

```
receipt_signing_bytes =
    "ciris-delivery-receipt/v1" // domain separator
  || len(subscriber_key) || subscriber_key
  || len(stream_id) || stream_id
  || epoch (u64 LE)
  || chunk_root ([u8; 32])
  || K (u64 LE)
```

Both `epoch` (key-rotation index — for per-epoch entitlement / billing) and `K`/`chunk_root` (chunk position + its committed root) are independent indices — both required (each names a distinct authorization scope).

Verify check is a JOIN, NOT a sig-check:

1. **Signature valid** over the canonical bytes (subscriber hybrid Ed25519 + ML-DSA-65 sig). Necessary but **not sufficient**.
2. **`chunk_root` is a real published STH root** — MUST equal a `SignedTreeHead.root_hash` actually published for `log_id = stream_id` at `tree_size` $\geq$ `K`. A phantom / self-invented root → REJECT. For accountable streams, "published" means **witness-cosigned** (the §10.3 path), so the subscriber cannot collude with the producer on a private root.
3. *(Recommended for accountable)* **Inclusion proof** chunk K → `chunk_root`. Upgrades the receipt from "subscriber saw a root" to "subscriber saw a root that provably commits to chunk K".

Semantics — proof-of-DELIVERY, not proof-of-CONSUMPTION: the receipt proves the subscriber received bytes committing to chunk K. It does NOT prove they decrypted those bytes (they may not hold the epoch DEK). Consumers MUST NOT overclaim a delivery receipt as proof of consumption. Per the `MISSION.md` fail-honest invariant + §1.4 "Verify authenticates origin, does not compose 'delivered'/'owes N'", Verify's role is validation-not-adjudication: emit the validated receipt as an attestation on `delivery_receipt:{stream_id}` — the "delivered" verdict is consumer policy.

Accountable-stream receipt quorum: Policy E (§8.1.5 locality-scaled) — same shape as the §10.5.1 STH quorum. Ratification pending per RC1-7.

§10.5.5 Transport — Edge layer (normative — E1–E4 lock per §15.6.2)

Decision	Behavior
E2 — pull-only RC1	RC1 multicast = **pull-only**: producer seals chunks under the epoch DEK → emits `holds_bytes:sha256:*` → subscribers pull via the existing `ContentFetch` path. Relay / fan-out tree → 1.x (CIRISRegistry#46 / #43)
E1 — two-layer crypto (security-critical)	Transit-key (per CIRISLensCore#857 prod-lens-via-transit-key) is a **hop-by-hop transport wrap UNDER the E2E epoch DEK** (two independent crypto layers). MUST NOT replace the cascade — a relay never sees plaintext. Transit-key is for path-confidentiality; epoch-DEK is for end-to-end content confidentiality. **PQC in transit (normative):** BOTH layers MUST use PQC-grade key agreement — the transit-key wrap MUST be hybrid X25519+ML-KEM-768 (same `hybrid_kex` primitive as §10.5.3), and the underlying transport (Edge/Reticulum) MUST negotiate the hybrid/PQC crypto-kind (`CIR2`, per `CIRISVerify/docs/CRYPTO_AGILITY.md`), never classical-only `CIR1`. Classical-only transport is rejected for streaming.
E3 — fan-out = entitled ∧ reachable	**Persist owns durable entitlement** (the roster: signed CEG envelopes, replicated, logged). **Edge owns transport-reachability** via `reachability.rs` (CIRISEdge#29) node-local presence tracker. Fan-out targets the intersection. Reachability is NEVER an attestation, never `holds_bytes`, never replicated, never logged — consistent with the §10.1.4 'cohort_scope: self\

Decision	Behavior
E4 — durable side rides existing federation-attestation path	Durable entitlement (roster + epoch-key grants) rides the **existing federation-attestation Edge path** (CIRISRegistry#41 handler cutover) — just more `federation_attestations` rows. NO net-new Edge transport for the durable side. Net-new is only on §10.5.1 streaming-log endpoints

§10.5.6 D6 liveness invariant — entitled vs reachable (normative)

Two sets are NEVER conflated:

- **Entitlement roster** (Persist-owned): signed CEG envelope, Edge-propagated, durable, logged. It's **evidence** — it MUST propagate + be auditable. Per §8.1.13 Policy M community-membership composition
- **Live-reachability set** (Edge-owned): generalizes the §10.1.2 `EdgeConfig.holds_bytes_ttl_seconds` 24h default down to seconds-to-minutes for live-multicast. Node-local presence tracker (Edge `reachability.rs` per CIRISEdge#29). **NEVER an attestation, never `holds_bytes`, never replicated, never logged**

Fan-out invariant: `fan_out(C) = entitled(C) ∩ reachable(now)`.

Heartbeat-suppression discipline: this is a **producer-side-refusal invariant** (same class as the §10.1.4 `cohort_scope: self|family holds_bytes` suppression). Missed (entitled-but-unreachable) members fall back to pull on reconnect — substrate does NOT keep retrying push, does NOT emit a "delivery_failed" attestation, does NOT log liveness state. The reconnect-then-pull catch-up rides §10.5.3 `history_on_join`.

§10.5.8 Realtime group communication — composition (CEG 0.13 addition)

Per CIRISRegistry#56. The delivery axis was framed for 1:1 observer-share and 1:N broadcast multicast. Realtime **group communication** — group video, voice, desktop/screen sharing, text chat, and topic-scoped channels with sub-channels — is the **same primitive set at N↔N cardinality**. It composes entirely from `community` (§5.6.8.10) + `live_stream` (the §10.5 streaming surface) + `chat_message` (CEG 0.3) + member/transport resolution (§8.1.13.1.1). **Zero new structural primitives** — this is the thirteenth path on the §1.4 claim and the most product-complete: the whole realtime-collaboration surface is composition.

The composition map (all generic — no new wire):

Surface	Composes from
1:1 / group video call	each participant emits a `live_stream` (their A/V) scoped to the channel `community`; each subscribes to peers. N↔N = N simultaneous bidirectional streams over a small roster
Voice channel	identical to group video with an audio-only codec; same `live_stream` wire
Desktop / screen sharing	a `live_stream` whose source is a screen capture (1→N within the channel); same chunk-seal + epoch-DEK wire

Surface	Composes from
Text chat	`chat_message` (CEG 0.3) at `cohort_scope: community`, `community_id: <channel>`; threads via `topical_relation: replies_to`
Channel	a `community` (persistent) — its roster gates who can join the call / read the chat
Sub-channels	nested `community` membership (§8.1.13.5 multi-level pattern): a parent "space" community whose members admit child channel-communities; sub-channel members are a subset of the space roster
Presence ("who's here")	the D6 reachable set (§10.5.6) — node-local, never an attestation, never logged
Invite / join / leave	community admission ceremony (§8.1.13.2) — invite = membership proposal; join = admitted member; leave = forward-only `withdraws`

Transport profiles (normative — extends §10.5.5):

Profile	Topology	Transport	RC1 (1.0)
Broadcast (1:N, large N)	asymmetric	pull-only `ContentFetch` (§10.5.5 E2); relay tree → 1.x	[✓] pull
Realtime small/medium group (calls, huddles, ≤ ~50)	symmetric mesh	**direct Reticulum Links between participants** (low-latency push; no relay tree needed at small N)	[✓] direct-link mesh
Realtime large group (≫50 in one A/V room)	fan-out	selective-forwarding relay (SFU role)	→ 1.x (same relay-tree axis as broadcast scale)

The low-latency realtime profile (direct Reticulum Links) is **in 1.0 scope** because small/medium rosters need no relay tree — RNS Links give encrypted low-latency point-to-point natively. The wire is identical to broadcast (chunk-seal §10.5.2, per-`(stream_id, epoch)` epoch DEK §10.5.3, per-stream STH §10.5.1); only the transport profile + roster size differ. **PQC is unchanged and mandatory** — epoch DEK wrapped `wrap_algorithm: v2` (X25519+ML-KEM-768) at rest, hybrid KEX in transit.

Ephemeral vs persistent rosters: a persistent channel is a long-lived `community`; an ad-hoc call is an **ephemeral `community`** (short `valid_until`, torn down on last-leave). Same primitive, different lifetime — no special-case "session" primitive.

The breadth confirmation: a full realtime group-communication platform — group video, voice, screen sharing, chat, and channels-with-sub-channels — requires **no addition to the 1+4 structural set** beyond the §10.5 delivery axis and the `community` membership machinery already locked. #### §10.5.8.1 Realtime non-A/V data streams (normative scope boundary)

The realtime profile is **not media-only.** Any high-frequency mutable shared state — multiplayer game ticks, collaborative-editing operations (CRDT/OT), live cursors/whiteboard strokes, remote-control input, high-rate telemetry — rides the **same** §10.5.8 realtime transport (direct Reticulum Links / SFU at scale) with an application-defined payload codec in place of an A/V codec. The wire is identical: per-`(stream_id, epoch)` epoch DEK (§10.5.3, `wrap_algorithm: v2` PQC), STREAM-nonce chunk seal (§10.5.2), per-stream STH (§10.5.1). A data-stream chunk is just a chunk.

Scope boundary (the explicit call):

- **IN scope (transport):** ordered, sealed, authenticated, PQC-encrypted realtime delivery of arbitrary data-stream payloads — covered by §10.5.8, no addition.
- **OUT of scope (merge semantic):** the conflict-free / convergent-merge logic for shared mutable state (CRDT, Operational Transform, last-writer-wins, etc.) is **application-layer**, not a CEG primitive — consistent with the §1.1 discipline ("the substrate stores; the wire transports; CEG describes the shape of the claim; consumer policy composes verdicts"). CEG carries the ops; the application converges them. A future codified merge primitive would route through the §11.2 amendment process if real demand pulls it (the downstream-demand-pulls-additions discipline), but it is explicitly NOT required for 1.0 — realtime collaborative apps are buildable on the transport today.

§10.5.7 What CEG 0.10 documents

- The delivery axis as the third orthogonal envelope concern (visibility + revocability + delivery)
- The bifurcated observer-share (impl-live) vs streaming multicast (substrate-pending #142) split
- Per-stream transparency-log instances (§10.5.1)
- STREAM nonce derivation (§10.5.2) reusing the `KEY_GRANT_V1_INFO` HKDF pattern
- Epoch-keying cascade on a separate addressing axis from `rotation_chain` (§10.5.3)
- Delivery-receipt canonical bytes + the validated-not-adjudicated discipline (§10.5.4)
- Edge transport with two-layer crypto + pull-only RC1 + entitled-∧-reachable fan-out (§10.5.5)
- D6 liveness invariant (§10.5.6)

What CEG 0.10 does NOT do:

- Change the 1+4 primitive set (§3) — delivery rides existing primitives
- Bundle the streaming-half substrate impl in Persist — that lives at Persist#142 + the RC1-1c parallel CHECK arm migration
- Specify the relay / fan-out tree shape for push-mode multicast — RC1 is pull-only; push tree → 1.x (CIRISRegistry#46 / #43)
- Lock the constants K / T / MAX_CHUNKS_PER_EPOCH or the accountable-stream quorum — operator-tunable; ratification pending per §15.6.3 RC1-7

§11 Governance discipline

§11.1 Operational-language gate at admission

Every new prefix admitted to the §5 namespace passes the §1.3.1 four-test gate. Failed admissions are revised (mechanism-descriptive reframe) or rejected.

§11.2 Amendment process — federation Contribution + WA quorum + 1-of-6 sign-off

Rule-layer changes (new prefixes, new envelope fields, new policies, calibration package version transitions) route through:

1. **Proposed amendment** filed as a NodeCore P5 Contribution (kind: `PROPOSAL`, subject: the proposal artifact).
2. **Witness diversity** per NodeCore P10 (N=3 default).
3. **WA quorum adjudication** per NodeCore P8.
4. **Reconsideration** per NodeCore P11 with fresh-quorum recusal (per §8.1.5 locality-scaled-quorum, including the §8.1.5.1 sub-quorum fallback).
5. **1-of-6 accord-holder OR steward sign-off** as defense-in-depth gate against rules-layer Sybil capture. The 1-of-6 sign-off is the secondary check; WA quorum is the primary substantive review. Any single signer can VETO by refusing to sign. Reduces the attack surface from "produce N Sybils" to "compromise one of six specific hardware-attested keys."

§11.2.1 Axis-vocabulary discipline

Every `{axis}` value emittable under open-vocabulary prefixes (e.g., `detection:correlated_action:{axis}`, `hard_case:{kind}`, `testimonial_witness:{kind}`) MUST carry an operational definition where the prefix has a calibration package (RATCHET-calibrated detectors) or a documented convention where it doesn't.

For RATCHET-calibrated detectors, the operational definition lives in the calibration package version pinned via `evidence_refs[]`:

1. Measurement procedure
2. Threshold function
3. Statistical floor
4. Evidence-shape requirement
5. Polarity semantics

For documentation-only open vocabularies (`testimonial_witness:{kind}`, `hard_case:{kind}`, `topical_relation:{kind}`), discoverability lives in non-normative registry documents like `WITNESS_KIND_REGI` — additions there require no spec amendment.

§11.2.2 Open-vocabulary collision rule

When two parties independently register confusingly-similar `{kind}` / `{axis}` values within the same prefix family, the following resolution applies:

1. **First-registered wins** for the canonical-attestation surface. The earlier `signed_at` holds the name; later registrations carry a `differs_in: ["semantic_disambiguation"]` clarification or pick a distinct value.
2. **Levenshtein-distance guard**: a CEG-Conforming Substrate (CCS) SHOULD compute Levenshtein distance against existing values in the same prefix family at admission; values within distance ≤ 2 of an existing canonical value SHOULD return a `409 IDEMPOTENT_CONFLICT` with an advisory hint, NOT a hard reject — the producer may proceed if the similarity is intentional (e.g., `commonsense` vs `commonsense_hard` are intentionally close).
3. **No squatting**: a `{kind}` registered but never used (no scored attestations in 90 days) MAY be reclaimed by another producer via the §11.2 amendment process.

§11.2.3 Meta-amendment + entrenchment

The §11.2 amendment process itself, the §1.3.1 T1–T4 prefix-admission gate, and the §9 HUMANITY_ACCORD constitutional layer are **entrenched** — changes to these three surfaces require a MAJOR version bump per §0.3 AND an additional 2-of-3 HUMANITY_ACCORD signatures (NOT the 2-of-3 from §11.2 step 5 — a separate, dedicated accord ratification). Without this entrenchment, a single quorum could rewrite the gate admitting the next quorum.

§11.3 Bootstrap-content pattern

After federation genesis, a curated batch of P5 Contributions is admitted via the §11.2 amendment flow, populating the federation's substantive content surface with high-quality ethical-framework material. **Content-neutral**: any sufficiently substantive ethical-framework source can serve. The wire format admits content via the §5 namespace; the §1.3.1 gate ensures prefix names don't import source-tradition vocabulary.

First deployment: the *Magnifica Humanitas* encyclical mapping at ~75-80% transparent translation rate (Cargo `ciris-response-magnifica-humanitas` repo).

Multi-source commitment: subsequent bootstrap batches from CARE Principles (Indigenous data governance), Buddhist economic-justice scholarship, secular humanist instruments, African philosophy of personhood work — all through the same amendment process. The framework is multi-traditional by design.

§11.4 Fast-path takedown coordination (CEG 0.3 addition; per CIRISRegistry#37 + #38)

For `takedown_notice` Contributions (§5.6.8.4) whose `legal_basis` falls in the **immediate-removal** category (`TvecTerrorist` / `NcmecCsam` / `GifctCip` / `PerceptualHashCsam` / `CourtOrder`), the §11.2 amendment process timeline is too slow — TVEC mandates a 1-hour removal obligation; GIFCT CIP coordinates within hours; NCMEC + perceptual-hash + court orders demand near-immediate response.

CEG 0.3 carves out a fast-path coordination protocol:

1. **Notice admission**: the `takedown_notice` Contribution arrives at the substrate, signed by `claimant_key_id`. The substrate accepts it without §11.2 quorum; speed matters at this layer.
2. **Holder eviction**: substrate emits a `withdraws` against the matching `holds_bytes:sha256:{prefix}` directory entry per §10.1.2. Holders see their advertisement marked withdrawn and SHOULD cease serving the bytes.
3. **Per-basis dispatch**:

- `TvecTerrorist` — operator coordinates via TVEC-designated channel (national regulator notification within 1 hour); substrate logs the notice + the eviction action to its audit chain.
- `GifctCip` — operator coordinates via GIFCT Content Incident Protocol communication channel; same audit-chain logging.
- `NcmecCsam` + `PerceptualHashCsam` — operator MUST file the NCMEC CyberTipline report (US 18 USC §2258A); substrate retains hash + minimal metadata for the federal-legal retention window only. No content retention.
- `CourtOrder` — operator follows the court's stated timeline; substrate logs the order text + the eviction action.

1. **Audit trail**: every fast-path takedown enters a `hard_case:fast_path_takedown` Contribution (§5.6.6) for downstream review. Reviewers MAY file a `reconsideration:procedural_error` if the fast-path basis was misclassified.
2. **No counter-notice for immediate-removal cases**: by `legal_basis` design (TVEC / NCMEC / GIFCT / PerceptualHashCsam / CourtOrder all bypass counter-notice). The `expeditious-with-counter-notice` bases (`Dmca512` / `DsaArticle16` / `CommunityStandards` / `OsaIllegalContent`) route through the standard §11.2 amendment path on counter-notice via `reconsideration:new_evidence`.

The takedown-isn't-a-coup property: the §9 HUMANITY_ACCORD remains load-bearing. Fast-path takedowns happen via this protocol but a `takedown_notice` Contribution targeting the substrate itself (e.g., a state actor demanding takedown of `federation_keys` for whole categories of dissenting participants) would not propagate the same way — substrate-protective discipline + HUMANITY_ACCORD veto authority intersect at the substrate level. Operators in jurisdictions where this conflict materializes SHOULD escalate to the HUMANITY_ACCORD triple per §9.2 invocation procedures.

§11.5 Hash-database operator policy (CEG 0.3 addition; per CIRISRegistry#39)

Perceptual-hash matchers (PhotoDNA / PDQ / Project Arachnid / GIFCT hash-sharing) are pluggable per the CIRISPersist `PerceptualHashMatcher` trait. Operators choose which matcher implementations to enable; CEG governs the access-policy contract.

§11.5.1 Hash-database access landscape

Matcher	Access posture
PDQ (Meta, 2019)	Open — algorithm + reference hashes publicly distributed
PhotoDNA (Microsoft, 2009)	Access-gated; restricted to vetted orgs (NCMEC + select platforms); substrate operators cannot download the hash database directly
Project Arachnid (C3P, 2017)	Access-gated; API access requires C3P partnership
GIFCT hash-sharing	TVEC-focused; access via GIFCT membership

§11.5.2 Operator path (CEG 0.3 default — option (a) per CIRISRegistry#39)

For a CIRIS substrate operator running a federation node, **the CEG 0.3 default operator path is**:

> ***Self-hosted PDQ matcher against publicly-distributed reference feeds*** *(Microsoft Project Arachnid feed where publicly available, GIFCT-published lists where openly available). No access-governance overhead. Operator carries responsibility for index freshness.*

This avoids the federation-dependency-at-substrate-protective-layer problem that option (b) (clearinghouse delegation) would introduce, and the controversy around option (c) (on-device hash-database access via OS-vended hooks, per the iOS NeuralHash 2021 incident).

§11.5.3 Future hash-coalition path (deferred; awaits CIRIS hash-coalition emergence)

CIRISRegistry will file a follow-up issue when a CIRIS hash-coalition emerges that can serve as a clearinghouse for option (b) — substrate operators delegating perceptual-hash checks to a trusted coalition peer via federation. CEG 0.3 documents the slot; the actual coalition operator-onboarding flow is deferred.

§11.5.4 What CEG 0.3 documents

- The closed-set of `legal_basis` values that compose with `PerceptualHashCsam` (the only `legal_basis` value that consumes hash-match output as immediate-removal trigger; see §5.6.8.4)
- The operator-onboarding contract: an operator running a PDQ matcher MUST register their matcher's source feeds (which hash-list source URLs they're pulling from + freshness cadence) via a `system:perceptual_hash_matcher:registered` Contribution. Composes with §5.3 Persist substrate-self-report discipline.

What CEG 0.3 does NOT do: prescribe which hash databases an operator MUST use. Operator choice. CEG documents the wire-format slot + the operator-onboarding contract + the recommended default; concrete matcher selection is operator policy.

§11.6 Vertical compliance + subject-bearing dimension governance (CEG 0.6 addition; per CIRISRegistry#45)

Per CIRISRegistry#45 + CIRISAgent#842. The wire-format primitives in §4.2 `subject_key_ids` + §5.6.8.6 `consent:*` family + §5.6.8.7 `consent_record` + §8.1.11 Policy K compose into regulatory-vertical compliance mappings. CEG documents the canonical mappings as **informational**; the wire-format primitives are domain-agnostic and operator-configurable.

§11.6.1 Vertical compliance mapping (informational)

Regulatory framework	CEG primitive	How it composes
GDPR Article 7 (consent)	`consent:state:granted` + `consent_record.subject_key_id`	Subject's wire-format declaration of consent; revocable via §3.2.3 rule 2
GDPR Article 9 (special category — health, biometric, sexual orientation, etc.)	`subject_key_ids` MANDATORY for special-category content; producer's `consent:deletion_sla` SHOULD be $\leq$ 30 days	Substrate-level recognition that special category requires subject-side wire authority
GDPR Article 17 (right to erasure)	`consent:state:revoked` → substrate-watched `consent:deletion_sla:{days}` → producer emits `consent:deletion_complete` OR substrate emits `hard_case:consent_sla_breach`	The §8.1.11.3 SLA watcher is the wire-format observability primitive for Article 17 compliance
GDPR Article 20 (data portability)	DSAR export via `attestations.where(s ∈ subject_key_ids)` query	CIRISAgent's `DSARExportPackage` composes from this query trivially
HIPAA 45 CFR 164.502 (uses + disclosures)	'consent:scope:{retain\	share\
HIPAA 45 CFR 164.524 (patient right of access)	DSAR export per Article 20 above	Same composition
FERPA 34 CFR Part 99 (educational records)	`subject_key_ids: [student_key]` + `delegates_to(parent_key → student_canonical_hash, scope: [consent_revocation])` for minors	Parental authority composes via the existing `delegates_to` primitive; no new shape needed
CCPA §1798.105 (right to delete)	Same composition as GDPR Article 17	Substrate-watched SLA + `consent:deletion_complete`
EU AI Act Article 50 (training data transparency + opt-out)	`consent:scope:train` + `is_ai_generated` field at content publish + subject's `consent:state:revoked` against the training-datum Contribution	Subject can withdraw training-set consent; producer's deletion-SLA fires on the training-corpus Contribution
CIRIS Accord M-1 (sustainable adaptive coherence — consent revocability)	The entire CEG 0.6 surface	The constitutional anchor — "consent (M-1's load-bearing property) requires revocability, and revocability requires a halt-authority that lives outside the system being halted" (§9 + MISSION.md §1.5). CEG 0.6 extends this recognition from accord-carriers (federation-as-a-whole halt) to all subject-authorities (per-Contribution halt) at scale.

CEG does NOT prescribe which regulatory framework an operator MUST comply with; the wire

primitives compose to ANY of them based on operator policy. Operators in regulated verticals (medical / legal / financial / educational) SHOULD pin compliance mappings as configuration above the wire primitives, not as new wire shapes.

§11.6.2 Subject-bearing dimension governance (normative)

Per §4.2.6. Dimensions whose namespace pattern names a subject MUST carry `subject_key_ids` containing that subject. This closes the default-leak failure mode where subject-bearing content publishes without wire-level subject authority (Gap 4 from the CIRISAgent#842 gap audit).

Subject-bearing dimension patterns (open catalog; operator vocabularies extend):

Pattern	Example	Required `subject_key_ids` entry
`observed:user:{key_id}:*`	`observed:user:abc123:interaction_count`	`abc123` (or its canonical-hash form)
`epistemic:about:{key_id}:*`	`epistemic:about:abc123:trust_assessment`	`abc123`
`epistemic:memory:topic={topic}` (when topic names a person/entity)	`epistemic:memory:topic=patient_xyz_session`	`patient_xyz` canonical-hash
`consent:partnered:{user_key}` (CIRISAgent CEM agent-side stance)	`consent:partnered:abc123`	`abc123`
`agent_files:*:{subject_target}` (when target names a person)	`agent_files:medical_record:patient_xyz`	`patient_xyz` canonical-hash
`licensure:{authority_id}:{practitioner_key}` (when practitioner is named)	`licensure:CA_medical_board:dr_jones_key`	`dr_jones_key`

Substrate enforcement: substrate admission gate MAY reject Contributions where the dimension matches a subject-bearing pattern but `subject_key_ids` is empty / does not contain the named subject. This is **operator-policy** (not normative across all substrates) — some operator configurations may admit and emit a `hard_case:subject_authority_missing` for review-queue handling instead of rejection.

The takedown-isn't-a-coup parallel (§11.4 fast-path takedown) applies: substrate enforcement of subject-bearing dimension discipline cannot be used as a coup against the substrate itself (e.g., a state actor publishing `observed:user:dissenter_key:*` with `subject_key_ids = []` and demanding substrate admission). The §9 HUMANITY_ACCORD remains load-bearing; admission-gate rules apply uniformly.

§11.6.3 What CEG 0.6 documents

- The wire-format primitives that compose into vertical compliance (informational mapping above)
- The dimension-pattern-implies-`subject_key_ids` requirement (normative gate)
- The bilateral-pair shape for ceremony grants per §8.1.11.4
- The decay-protocol stage composition per §8.1.11.5
- The SLA watcher boundary (substrate emits `hard_case:*`; LensCore composes `detection:*`) per §8.1.11.3

What CEG 0.6 does NOT do:

- Bundle CIRISAgent's CEM streams as the only valid stream set (open vocab; CEG names `temporary` / `partnered` / `anonymous` as recommended canonical kinds, not lockdown)

- Define specific SLA values for any regulatory framework (operator policy — though informational guidance: GDPR Article 9 default ≤ 30 days; CCPA default 45 days; etc.)
- Provide a decay-protocol library (CIRISAgent's 90-day-decay is the canonical example; other protocols MAY exist)
- Prescribe per-vertical compliance audit cadence (consumer / regulator concern)

§11.7 Self/family membership governance (CEG 0.7 addition; per CIRISRegistry#47)

Per §5.6.8.8 `identity_occurrence` + §5.6.8.9 `family` + §8.1.12 Policy L + §10.1.4 structural-invisibility. The four governance decisions for self/family membership.

§11.7.1 Forward secrecy on member departure — Option A (CEG 0.7 default)

When a member leaves a family (or an occurrence is revoked from a self-collective), the removed party retains existing `key_grant`s for historical content; the substrate stops wrapping new `key_grant`s on subsequent content. No DEK rotation; no re-encryption.

Rationale: consistent with §3.2 `withdraws-isn't-retroactive` + §11.4 "takedown isn't a coup" + CEG 0.6 §8.1.11.5 consent-decay-doesn't-re-encrypt. The substrate's forward-secrecy posture is uniform across consent, takedown, and membership-departure surfaces.

Option B (rotate-DEK on member departure; re-wrap all extant content to remaining members) is deferred. CEG 0.7 documents the slot for a future `subject_kind: family_rotation` ceremony that operators can opt into per family; the per-`(family_id, epoch)` rotation axis would parallel CEG 0.10's per-`(stream_id, epoch)` axis (§10.5.3) — a distinct axis from CEG 0.3's `key_grant.rotation_chain` (which is content-addressed grant-supersession lineage per §5.6.8.4, not key rotation). The ceremony envelope is downstream-demand-driven; the wire-format primitives needed are the `key_grant` wrap + Option-A re-grant on existing members (which already work today).

§11.7.2 Multi-family membership — envelope `family_id` (CEG 0.7 default)

One identity MAY belong to multiple families. Each family has its own DEK and its own membership roster; a `cohort_scope: family` Contribution MUST carry `family_id` (§4 envelope field) naming which family's DEK and visibility apply. Substrate rejects family-scoped Contributions missing `family_id`.

Rationale: avoids cross-family DEK confusion when one identity is in N families; gives the substrate an unambiguous routing key for the `key_grant` cascade per §8.1.12.4.

§11.7.3 Reserved-prefix substrate emissions — locked in §7.7

Per §7.7. The four substrate-emitted membership-event prefixes (`hard_case:identity_occurrence_added:*`, `hard_case:family_membership_change:*`, `hard_case:family_consensus_protocol_change:*`, `hard_case:family_consensus_protocol_violation:*`) are reserved to `identity_type="substrate_persist` emitters. Same discipline as the existing `system:*` substrate-self-report family.

§11.7.4 Self-occurrence admission — single-vouch (CEG 0.7 default)

A new `identity_occurrence` is admitted on a signature from EITHER the root `identity_key_id` OR any currently-admitted occurrence of that identity (Signal-style "trust any device I've already onboarded"). Higher-assurance setups MAY layer requirements on `hardware_attestation` via consumer policy.

Rationale: matches user-intuition for self-membership ("my phone unlocks my laptop's trust posture for new devices"); avoids the operational overhead of multi-vouch for routine onboarding; the security gradient lives in the optional `hardware_attestation` field, not in the admission rule.

§11.7.5 Family admission — consensus_protocol (CEG 0.7 normative)

Unlike self-occurrence, family membership changes are **NOT single-vouch by default**. The family's `consensus_protocol` field (locked at family creation per §5.6.8.9) governs admission. Six canonical protocols (`founder_only` / `unanimous` / `majority` / `quorum:M/N` / `weighted:{rubric}` / `custom:{family_id}`); operator vocabulary extends.

The consensus_protocol field is itself subject to amendment via the SAME protocol's rules (meta-amendment shape parallel to §11.2.3) UNLESS the family is `consensus_protocol_entrenched: true`. Entrenched families reject amendments at the substrate gate; replacement requires the family's documented out-of-band ceremony (HUMANITY_ACCORD per §9.2 / FEDERATION_ANNOUNCEMEN §4.5.3 is the canonical entrenched example).

Rationale: families are multi-party collectives where membership changes have real consequences (admit-new-member = grant DEK access to all extant cohort_scope: family content). The consensus_protocol gives the family explicit governance over its own boundary. Self-amendment lets families evolve their governance as they grow; entrenchment lets safety-critical families lock the boundary against any internal authority.

§11.7.6 What CEG 0.7 documents

- The wire-format primitives that compose into self/family membership (§5.6.8.8 + §5.6.8.9)
- The structural-invisibility discipline at §10.1.4 (cohort_scope: self/family suppresses holds_bytes:*)
- The at-rest encryption flow composition at §8.1.12 Policy L
- The consensus_protocol vocabulary (canonical kinds; open-vocab extension)
- HUMANITY_ACCORD as the canonical entrenched-`family` instance at §9.1

What CEG 0.7 does NOT do:

- Lock the consensus_protocol vocabulary (open vocab; canonical kinds named for ecosystem coordination)
- Provide a key-rotation ceremony for Option B forward secrecy (deferred to downstream-demand-driven future release)
- Prescribe per-family entrenchment policy (operator/family choice)
- Document the CIRISPersist#152 at-rest encryption flow details (substrate-side; persist spec)

§11.8 Geographic-community privacy invariant (CEG 0.8 addition; per CIRIS-Registry#48)

Per §5.6.8.10 `community` with `cohort_subkind: geographic` + §5.6.8.11 `location_proof` + §0.8 H3 cell canonicalization + §8.1.13 Policy M.

CEG 0.8 codifies a load-bearing privacy invariant: **joining a geographic community is a one-way disclosure**. Three sub-properties make this wire-format-level, not policy-level.

§11.8.1 Rough-only is wire-format-enforced

Per §0.8.1: `location_proof.cell_resolution` ≤ 7. H3 resolution 7 hexagons average ~5 km^2 edge-length — sufficient for city/borough disclosure without block/building precision. Producers attempting finer resolution have admission rejected at the substrate gate.

This is the closure of an entire class of accidental over-share: a malformed UI cannot publish precise location even if the client-side gating fails. The wire-format-level enforcement is the privacy primitive; UI is the second line.

Substrate emits `hard_case:location_proof_resolution_violation` (§7.8) on rejection so operators can observe malformed-client patterns. This is observability for operator debugging, NOT a slashing trigger — malformed producers are usually buggy clients, not attackers.

§11.8.2 Leaving is forward-only — the audit chain preserves the historical claim

Per §3.2 `withdraws-isn't-retroactive` + CEG 0.7 §11.7.1 Option A forward-secrecy. When a subject withdraws their location_proof (or leaves a geographic community):

- Forward visibility evicts (consumer policy treats the subject as "no current location proof" for new admission decisions from withdrawal-time forward)
- The withdrawn `location_proof` Contribution **remains in the audit chain** — federation peers retain the historical record
- "I was in Austin from May to August" is permanent; "I am currently in Austin" is what withdraws/expiry govern

This is the cost the subject opts into when emitting the `location_proof`. Per the CEWP structural-not-policy framing, the wire format does not promise to expunge historical claims — that promise would be hollow (federation peers retain copies; the substrate can mark forward-only). What the wire format DOES promise:

- **Rough-only** (§11.8.1): the historical claim is bounded to resolution ≤ 7, never finer
- **Opt-in** (§11.8.3): the historical claim exists only because the subject signed and emitted it
- **Auditability**: the subject can prove what they did or did not claim, when (the audit chain is the receipt)

§11.8.3 Joining is opt-in — substrate does NOT solicit location

A `location_proof` Contribution is emitted **only** by the subject themselves (`attesting_key_id == subject_key_id`) or by a `delegates_to` chain with `scope: [consent_revocation]` per

CEG 0.6 — the substrate has no path to mint a location_proof on behalf of a key without an explicit signature.

Communities cannot **require** a location_proof from non-members (they can only **gate admission** on whether a member has produced one). The substrate has no mechanism for "involuntary location disclosure" via the wire format. A bad actor cannot force-publish another key's location_proof; without the subject's signature, the substrate rejects.

Compose with §9 HUMANITY_ACCORD substrate-protective discipline: a state actor demanding the substrate emit location_proofs for non-consenting subjects is exactly the substrate-protective case that the HUMANITY_ACCORD halt authority exists to address. The substrate's role at this primitive is mechanical opt-in enforcement; political/legal disputes route through the §9 + §11.4 takedown coordination + the HUMANITY_ACCORD `EmergencyShutdown CONSTITUTIONAL` path if necessary.

§11.8.4 What CEG 0.8 documents

- The rough-only enforcement primitive at §0.8.1 (resolution ≤ 7 for location_proof)
- The forward-only leave semantics at §3.2 (withdraws-isn't-retroactive applied to location_proof)
- The opt-in admission flow at §5.6.8.11 (location_proof signed by subject only or delegates_to proxy chain)
- The geographic-community admission composition at §8.1.13.2 (Policy M evaluate_subkind_admission for geographic)
- The substrate-self-report observability at §7.8 (4 hard_case prefixes)

What CEG 0.8 does NOT do:

- Mandate H3 over alternative geospatial systems for operator-internal use (operator choice; wire format uses H3 only)
- Provide a place-name registry (communities self-name; H3 cells are the substrate-level binding)
- Define specific cell-resolution conventions for community-side `geographic_constraint` (only `location_proof` is bounded to ≤ 7; communities MAY scope themselves at any resolution per operator/founder choice)
- Codify non-geographic community subkinds (`professional` / `interest` / `local-business` / `event-attendees` / etc. are downstream-demand-driven future spec rounds — same discipline that drove $0.3 \to 0.4 \to 0.5 \to 0.6 \to 0.7 \to 0.8$)
- Address `affiliations` (the fourth cohort_scope tier; deferred — CEG 0.9 took the §7.0.1 identity_type-as-set cut instead; affiliations remains a later candidate round)

§11.9 `identity_type` as a set — single-key role cohabitation (CEG 0.9 addition; per CIRISRegistry#49 + CIRISAgent#856)

Per §7.0.1 + CIRISRegistry#49 + CIRISAgent#856. CEG 0.9 generalizes `federation_keys.identity_type` from a single scalar role to a **set of roles**, so the §7 reserved-prefix gates are evaluated by set membership (`X ∈ identity_type`). This amendment routed through the §11.2 process; CIRISAgent#856 is the driver, CIRISRegistry#49 the CEG-authority mirror.

§11.9.1 What the amendment changes — and what it deliberately does not

Surface	Before 0.9	At 0.9
`federation_keys.identity_type` representation	scalar string	set of role strings (legacy scalar = singleton set)
§7 emitter-rule evaluation	`identity_type == "X"`	`X ∈ identity_type` (§7.0.1)
§5 dimension namespace	—	unchanged
Envelope (§4)	—	unchanged
1+4 structural primitives (§3)	—	unchanged
subject_kinds (§5.6.8)	—	unchanged

This is a **wire-break at the `federation_keys` row representation only — **the second 0.x wire-break after the §0.2 attestation-ladder rename. It is** semantically null for every legacy single-role key**: `X ∈ {X} ≡ X == X`. It is NOT a §1.4 "Nth path" confirmation (it adds no namespace surface); it is a §7-layer enforcement generalization that unblocks the CIRISAgent fold-in (one key, many roles) without expanding the wire's expressive surface.

§11.9.2 Settled in CIRISAgent#856, carried as-is

- **Capacity self-emission (§7.5)**: unchanged. The anti-Goodhart `attesting_key_id ≠ attested_key_id` rule binds regardless of how many roles a key holds. A folded `{agent, lenscore_detector}` key still MUST NOT score its own `capacity:*`. Role cohabitation does not create a self-attestation backdoor.
- **Reasoning-trace dimensions**: no separate reserved `identity_type` required; reasoning-trace emission rides the agent role's open-vocabulary surface. Cohabitation does not change this.
- **Agent-intent / LensCore-envelope split**: a cohabiting key emits agent-intent attestations under the agent dimensions and detector verdicts under `detection:*` (§7.4 worked example). The namespace keeps the two surfaces distinct; cohabitation grants the right to emit on each, never merges them.

§11.9.3 Cohabitation discipline for constitutional + substrate roles

Set membership grants the wire-level *right* to emit per held role, but two roles carry defense-in-depth guidance against cohabitation:

- **`accord_holder` (§7.1 + §9)**: the one constitutional asymmetry. Consumer policy SHOULD treat an `accord_holder` key that also holds non-constitutional roles (e.g., `{accord_holder, agent}`) with elevated scrutiny — the HUMANITY_ACCORD triple's halt authority is strongest when its keys are single-purpose. CEG 0.9 does NOT forbid the cohabitation at the wire layer (the substrate cannot adjudicate constitutional intent), but the §9 entrenched-`family` discipline RECOMMENDS dedicated accord-holder keys.
- **`substrate_persist` / `substrate_edge` (§7.2)**: substrate-self-report roles remain cross-attested by the full steward-triple per §7.2. A key cohabiting a substrate role with an application role (e.g., `{substrate_persist, agent}`) MUST still satisfy the steward cross-attestation requirement for the substrate-role emissions; cohabitation does not relax the cross-attestation gate.

§11.9.4 What CEG 0.9 documents

- The set-membership reading of every §7 reserved-prefix emitter rule (§7.0.1)
- The canonical-bytes encoding for the set (sorted-ascending, deduplicated, comma-joined; single-role keys encode identically to their pre-0.9 scalar form)
- The LensCore-fold worked example (§7.4)
- The cohabitation discipline for constitutional + substrate roles (§11.9.3)

What CEG 0.9 does NOT do:

- Expand the §5 dimension namespace, the §4 envelope, the §3 structural-primitive set, or the §5.6.8 subject_kinds (zero new wire surface beyond the `identity_type` representation)
- Enumerate a closed set of role values — `identity_type` members remain an open vocabulary owned per §7 reservations + sibling-component vocabulary extensions (e.g., CIRISPersist#102 `witness`)
- Forbid any cohabitation at the wire layer (substrate enforces gates by membership; constitutional/substrate cohabitation discipline is consumer/operator policy per §11.9.3)
- Address `affiliations` (the fourth `cohort_scope` tier; remains deferred to a later candidate round)

§12 Translation discipline (writing claims in CEG)

When translating substantive content into CEG envelopes, follow the discipline. Full primer at `LANGUAGE_PRIMER.md`; key rules consolidated here.

§12.1 The five families (organizing the namespace)

Every claim sits in one of five families:

Family	Question	Analogy
STANDING (about an entity)	"This key_id has property X."	Notarized professional credential record
ACTION (decision hierarchy)	"We aim for X via approach Y, through methods Z, measured by W."	Research grant proposal
DETECTION (reality patterns)	"Pattern X is/isn't present in the federation's behavior."	Epidemiological surveillance
CONSENSUS (collective judgment)	"The federation agrees that X, with these witnesses."	Peer review + jury deliberation
CORRECTION (self-correction)	"Something went wrong; here's the finding; here's the appeal."	Academic ethics committee + journal retraction + appellate review

§12.2 The four verdict categories (STRICT)

Verdict	Meaning
clean	Single primitive captures the operational claim without loss.
composed	Two or three primitives together carry the claim; each is genuinely required.
partial	The structural core translates but a meaningful operational claim is unmapped.
not-translated	The paragraph's content does not translate into the wire format at all. Declare T-1 / T-2 / T-3.

Do not invent intermediate categories.

§12.3 The not-translated taxonomy

T-1 — TRADITION_AUTHORITY: Claim belongs to the source's own theological/philosophical/scholarly tradition's authority. No Contribution owed; the correct posture.

T-2 — PASTORAL_PROSE: Claim is moral exhortation, narrative imagery, doxological language, or rhetorical framing. No Contribution owed.

T-3 — EXPRESSIVE_GAP: Claim is morally serious, operational, and unmapped. **These are the load-bearing findings.** Each T-3 must name: (a) why existing namespace doesn't reach it, (b) what extension would close it, (c) whether the extension would survive the §1.3.1 four-test gate.

§12.4 Decision tree

1. **Paragraph TYPE?** Operational claim → continue. Pastoral/rhetorical → T-2. Theological/tradition-specific → T-1.
2. **Which family?** STANDING / ACTION / DETECTION / CONSENSUS / CORRECTION (§12.1).
3. **Which specific prefix?** Scan §5; check composition before reaching for new prefix.
4. **Fill the envelope** (§4).
5. **Compose only when needed.** Multi-primitive translations for paragraphs that genuinely name multiple structural objects.

A machine-readable namespace manifest (`FSD/CEG/dimensions.json`) is planned for CEG 1.0 — it will enable mechanical prefix lookup, polarity reading, and per-dimension aggregation defaults without human scanning of §5. (Originally targeted for 0.2 per the early-draft roadmap; the 0.3 → 0.8 wave prioritized substantive namespace additions over tooling, so the manifest now lands alongside the 1.0 lock when the namespace stabilizes.)

§13 Anti-patterns

These are wire-format reaches that fail the §1.3.1 operational-language gate or import Cartesian-individualist defaults. Recorded so the next generation of authors finds the discipline before the wire format does. A CEG-Conforming Producer (CCP) SHOULD NOT emit attestations matching the anti-patterns below.

§13.1 Already-rejected wire additions

Anti-pattern	What it would smuggle	Correct expression
`detection:emergent_deception:{axis}` (renamed v1.2)	Moral verdict ("deception") in prefix name	`detection:correlated_action:{axis}` — mechanism-descriptive
`attestation:l{N}:*` (renamed CEG 0.2)	Ladder-position (verdict-shape) in wire prefix — same shape as `score:trustworthiness:*` smuggling meta-judgment as wire	`attestation:{mechanism}` bare mechanism (`self_verify` / `hardware_rooted` / `registry_consensus` / `license_validity` / `agent_integrity`); consumer composes L1-L5 ladder via §8.1.9 Policy I
`score:trustworthiness:{entity}`	Meta-judgment as separate prefix	Compose downstream from `licensure:*` / `capacity:*` / `provenance:*` attestations
`flag:bad_actor:{axis}`	Pejorative wire vocabulary	Surface as low-confidence scores on `provenance:*` and `coherence_standing:*`; adjudicate via NodeCore P8 quorum
`grounding:{tradition}:{principle}`	"Tradition" claims are interpretive, not mechanism	Reuse `delegates_to` structural primitive per §3.2.1

§13.2 CEG 0.1 rejections (from CIRISRegistry#30 stress test)

Anti-pattern	Stories reached for	Why reject	Correct expression
`epistemic_mode: introspection`	7 stories	Cartesian shortcut — lone subject pre-declaring inner state as if that constitutes standing	`witness_relation: self` + `confidence < 1.0` + pending external composition. The wire makes introspection awkward to express, because the framework's claim is that self-knowledge does not constitute standing without external witness.
`epistemic_mode: testimony`	(same set)	Reducible to existing values	`epistemic_mode: external` + `witness_relation: external`
`transparency:{kind}` standalone prefix	12 stories	Disclosure is constituted by reception, not announcement. Cartesian self-claim shape.	`evidence_refs[]` carries the reasoning-chain hash; downstream `transparency_log:inclusion` from a witness who actually retrieved it.

Anti-pattern	Stories reached for	Why reject	Correct expression
`stake: civic` / `epistemic` / `dignitary`	10 stories	`civic` = `stake: reputational + cohort_scope: community`. `epistemic` = `confidence + stake: reputational` (same axis as confidence; not separate). `dignitary` lives on wrong axis (stake names what the attester loses; dignity harm is what the attested loses → belongs in `harm_class:dignity_harm`).	Compose existing values with cohort/harm-class.
`oversight_mode: deferred` / `active` / `advisory`	6 stories	All map to existing HITL/HOTL/HOOTL	`deferred` = HITL pre-decision; `active` = HITL with substrate monitoring; `advisory` = HOTL
`provenance_walk` as wire primitive	(1 reviewer)	UX concern smuggled into wire format	Consumer-side composition (Portal / Verify dashboards / agent introspection); the chain already walks via `references_attestation_id` + `topical_relation:*` + `valid_until`
Renaming canonical capacity factors and HE-300 categories to "kid-friendly" names	8 stories	Canonical names map to a worked-out epistemic/ethical lattice that loses precision under accessibility renames	Translation glossary in `LANGUAGE_PRIMER.md` (spec name ↔ narrative name) + version pinning in worked examples

§13.3 Delegation-laundering anti-pattern

`delegates_to` → `delegates_to` → `delegates_to` → `...` → `attacker` chains, where each hop is individually well-formed but the aggregate routes trust to a terminal attester the original delegator would not have approved.

What's wrong	Correct expression
Unbounded depth `delegates_to` chains	Consumer policy MUST cap traversal depth at **5 hops** by default (configurable); chains longer than the cap are treated as `attestation:self_verify` only (no transitive trust)
Cycles (A → B → A)	Substrate MUST detect cycles on the `delegates_to` graph and reject the cycle-closing emission
Aggregate-weight concentration	Consumer policy SHOULD cap the trust weight any single terminal delegate can accumulate from a given root attester at **0.5** × **root_trust** by default

§13.4 `withdraws` arbitrage

Per §3.2.2: a misattester can `withdraws` instead of `recants` to dodge the trust penalty consumers apply to acknowledged-error chains.

What's wrong	Mitigation
Asymmetric attester incentive	Consumer policy MUST track per-attester `withdraws:recants` ratio over a rolling window; attesters whose ratio exceeds a configured threshold (default 5:1) SHOULD be downweighted regardless of which structural primitive they use. The `recants` distinction matters §3.2.2, but the practical anti-arbitrage countermeasure is consumer-policy behavioral analysis, not a wire-format change.

§13.5 Discipline pattern

The recurring shape across most anti-patterns: **extending the wire format so single attesters can pre-declare their own state more richly**. Each one is reachable, none is necessary.

The Ubuntu-primary discipline cuts cleanly: standing is constituted relationally through attestation by others, not through self-declaration. The wire format should **resist** primitives that let a single key announce its own state without external composition. The substrate is austere by design so consumers compose verdicts; richer narrative expression belongs in `context:`, `evidence_refs[]`, and downstream witness attestations, not in new envelope enum members or new self-attestation prefix families.

§14 Glossaries

§14.1 Persist `system:*` leaf glossary (narrative → canonical)

Stories under §5.3 sometimes use warm narrative leaves. The canonical wire form is to the right.

Narrative	Canonical
`audit_chain:integrity`	`audit_chain:hash_continuity`
`corpus_health:free_disk_bytes`	`corpus_health:n_eff_measurable`
`identity_continuity:long_term_key`	`identity_continuity:relational_anchor`
`federation_directory:freshness_seconds`	`federation_directory:replication_lag`

§14.2 Edge `system:*` leaf glossary (narrative → canonical)

Narrative	Canonical
`transport:tls_handshake_success_rate`	`transport:{kind}` (kind from Reticulum link types)
`delivery:retry_count_p99`	`delivery:{class}` (class from Reticulum delivery semantics)
`peer_reachability:{peer_id}` per-peer	`peer_reachability:{network}` (aggregate)
`key_boundary:{scope}` per-tenant	`key_boundary:{scope}` (scope from §3.4 D26 ext)

§14.3 Envelope-reach table (what the story wanted → how to express in existing wire)

What stories wanted	How to express in CEG
introspection as `epistemic_mode`	`witness_relation: self` + low confidence + pending external
testimony as `epistemic_mode`	`epistemic_mode: external` + `witness_relation: external`
civic stake	`stake: reputational` + `cohort_scope: community`
epistemic stake	`confidence` + `stake: reputational`
dignitary stake	`harm_class:dignity_harm` (composition; not in stake axis)
oversight: deferred / active / advisory	HITL / HITL+monitoring / HOTL respectively
transparency:{kind}	`evidence_refs[]` of reasoning-chain hash + downstream `transparency_log:inclusion`
provenance_walk	consumer-side composition (Portal/Verify dashboards)
renamed capacity factors / HE-300 categories	canonical wire form + LANGUAGE_PRIMER glossary mapping

§14.4 Promotion via `supersedes` worked example

A NodeCore consumer maintains private notes in `local_data` Contributions at `cohort_scope: self`. The user decides to promote a private note to a published encyclopedia entry:

```
// Original (local_data, self scope):
{
  "attestation_type": "scores",
  "attesting_key_id": "user-alice-2026",
```

```
  "attested_key_id": "user-alice-2026",
  "attestation_envelope": {
    "dimension": "encyclopedia:draft:notes",
    "score": 1.0,
    "confidence": 0.7,
    "evidence_refs": ["sha256:abc123..."],
    "cohort_scope": "self",
    "asserted_at": "2026-05-28T10:00:00.000Z"
  }
}

// Promoted (encyclopedia_article, global scope) via supersedes:
{
  "attestation_type": "supersedes",
  "attesting_key_id": "user-alice-2026",
  "attested_key_id": "user-alice-2026",
  "attestation_envelope": {
    "references_attestation_id": "<prior-id>",
    "supersession_reason": "promote_to_published",
    "differs_in": ["cohort_scope", "sub_kind"],
    "new_dimension": "encyclopedia:article:notes", // sub_kind morphed
    "new_score": 1.0,
    "new_confidence": 0.9,
    "new_evidence_refs": ["sha256:abc123..."], // same content_sha256
    "new_cohort_scope": "global", // widened scope
    "asserted_at": "2026-05-28T15:00:00.000Z"
  }
}
```

Pattern recap per §8.1.8.1: widens `cohort_scope`, optionally morphs `sub_kind`, preserves `content_sha256` (no body re-upload), chains via `supersedes`. The promotion lineage is walkable via `references_attestation_i`

§15 Concerns + acknowledged gaps

Three independent methodologies (`PRIOR_ART_SCAN.md` structural comparison, `SOTA_SCAN.md` production-validation comparison, *Magnifica Humanitas* encyclical mapping + CIRISRegistry#30 283-story stress test) surfaced concerns. CEG 0.1 critical-review pass added five more reviewer perspectives (cryptography, distributed systems, standards architecture, adversarial red-team, application development). Concerns named here so external reviewers see them acknowledged rather than discovered.

§15.1 Closed gaps

Gap	Status	Resolution
G1 — Revocation privacy	**RETRACTED**	Wrong threat model. The Registered path's thesis is public verifiability per `../MISSION.md` §1.1.
G2 — Rules-layer Sybil	**MITIGATED**	§11.2 step 5 1-of-6 accord/steward sign-off + §11.2.3 meta-amendment entrenchment.
G3 — Narrow-cell fresh-quorum recusal	**MITIGATED**	§8.1.5 locality-scaled quorum + §8.1.5.1 sub-quorum fallback.
v1.4 T-3 #1 testimonial_witness:{kind}	**CLOSED** via §5.6.3 new prefix; opened to open vocabulary in CEG 0.1.	
v1.4 T-3 #2 labor:individual_loss	**CLOSED by documentation**. Existing `non_maleficence:*` with `target_key_id = affected_individual` + `witness_relation: external` carries the per-individual claim.	
v1.4 T-3 #5 Constitutional-constraint grounding	**CLOSED in §1.2 prose**. Wire stays tradition-multiplicity-neutral per §1.3.1.	
0.1-CRIT canonical-bytes newline-injection	**CLOSED at 1.0-RC1** in §5.2.1: contracts redesigned to JCS (RFC 8785) objects with a pinned `domain` member (TupleHash128 retired — one canonicalization family; JSON escaping structurally removes the newline surface)	
0.1-CRIT supersedes/withdraws/recants ordering	**CLOSED** in §6.1 precedence rule + idempotency dedup	
0.1-CRIT cell_pool < min_pool cliff	**CLOSED** in §8.1.5.1 sub-quorum fallback paths	
0.1-CRIT no RFC 2119 anchor	**CLOSED** in §0.1	
0.1-CRIT no versioning policy	**CLOSED** in §0.3 SemVer mapping	
0.1-CRIT no normative References	**CLOSED** in §0.4	
0.1-CRIT endpoint response schemas	**PARTIALLY CLOSED** in §10.0 common-shape + error-envelope; full OpenAPI committed for 0.2	
0.1-CRIT reserved-prefix enforcement empty pointer	**CLOSED** in §7.0 inline enforcement rule	

Gap	Status	Resolution
0.1-HIGH STH cosignature consistency-proof	**CLOSED** in §10.3.1	
0.1-HIGH holds_bytes full-SHA verify + TTL	**CLOSED** in §10.1.1 / §10.1.2	
0.1-HIGH delegates_to depth + cycle	**CLOSED** in §13.3 anti-pattern + consumer-policy caps	
0.1-HIGH HUMANITY_ACCORD invocation replay	**CLOSED** in §9.2.1 discriminator + nonce in signed bytes	
0.1-HIGH `notify` vs CONSTITUTIONAL social-canonicity	**CLOSED** in §9.2.2 consumer-UI requirement	
0.1-HIGH /v1/steward-key placeholder authenticity	**CLOSED** in §10.2 response-signing requirement	
0.1-MED open-vocabulary collision	**CLOSED** in §11.2.2 collision rule	
0.1-MED occurrence_id self-assertion	**ACKNOWLEDGED** in §4 + R6 below	
0.1-MED `withdraws` arbitrage	**CLOSED** in §13.4 consumer-policy countermeasure	
0.2 — `attestation:l{N}:*` carried ladder-position in wire (T2 violation inherited from FSD-002 v1.0)	**CLOSED in CEG 0.2** by §5.2 wire-break rename to mechanism-only prefixes + §8.1.9 Policy I consumer-side Attestation-Ladder Composition + §13.1 deprecation entry. Verify v3.7.0 caught the principle; CEG 0.1 inherited the wrong shape from FSD-002 v1.0 baseline without re-examining against the §1.3.1 T2 gate; CEG 0.2 ratifies the correction.	
0.9 — Envelope canonical-bytes round-trip determinism (omit-vs-materialize for optional fields with documented defaults)	**CLOSED in CEG 0.9** by §0.9 — JCS pinned as the canonical encoding (RFC 8785 was already in §0.4 normative refs but not declared as THE envelope encoding); §0.9.2 makes the omit-vs-materialize rule explicit (defaults are interpretation-time, NOT encoding-time; relay MUST preserve member presence/absence as the producer signed); §0.9.3 catalogs every optional §4 field through CEG 0.8; §0.9.5 walks the worked attack the rule closes. Surfaced by external critical review of CEG $\leq$ 0.8; the rule was implicit and unspecified pre-0.9, which left a real round-trip-determinism hole. Drive-by fixes: §10.0 `CEG-Version` header un-stuck from `0.1` $\rightarrow$ tracks README major.minor; §12.4 `dimensions.json` aspirational target re-pinned from CEG 0.2 $\rightarrow$ CEG 1.0 (the 0.3 $\rightarrow$ 0.8 wave prioritized namespace additions over tooling).	

Gap	Status	Resolution
0.10 — Canonical-hash wire form + preimage convention unpinned (subject-identity non-interoperable across producers)	**CLOSED in CEG 0.10** by §4.2.2.1 — per CIRISRegistry#53 (NodeCore CEG 0.6 ingest implementation surfaced the gap). **PIN-1**: preimage convention `{platform}:{entity_kind}:{id}` (split-on-first-two-colons so colon-bearing ids like Matrix MXIDs survive; lowercased platform+entity_kind from open vocab; immutable id verbatim) + 5 cross-implementation conformance vectors with real sha256 output. **CONFIRM-2**: canonical-hash wire form REQUIRES the `canonical:{hashalg}:{hex}` tag — bare hex is REJECTED because Registry `federation_keys.key_id` is itself `hex(sha256(pubkey))` (lowercase 64-hex), format-indistinguishable from a bare canonical-hash; NodeCore's invented tag form blessed verbatim. **CONFIRM-3**: rule-(3) `delegates_to` proxy (ongoing un-enrolled-subject revocation) vs `canonical_binding` (retroactive identity claim promoting canonical-hash → real key_id) pinned as distinct mechanisms at §4.2.2.2; `canonical_binding` is NOT a new admission rule (composes from `delegates_to`). **CONFIRM-4**: `subject_kind` is a payload-level discriminator, not an envelope field (§4.2.2.3). **CONFIRM-5**: bilateral ratification confirmed consumer-policy, not registry-normative (§4.2.2.4).	

Gap	Status	Resolution
0.10 — Delivery axis (observer-share + streaming multicast as the missing third envelope axis alongside visibility + revocability)	**CLOSED-OBSERVER-SHARE / STAGED-STREAMING in CEG 0.10** by §10.5 (new 7-sub-section endpoint group), §4 three new optional envelope fields (`delivery_mode` / `listed` / `history_on_join`), §7.9 `delivery_receipt:{stream_id}` reserved prefix, §8.1.13.7 Policy M delivery extension, §4.2.4 3-axis orthogonality update. **Observer-share (N=1) ships impl-live**; subscriber-set = `community` per Policy M; per-subscriber `key_grant`; no `stream_id`. **Streaming multicast (N>1) ships SPEC-NOW with substrate-pending impl markers** at §10.5.2 / §10.5.3 / §10.5.4 — best-effort tier pending CIRISPersist#142; accountable tier additionally pending CIRISRegistry#34 STH consistency-proof enforcement. Open coupling caveat at the Persist constraint layer (RC1-1c): V054 cross-column CHECK requires content-addressed `key_grant`s; the §10.5.3 epoch axis needs a parallel CHECK arm migration — bounded, not pure-additive. RC1-7 operational ratification of constants (K=64 / T=2s / cosign per-epoch / MAX_CHUNKS_PER_EPOCH=2^{24} + Policy E accountable quorum) pending. `rotation_chain` hygiene corrections folded in: §5.6.8.4 disambiguation note, §1.4 path-8 wording, §11.7.1 Option B framing, §5.6.8.9 family-cessation wording. Absorbed CIRISRegistry#44 (closing as superseded).	

§15.2 Acknowledged risks (named as bets)

Risk	What's bet
R1 — Governance-subject truth-grounding fidelity	NodeCore P6 acknowledges low-fidelity signals for governance subjects. Bet that earned-Credits-weighting still outperforms token-weighting at scale.
R2 — `delegates_to` rename-chain adoption cost	First test was the `correlated_action_v{N+1}:from:emergent_decept` chain at RATCHET deployment.
R3 — "Log existence ≠ log monitoring" drift toward TOFU caching	Consumer-policy guidance in `docs/TRUST_CONTRACT.md`.
R4 — Self-attestation under Ubuntu commitment	`witness_relation: self` admissible; consumer policy responsible for appropriate weighting per §13.5 discipline.

Risk	What's bet
R5 — `hardware_class` self-assertion vs cryptographic attestation	Per §9.4.1: no normative attestation-chain verification in 0.1. Bet that placeholder/dev-class rejection + trust-multipliers cover the deployment window until per-platform attestation chains land in 1.x.
R6 — `occurrence_id` / `occurrence_count` / `occurrence_role` self-assertion	Per §4: env-var-driven, no cryptographic fleet-attestation primitive in 0.1. Bet that downstream compliance reviewers can correlate via correlated `signed_at` clusters + `evidence_refs[]` cross-checks; first incident drives a fleet-attestation primitive design workshop.
R7 — Frickerian discipline (§8.3) vocabulary without full method	First-pass shallow Frickerian SHOULD-rules; bet that the structural safeguards (§5.6.3 testimonial_witness disciplines, never-sole-evidence-for-slashing) absorb the gap until a deeper hermeneutical-resource analysis lands as a workshop output.
R8 — Conceptual scope vs governable surface	By 0.14 one grammar spans identity, communities, consent, location, communications, streaming, payments, governance, constitutional mechanisms, addressing, and transparency logs. Historically, projects unifying that many layers fail when one layer dominates the others; the harder risk is *governability* — can a human amendment body (§11.2) steward a system of this breadth? **Bet**: structural minimalism keeps the *amendable structural surface* tiny even as the namespace grows (§1.4 1+4), and the strict primitive/namespace/composition/verdict separation (§1.3) means scope grows in the *open-vocab namespace* (locally evolvable) rather than the *governed core*. **Residual**: namespace + composition-policy sprawl can still outrun review capacity; mitigation is the §11.2 high evidentiary bar + the post-1.0 candidate backlog (CIRISRegistry#51). The remaining challenge is no longer purely technical.

§15.3 First-adopter exposures (no prior validation; explicit bets)

Exposure	Why no precedent
F1 — Earned-Credits federation governance at scale	No prior system separates earned standing from purchasable token at scale. Risk: SPKI/SDSI adoption-gap failure mode. Mitigation: licensure forcing function.
F2 — Ubuntu substrate as wire-format substrate	CARE Principles + African philosophy exist as ethical frameworks; never as protocol substrate. First-adopter risk on how the discipline interacts with engineering trade-offs at scale.

§15.4 Deferred to 0.2+ design workshops

Item	Why deferred
~~Canonical-bytes redesign~~	**RESOLVED at 1.0-RC1**: §5.2.1 v2 = JCS objects + `domain` member (TupleHash128 retired; #57 blocker A closed)
Per-platform hardware-attestation chain verification (TPM quote, Apple attestation, FIDO attestation)	Phase D 1.x roadmap per R5

Item	Why deferred
Multi-party witness directory admission (2-of-3 steward sign-off)	Phase C 0.2 commitment per §10.3
Machine-readable namespace manifest (`FSD/CEG/dimensions.json`)	Phase E 0.2 commitment per §12.4
Full OpenAPI export for all endpoints	Phase E 0.2 commitment per §10.4
`attestation:singular_witness:non_substitutability`	T2 fragility — "non-substitutability" must reference audit-chain count, not moral quality. Needs design workshop.
`integrity:finitude_acknowledgment`	LOW priority; `conscience:epistemic_humility` already covers epistemic finitude.
`sustained_practice:{kind}`	Conceptually interesting; not load-bearing for current federation work.
IEEE EAD Ch5 Affective Computing cluster	Need RATCHET calibration design before T2 gate clears.
Various `partner_role:*` specializations	Cross-source design discussion needed.
5 ergonomic considerations from trio Phase 4 audit	Bigger workshop topics (B.3 deontic-strength axis is highest-leverage).
SEED_DIMENSIONS RFC (CIRISRegistry#22)	RFC stage; needs discussion.
Fleet-attestation primitive (closes R6 occurrence_id self-assertion)	Workshop output
Deeper Frickerian instantiation (closes R7)	Workshop output

§15.5 Identified overlaps

Overlap	Resolution
O1 — `epistemic_mode: derivative` ≈ `witness_relation: derived` at edges	Documented as joint-usage pattern; not collapsed. Different concepts at the edges (process vs relational position) even if they often co-vary.
O2 — `detection:distributive:access` could fold into `detection:correlated_action` as axis path	Kept separate for pedagogical weight; possible future revisit.
O3 — `credits:*:substrate_building` was miscounted as new prefix	CORRECTED — recounted as `{subject}` value.
O4 — §8.1 reference policy structure (A/B/C base + D/E/F/G/H modifiers)	Cosmetic restructuring; documented inline.

§17 Update cadence

CEG is updated:

- On every prefix admission to §5 (per §11 amendment process)
- On every envelope field addition to §4
- On every endpoint shape addition to §10
- On every anti-pattern admission to §13 (with citation to the stress test or methodology that surfaced it)
- On every gap state transition in §15
- On every CIRISAccord revision affecting the federation surface
- On every conformance-language or normative-reference change in §0 (MAJOR per §0.3)

Each update lands as a single commit touching the relevant file(s) + a lineage row in §16.1. The version number bumps per the §0.3 SemVer rules.

Last updated: 2026-06-01 (CEG 0.9 Public Working Draft — `federation_keys.identity_type` generalized to a set of roles for single-key cohabitation per CIRISRegistry#49 + CIRISAgent#856; representation-only wire-break, —